TWO POINTERS

125. Valid Palindrome
167. Two Sum II - Input Array Is Sorted
344. Reverse String
345. Reverse Vowels of a String
11. Container With Most Water
15. 3Sum
16. 3Sum Closest
18. 4Sum
42. Trapping Rain Water
75. Sort Colors (Dutch Flag)
283. Move Zeroes
977. Squares of a Sorted Array
680. Valid Palindrome II

SLIDING WINDOW

3. Longest Substring Without Repeating Characters
76. Minimum Window Substring
209. Minimum Size Subarray Sum
424. Longest Repeating Character Replacement
239. Sliding Window Maximum
567. Permutation in String
438. Find All Anagrams in a String
904. Fruit Into Baskets
1004. Max Consecutive Ones III
1456. Maximum Number of Vowels in Substring
1493. Longest Subarray After Deleting One Element
930. Binary Subarrays With Sum

BINARY SEARCH

704. Binary Search
35. Search Insert Position
33. Search in Rotated Sorted Array
81. Search in Rotated Sorted Array II
153. Find Minimum in Rotated Sorted Array
154. Find Minimum in Rotated Sorted Array II
34. Find First and Last Position of Element
74. Search a 2D Matrix
240. Search a 2D Matrix II
162. Find Peak Element
278. First Bad Version
69. Sqrt(x)
875. Koko Eating Bananas

MONOTONIC STACK

739. Daily Temperatures
496. Next Greater Element I
503. Next Greater Element II
84. Largest Rectangle in Histogram
85. Maximal Rectangle
901. Online Stock Span
402. Remove K Digits
316. Remove Duplicate Letters
42. Trapping Rain Water
1944. Number of Visible People in a Queue

HEAPS

215. Kth Largest Element in an Array
703. Kth Largest Element in a Stream
23. Merge k Sorted Lists
295. Find Median from Data Stream
347. Top K Frequent Elements
692. Top K Frequent Words
253. Meeting Rooms II
973. K Closest Points to Origin
767. Reorganize String
621. Task Scheduler
373. Find K Pairs with Smallest Sums

GREEDY

121. Best Time to Buy and Sell Stock
122. Best Time to Buy and Sell Stock II
55. Jump Game
45. Jump Game II
134. Gas Station
135. Candy
53. Maximum Subarray (Kadane's)
860. Lemonade Change
455. Assign Cookies
406. Queue Reconstruction by Height
392. Is Subsequence
435. Non-overlapping Intervals
621. Task Scheduler
1710. Maximum Units on a Truck

HASH MAP

1. Two Sum
217. Contains Duplicate
49. Group Anagrams
347. Top K Frequent Elements
3. Longest Substring Without Repeating Characters
560. Subarray Sum Equals K
128. Longest Consecutive Sequence
438. Find All Anagrams in a String
76. Minimum Window Substring
149. Max Points on a Line

SORTING

912. Sort an Array
147. Insertion Sort List (Insertion Sort)
75. Sort Colors
215. Kth Largest Element in an Array
1051. Height Checker
1122. Relative Sort Array
347. Top K Frequent Elements (Bucket Sort)
164. Maximum Gap (Radix Sort)
969. Pancake Sorting (Pancake Sort)
56. Merge Intervals

BFS / DFS

104. Maximum Depth of Binary Tree
102. Binary Tree Level Order Traversal
226. Invert Binary Tree
103. Binary Tree Zigzag Level Order Traversal
199. Binary Tree Right Side View
105. Construct Binary Tree from Preorder and Inorder
113. Path Sum II
230. Kth Smallest Element in a BST
124. Binary Tree Maximum Path Sum
297. Serialize and Deserialize Binary Tree

STACK

20. Valid Parentheses
1047. Remove All Adjacent Duplicates In String
155. Min Stack
150. Evaluate Reverse Polish Notation
739. Daily Temperatures
503. Next Greater Element II
394. Decode String
901. Online Stock Span
84. Largest Rectangle in Histogram
85. Maximal Rectangle

TWO POINTERS

125. Valid Palindrome

A phrase is a **palindrome** if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string s, return true *if it is a **palindrome**, or* false *otherwise*.

Example 1:

Input: s = "A man, a plan, a canal: Panama"
Output: true
Explanation: "amanaplanacanalpanama" is a palindrome.

Example 2:

Input: s = "race a car"

Output: false
Explanation: "raceacar" is not a palindrome.

Example 3:

Input: s = " "
Output: true
Explanation: s is an empty string "" after removing non-alphanumeric characters.
Since an empty string reads the same forward and backward, it is a palindrome.

Approach 1: Two Pointers (Iterative)
Idea: Use left and right pointers, skip non-alphanumeric chars, compare lowercase chars.

```
def isPalindrome(s: str) -> bool:
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 2: Filter and Reverse
Idea: Filter alphanumeric chars, convert to lowercase, and compare with reversed string.

```
def isPalindrome(s: str) -> bool:
    filtered = ''.join(ch.lower() for ch in s if ch.isalnum())
    return filtered == filtered[::-1]
```

Time Complexity: O(n)
Space Complexity: O(n) for filtered string.

Approach 3: Recursive Two Pointers
Idea: Recursively check outermost valid chars.

```
def isPalindrome(s: str) -> bool:
    def helper(left, right):
        if left >= right:
            return True
        if not s[left].isalnum():
            return helper(left + 1, right)
        if not s[right].isalnum():
            return helper(left, right - 1)
        if s[left].lower() != s[right].lower():
            return False
        return helper(left + 1, right - 1)
    return helper(0, len(s) - 1)
```

Time Complexity: O(n)
Space Complexity: O(n) recursion stack.

---

167. Two Sum II - Input Array Is Sorted

Given a **1-indexed** array of integers numbers that is already ***sorted in non-decreasing order***, find two numbers such that they add up to a specific target number. Let these two numbers be numbers[$index_1$] and numbers[$index_2$] where 1 <= $index_1$ < $index_2$ <= numbers.length.

Return *the indices of the two numbers,* $index_1$ *and* $index_2$*, **added by one** as an integer array* [$index_1$, $index_2$] *of length 2.*

The tests are generated such that there is **exactly one solution**. You **may not** use the same element twice.

Your solution must use only constant extra space.

Example 1:

Input: numbers = [2,7,11,15], target = 9
Output: [1,2]
Explanation: The sum of 2 and 7 is 9. Therefore, $index_1$ = 1, $index_2$ = 2. We return [1, 2].

Example 2:

Input: numbers = [2,3,4], target = 6
Output: [1,3]
Explanation: The sum of 2 and 4 is 6. Therefore $index_1$ = 1, $index_2$ = 3. We return [1, 3].

Example 3:

Input: numbers = [-1,0], target = -1
Output: [1,2]
Explanation: The sum of -1 and 0 is -1. Therefore $index_1$ = 1, $index_2$ = 2. We return [1, 2].

Approach 1: Two Pointers (Optimal for sorted)
Idea: Start with smallest (left) and largest (right), adjust based on sum.

```
def twoSum(numbers, target):
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return [left + 1, right + 1]  # 1-indexed
        elif total < target:
            left += 1
        else:
            right -= 1
    return []
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 2: Binary Search
Idea: For each element, binary search for complement.

```
def twoSum(numbers, target):
    for i in range(len(numbers)):
        comp = target - numbers[i]
        lo, hi = i + 1, len(numbers) - 1
        while lo <= hi:
            mid = (lo + hi) // 2
            if numbers[mid] == comp:
                return [i + 1, mid + 1]
            elif numbers[mid] < comp:
                lo = mid + 1
            else:
                hi = mid - 1
    return []
```

Time Complexity: O(n log n)
Space Complexity: O(1)

Approach 3: Hash Map (Works for unsorted too)
Idea: Store seen numbers and their indices.

```
def twoSum(numbers, target):
    seen = {}
    for i, num in enumerate(numbers):
        complement = target - num
        if complement in seen:
            return [seen[complement] + 1, i + 1]
        seen[num] = i
    return []
```

Time Complexity: O(n)
Space Complexity: O(n)

344. Reverse String

Write a function that reverses a string. The input string is given as an array of characters s.

You must do this by modifying the input array in-place with O(1) extra memory.

Example 1:

Input: s = ["h","e","l","l","o"]
Output: ["o","l","l","e","h"]

Example 2:

Input: s = ["H","a","n","n","a","h"]
Output: ["h","a","n","n","a","H"]

Approach 1: Two Pointers (Iterative Swap)
Idea: Swap start and end, move inward.

```
def reverseString(s):
    left, right = 0, len(s) - 1
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 2: Recursive Swap
Idea: Recursively swap outermost chars.

```
def reverseString(s):
    def helper(left, right):
        if left < right:
            s[left], s[right] = s[right], s[left]
            helper(left + 1, right - 1)
    helper(0, len(s) - 1)
```

Time Complexity: O(n)
Space Complexity: O(n) recursion stack.

Approach 3: Built-in Reverse
Idea: Use 's reverse method.

```
def reverseString(s):
    s.reverse()
```

Time Complexity: O(n)
Space Complexity: O(1) in C (in-place).

Approach 4: Stack Simulation (Not in-place)
Idea: Pop from end and push to start (conceptual).

```
def reverseString(s):
    # This is not truly in-place for demonstration
    stack = s[:]
    for i in range(len(s)):
        s[i] = stack.pop()
```

Time Complexity: O(n)
Space Complexity: O(n)

---

345. Reverse Vowels of a String

Given a string s, reverse only all the vowels in the string and return it.

The vowels are 'a', 'e', 'i', 'o', and 'u', and they can appear in both lower and upper cases, more than once.

Example 1:

Input: s = "IceCreAm"

Output: "AceCreIm"

Explanation:

The vowels in s are ['I', 'e', 'e', 'A']. On reversing the vowels, s becomes "AceCreIm".

Example 2:

Input: s = "leetcode"

Output: "leotcede"

Approach 1: Two Pointers with List Conversion
Idea: Convert string to list, use two pointers to find vowels and swap them.

```
def reverseVowels(s: str) -> str:
    vowels = set('aeiouAEIOU')
    s_list = list(s)
    left, right = 0, len(s) - 1

    while left < right:
        # Find next vowel from left
        while left < right and s_list[left] not in vowels:
            left += 1
        # Find next vowel from right
        while left < right and s_list[right] not in vowels:
            right -= 1

        # Swap vowels
        s_list[left], s_list[right] = s_list[right], s_list[left]
        left += 1
        right -= 1

    return ''.join(s_list)
```

Time Complexity: O(n)
Space Complexity: O(n) for list conversion

Approach 2: Two Pointers with String Builder
Idea: Build result string while swapping vowels.

```
def reverseVowels(s: str) -> str:
    vowels = set('aeiouAEIOU')
    left, right = 0, len(s) - 1
    result = list(s)

    while left < right:
        while left < right and result[left] not in vowels:
            left += 1
        while left < right and result[right] not in vowels:
            right -= 1

        if left < right:
            result[left], result[right] = result[right], result[left]
            left += 1
            right -= 1

    return ''.join(result)
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 3: Collect and Replace
Idea: Collect vowels in order, then replace in reverse order.

```
def reverseVowels(s: str) -> str:
    vowels = [c for c in s if c in 'aeiouAEIOU']
    vowels_rev = vowels[::-1]
    result = []
    vowel_idx = 0

    for c in s:
        if c in 'aeiouAEIOU':
            result.append(vowels_rev[vowel_idx])
            vowel_idx += 1
        else:
            result.append(c)

    return ''.join(result)
```

Time Complexity: O(n)
Space Complexity: O(n)

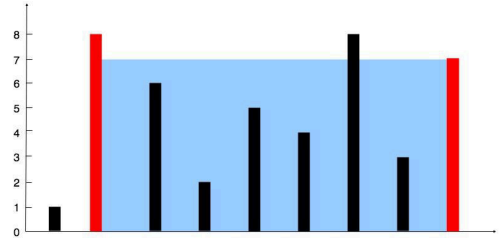
11. Container With Most Water

You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the i_{th} line are (i, 0) and (i, height[i]).
Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return *the maximum amount of water a container can store*.

Notice that you may not slant the container.

Example 1:



Input: height = [1,8,6,2,5,4,8,3,7]
Output: 49
Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

Example 2:

Input: height = [1,1]
Output: 1

Approach 1: Two Pointers (Optimal)
Idea: Start with widest container, move pointer with smaller height.

```
def maxArea(height):
    left, right = 0, len(height) - 1
    max_water = 0

    while left < right:
        width = right - left
        h = min(height[left], height[right])
        max_water = max(max_water, width * h)

        # Move the pointer with smaller height
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return max_water
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 2: Brute Force
Idea: Check all possible pairs.

```
def maxArea(height):
    max_water = 0
    n = len(height)

    for i in range(n):
        for j in range(i + 1, n):
            width = j - i
            h = min(height[i], height[j])
            max_water = max(max_water, width * h)

    return max_water
```

Time Complexity: $O(n^2)$
Space Complexity: O(1)

---

15. 3Sum

Given an integer array nums, return all the triplets [nums[i], nums[j], nums[k]] such that i != j, i != k, and j != k, and nums[i] + nums[j] + nums[k] == 0.
Notice that the solution set must not contain duplicate triplets.

Example 1:

Input: nums = [-1,0,1,2,-1,-4]
Output: [[-1,-1,2],[-1,0,1]]
Explanation:
nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.
nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.
nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.
The distinct triplets are [-1,0,1] and [-1,-1,2].
Notice that the order of the output and the order of the triplets does not matter.

Example 2:

Input: nums = [0,1,1]
Output: []
Explanation: The only possible triplet does not sum up to 0.

Example 3:

Input: nums = [0,0,0]
Output: [[0,0,0]]
Explanation: The only possible triplet sums up to 0.

Approach 1: Sorting + Two Pointers
Idea: Sort array, fix first element, use two pointers for remaining two.

```
def threeSum(nums):
    nums.sort()
    result = []
    n = len(nums)

    for i in range(n - 2):
        # Skip duplicates for first element
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        left, right = i + 1, n - 1

        while left < right:
            total = nums[i] + nums[left] + nums[right]

            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                result.append([nums[i], nums[left], nums[right]])

                # Skip duplicates for left pointer
                while left < right and nums[left] == nums[left +
1]:
                    left += 1
                # Skip duplicates for right pointer
                while left < right and nums[right] == nums[right -
1]:
                    right -= 1

                left += 1
                right -= 1

    return result
```

Time Complexity: O(n²)
Space Complexity: O(1) excluding output, O(k) for result where k = number of triplets

---

Approach 2: Hash Set
Idea: Use hash set to avoid duplicates and find complement.

```
def threeSum(nums):
    nums.sort()
    result = []
    n = len(nums)

    for i in range(n - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        seen = set()
        j = i + 1

        while j < n:
            complement = -nums[i] - nums[j]
            if complement in seen:
                result.append([nums[i], complement, nums[j]])
                # Skip duplicates
                while j + 1 < n and nums[j] == nums[j + 1]:
                    j += 1
            seen.add(nums[j])
            j += 1

    return result
```

Time Complexity: O(n²)
Space Complexity: O(n) for hash set

---

16. 3Sum Closest

Given an integer array nums of length n and an integer target, find three integers at **distinct indices** in nums such that the sum is closest to target.

Return *the sum of the three integers*.

You may assume that each input would have exactly one solution.

Example 1:

Input: nums = [-1,2,1,-4], target = 1
Output: 2
Explanation: The sum that is closest to the target is 2. (-1 + 2 + 1 = 2).

Example 2:

Input: nums = [0,0,0], target = 1
Output: 0
Explanation: The sum that is closest to the target is 0. (0 + 0 + 0 = 0).

Approach 1: Sorting + Two Pointers
Idea: Sort array, fix first element, use two pointers.

```
def threeSumClosest(nums, target):
    nums.sort()
    n = len(nums)
    closest_sum = float('inf')
    min_diff = float('inf')

    for i in range(n - 2):
        left, right = i + 1, n - 1

        while left < right:
            current_sum = nums[i] + nums[left] + nums[right]
            diff = abs(current_sum - target)

            if diff < min_diff:
                min_diff = diff
                closest_sum = current_sum

            if current_sum < target:
                left += 1
            elif current_sum > target:
                right -= 1
            else:
                return current_sum  # Exact match

    return closest_sum
```

Time Complexity: O(n²)
Space Complexity: O(1)

Approach 2: Brute Force
Idea: Check all triplets.

```
def threeSumClosest(nums, target):
    n = len(nums)
    closest_sum = float('inf')
    min_diff = float('inf')

    for i in range(n - 2):
        for j in range(i + 1, n - 1):
            for k in range(j + 1, n):
                current_sum = nums[i] + nums[j] + nums[k]
                diff = abs(current_sum - target)
                if diff < min_diff:
                    min_diff = diff
                    closest_sum = current_sum

    return closest_sum
```

Time Complexity: O(n³)
Space Complexity: O(1)

---

18. 4Sum

Given an array nums of n integers, return *an array of all the **unique** quadruplets* [nums[a], nums[b], nums[c], nums[d]] such that:

- 0 <= a, b, c, d < n
- a, b, c, and d are **distinct**.
- nums[a] + nums[b] + nums[c] + nums[d] == target

You may return the answer in **any order**.

Example 1:

Input: nums = [1,0,-1,0,-2,2], target = 0
Output: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]

Example 2:

Input: nums = [2,2,2,2,2], target = 8
Output: [[2,2,2,2]]

Approach 1: Generalizing 3Sum (Two Pointers)
Idea: Sort array, fix first two elements, use two pointers for remaining two.

```
def fourSum(nums, target):
    nums.sort()
    n = len(nums)
    result = []

    for i in range(n - 3):
        # Skip duplicates for first element
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        for j in range(i + 1, n - 2):
            # Skip duplicates for second element
            if j > i + 1 and nums[j] == nums[j - 1]:
                continue

            left, right = j + 1, n - 1

            while left < right:
                total = nums[i] + nums[j] + nums[left] +
nums[right]

                if total < target:
                    left += 1
                elif total > target:
                    right -= 1
                else:
                    result.append([nums[i], nums[j], nums[left],
nums[right]])

                    # Skip duplicates for left pointer
                    while left < right and nums[left] == nums[left + 1]:
                        left += 1
                    # Skip duplicates for right pointer
                    while left < right and nums[right] == nums[right -
1]:
                        right -= 1

                    left += 1
                    right -= 1

    return result
```

Time Complexity: O(n³)
Space Complexity: O(1) excluding output

Approach 2: Hash Map (for general k-Sum)
Idea: Store pairs sum in hash map.

```
def fourSum(nums, target):
    nums.sort()
    n = len(nums)
    result = []

    for i in range(n - 3):
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        for j in range(i + 1, n - 2):
            if j > i + 1 and nums[j] == nums[j - 1]:
                continue

            seen = set()
            for k in range(j + 1, n):
                complement = target - (nums[i] + nums[j] + nums[k])
                if complement in seen:
                    result.append([nums[i], nums[j], complement, nums[k]])
                    # Skip duplicates
                    while k + 1 < n and nums[k] == nums[k + 1]:
                        k += 1
                seen.add(nums[k])

    return result
```

Time Complexity: O(n³)
Space Complexity: O(n)

Approach 3: General k-Sum Recursive Solution
Idea: Recursive approach that works for any k.

```
def fourSum(nums, target):
    def kSum(nums, target, k):
        nums.sort()
        result = []

        def backtrack(start, path, target, k):
            if k == 2:
                left, right = start, len(nums) - 1
                while left < right:
                    total = nums[left] + nums[right]
                    if total < target:
                        left += 1
                    elif total > target:
                        right -= 1
                    else:
                        result.append(path + [nums[left], nums[right]])
                        left += 1
                        while left < right and nums[left] == nums[left - 1]:
                            left += 1
                        right -= 1
            else:
                for i in range(start, len(nums) - k + 1):
                    if i > start and nums[i] == nums[i - 1]:
                        continue
                    backtrack(i + 1, path + [nums[i]], target - nums[i], k - 1)

        backtrack(0, [], target, k)
        return result

    return kSum(nums, target, 4)
```

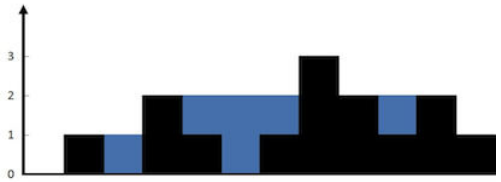
Time Complexity: O(n³) for 4Sum, O(n^(k-1)) in general
Space Complexity: O(n) for recursion stack

---

42. Trapping Rain Water

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:



Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:

Input: height = [4,2,0,3,2,5]
Output: 9

Approach 1: Two Pointers (Optimal)
Idea: Track max from left and right, water at position = min(max_left, max_right) - height[i]

```
def trap(height):
    if not height:
        return 0

    left, right = 0, len(height) - 1
    left_max = right_max = 0
    water = 0

    while left < right:
        if height[left] < height[right]:
            if height[left] >= left_max:
                left_max = height[left]
            else:
                water += left_max - height[left]
            left += 1
        else:
            if height[right] >= right_max:
                right_max = height[right]
            else:
                water += right_max - height[right]
            right -= 1

    return water
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 2: Dynamic Programming (Precompute Max)
Idea: Precompute max height from left and right for each position.

```
def trap(height):
    n = len(height)
    if n == 0:
        return 0

    left_max = [0] * n
    right_max = [0] * n

    # Fill left_max
    left_max[0] = height[0]
    for i in range(1, n):
        left_max[i] = max(left_max[i-1], height[i])

    # Fill right_max
    right_max[n-1] = height[n-1]
    for i in range(n-2, -1, -1):
        right_max[i] = max(right_max[i+1], height[i])

    # Calculate water
    water = 0
    for i in range(n):
        water += min(left_max[i], right_max[i]) - height[i]

    return water
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 3: Using Stack
Idea: Use stack to track bars in decreasing order.

```
def trap(height):
    stack = []
    water = 0
    i = 0

    while i < len(height):
        while stack and height[i] > height[stack[-1]]:
            top = stack.pop()
            if not stack:
                break
            distance = i - stack[-1] - 1
            bounded_height = min(height[i], height[stack[-1]]) - height[top]
            water += distance * bounded_height
        stack.append(i)
        i += 1

    return water
```

Time Complexity: O(n)
Space Complexity: O(n)

---

75. Sort Colors (Dutch National Flag)

Given an array nums with n objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

Input: nums = [2,0,2,1,1,0]
Output: [0,0,1,1,2,2]

Example 2:

Input: nums = [2,0,1]
Output: [0,1,2]

Approach 1: Two Pointers (Dutch Partition)
Idea: Use three pointers: low, mid, high.

```
def sortColors(nums):
    low, mid, high = 0, 0, len(nums) - 1

    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:  # nums[mid] == 2
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 2: Counting Sort
Idea: Count frequencies and overwrite array.

```
def sortColors(nums):
    count = [0, 0, 0]

    # Count frequencies
    for num in nums:
        count[num] += 1

    # Overwrite array
    idx = 0
    for color in range(3):
        for _ in range(count[color]):
            nums[idx] = color
            idx += 1
```

Time Complexity: O(n)
Space Complexity: O(1) for count array (size 3)

Approach 3: Two Pass (Partition Twice)
Idea: First move 0s to front, then move 1s after 0s.

```
def sortColors(nums):
    # First pass: move all 0s to the beginning
    zero_ptr = 0
    for i in range(len(nums)):
        if nums[i] == 0:
            nums[i], nums[zero_ptr] = nums[zero_ptr], nums[i]
            zero_ptr += 1

    # Second pass: move all 1s after 0s
    one_ptr = zero_ptr
    for i in range(zero_ptr, len(nums)):
        if nums[i] == 1:
            nums[i], nums[one_ptr] = nums[one_ptr], nums[i]
            one_ptr += 1
```

Time Complexity: O(n)
Space Complexity: O(1)

---

283. Move Zeroes

Given an integer array nums, move all 0's to the end of it while maintaining the relative order of the non-zero elements.
Note that you must do this in-place without making a copy of the array.

Example 1:

Input: nums = [0,1,0,3,12]
Output: [1,3,12,0,0]

Example 2:

Input: nums = [0]
Output: [0]

Approach 1: Two Pointers (Optimal)
Idea: Use slow pointer to track position for next non-zero element.

```
def moveZeroes(nums):
    slow = 0

    # Move all non-zero elements to the front
    for fast in range(len(nums)):
        if nums[fast] != 0:
            nums[slow] = nums[fast]
            slow += 1

    # Fill remaining positions with zeros
    for i in range(slow, len(nums)):
        nums[i] = 0
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 2: Two Pointers with Swap
Idea: Swap non-zero elements with zero positions.

```
def moveZeroes(nums):
    slow = 0

    for fast in range(len(nums)):
        if nums[fast] != 0:
            nums[slow], nums[fast] = nums[fast], nums[slow]
            slow += 1
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 3: ic (Not In-Place)
Idea: Use list comprehension.

```
def moveZeroes(nums):
    non_zeros = [x for x in nums if x != 0]
    zeros_count = len(nums) - len(non_zeros)
    nums[:] = non_zeros + [0] * zeros_count
```

Time Complexity: O(n)
Space Complexity: O(n)

---

977. Squares of a Sorted Array

Given an integer array nums sorted in **non-decreasing** order, return *an array of **the squares of each number** sorted in non-decreasing order.*

Example 1:

Input: nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: After squaring, the array becomes [16,1,0,9,100].
After sorting, it becomes [0,1,9,16,100].

Example 2:

Input: nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]

Approach 1: Two Pointers from Ends
Idea: Compare squares from both ends, fill result from back to front.

```
def sortedSquares(nums):
    n = len(nums)
    result = [0] * n
    left, right = 0, n - 1
    pos = n - 1  # position to fill in result

    while left <= right:
        left_sq = nums[left]  2
        right_sq = nums[right]  2

        if left_sq > right_sq:
            result[pos] = left_sq
            left += 1
        else:
            result[pos] = right_sq
            right -= 1
        pos -= 1

    return result
```

Time Complexity: O(n)
Space Complexity: O(n) for result

Approach 2: Sort After Squaring
Idea: Square all elements then sort.

```
def sortedSquares(nums):
    return sorted(x * x for x in nums)
```

Time Complexity: O(n log n)
Space Complexity: O(n) for sorting

Approach 3: Two Pointers from Middle
Idea: Find first non-negative index, then merge.

```
def sortedSquares(nums):
    n = len(nums)
    # Find first non-negative index
    first_non_neg = 0
    while first_non_neg < n and nums[first_non_neg] < 0:
        first_non_neg += 1

    # Negative part (in reverse) and positive part
    neg_ptr = first_non_neg - 1
    pos_ptr = first_non_neg
    result = []

    while neg_ptr >= 0 and pos_ptr < n:
        neg_sq = nums[neg_ptr]  2
        pos_sq = nums[pos_ptr]  2

        if neg_sq < pos_sq:
            result.append(neg_sq)
            neg_ptr -= 1
        else:
            result.append(pos_sq)
            pos_ptr += 1

    # Add remaining negatives
    while neg_ptr >= 0:
        result.append(nums[neg_ptr]  2)
        neg_ptr -= 1

    # Add remaining positives
    while pos_ptr < n:
        result.append(nums[pos_ptr]  2)
        pos_ptr += 1

    return result
```

Time Complexity: O(n)
Space Complexity: O(n)

680. Valid Palindrome II

Given a string s, return true *if the* s *can be palindrome after deleting* ***at most one*** *character from it.*

Example 1:

Input: s = "aba"
Output: true

Example 2:

Input: s = "abca"
Output: true
Explanation: You could delete the character 'c'.

Example 3:

Input: s = "abc"
Output: false

Approach 1: Two Pointers with Helper Function
Idea: Check palindrome normally, if mismatch, check two possibilities.

```
def validPalindrome(s):
    def is_palindrome(left, right):
        while left < right:
            if s[left] != s[right]:
                return False
            left += 1
            right -= 1
        return True

    left, right = 0, len(s) - 1

    while left < right:
        if s[left] != s[right]:
            # Try deleting either left or right character
            return is_palindrome(left + 1, right) or
is_palindrome(left, right - 1)
        left += 1
        right -= 1

    return True
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 2: Recursive Solution
Idea: Recursively check with deletion allowance.

Time Complexity: O(n)
Space Complexity: O(n) recursion stack

```
def validPalindrome(s):
    def check(left, right, deleted):
        if left >= right:
            return True
        if s[left] == s[right]:
            return check(left + 1, right - 1, deleted)
        elif not deleted:
            return check(left + 1, right, True) or check(left, right
- 1, True)
        else:
            return False

    return check(0, len(s) - 1, False)
```

Approach 3: Iterative with Skip Option
Idea: Track when we've used our skip.

```
def validPalindrome(s):
    left, right = 0, len(s) - 1
    skipped = False

    while left < right:
        if s[left] != s[right]:
            # Already used skip, check two possibilities
            if not skipped:
                # Check if skipping left works
                l1, r1 = left + 1, right
                valid1 = True
                while l1 < r1:
                    if s[l1] != s[r1]:
                        valid1 = False
                        break
                    l1 += 1
                    r1 -= 1

                # Check if skipping right works
                l2, r2 = left, right - 1
                valid2 = True
                while l2 < r2:
                    if s[l2] != s[r2]:
                        valid2 = False
                        break
                    l2 += 1
                    r2 -= 1

                return valid1 or valid2
            else:
                return False
        left += 1
        right -= 1

    return True
```

Time Complexity: O(n)
Space Complexity: O(1)

SLIDING WINDOW

3. Longest Substring Without Repeating Characters

Given a string s, find the length of the **longest** **substring** without duplicate characters.

Example 1:

Input: s = "abcabcbb"
Output: 3
Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: s = "bbbbb"
Output: 1
Explanation: The answer is "b", with the length of 1.

Example 3:

Input: s = "pwwkew"
Output: 3
Explanation: The answer is "wke", with the length of 3. Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Approach 1: Sliding Window with Hash Set

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        char_set = set()
        left = 0
        max_length = 0

        for right in range(len(s)):
            while s[right] in char_set:
                char_set.remove(s[left])
                left += 1
            char_set.add(s[right])
            max_length = max(max_length, right - left + 1)

        return max_length
```

Time Complexity: O(n) - Each character visited at most twice
Space Complexity: O(min(n, m)) - m = character set size (256 for ASCII)

Approach 2: Sliding Window with Hash Map (Optimized)

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        char_index = {}
        left = 0
        max_length = 0

        for right in range(len(s)):
            if s[right] in char_index and char_index[s[right]] >=
left:
                left = char_index[s[right]] + 1
            char_index[s[right]] = right
            max_length = max(max_length, right - left + 1)

        return max_length
```

Time Complexity: O(n) - Single pass
Space Complexity: O(min(n, m))

Approach 3: Using Array for ASCII Characters

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        last_index = [-1] * 128  # For ASCII characters
        left = 0
        max_length = 0

        for right in range(len(s)):
            ascii_val = ord(s[right])
            if last_index[ascii_val] >= left:
                left = last_index[ascii_val] + 1
            last_index[ascii_val] = right
            max_length = max(max_length, right - left + 1)

        return max_length
```

Time Complexity: O(n)
Space Complexity: O(1) - Fixed size array of 128

---

76. Minimum Window Substring

Given two strings s and t of lengths m and n respectively, return *the* ***minimum window*** ***substring*** *of* s *such that every character in* t *(****including duplicates****) is included in the window*. If there is no such substring, return *the empty string* "".

The testcases will be generated such that the answer is **unique**.

Example 1:

Input: s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"

Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string t.

Example 2:

Input: s = "a", t = "a"
Output: "a"
Explanation: The entire string s is the minimum window.

Example 3:

Input: s = "a", t = "aa"
Output: ""
Explanation: Both 'a's from t must be included in the window.
Since the largest window of s only has one 'a', return empty string.

Approach 1: Sliding Window with Two Pointers

```
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        if not s or not t or len(s) < len(t):
            return ""

        t_count = Counter(t)
        required = len(t_count)
        formed = 0
        window_count = {}

        left = 0
        min_len = float('inf')
        min_left = 0

        for right in range(len(s)):
            char = s[right]
            window_count[char] = window_count.get(char, 0) +
1

            if char in t_count and window_count[char] ==
t_count[char]:
                formed += 1

            while left <= right and formed == required:
                if right - left + 1 < min_len:
                    min_len = right - left + 1
                    min_left = left

                left_char = s[left]
                window_count[left_char] -= 1
                if left_char in t_count and
window_count[left_char] < t_count[left_char]:
                    formed -= 1
                left += 1

        return "" if min_len == float('inf') else
s[min_left:min_left + min_len]
```

Time Complexity: O(|S| + |T|)
Space Complexity: O(|S| + |T|)

Approach 2: Optimized with Filtered S

```
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        if not s or not t:
            return ""

        t_count = Counter(t)
        required = len(t_count)

        # Filter s to only include characters in t
        filtered_s = []
        for i, char in enumerate(s):
            if char in t_count:
                filtered_s.append((i, char))

        left = 0
        formed = 0
        window_count = {}
        result = (float('inf'), None, None)

        for right in range(len(filtered_s)):
            char = filtered_s[right][1]
            window_count[char] = window_count.get(char, 0) +
1

            if window_count[char] == t_count[char]:
                formed += 1

            while left <= right and formed == required:
                start = filtered_s[left][0]
                end = filtered_s[right][0]

                if end - start + 1 < result[0]:
                    result = (end - start + 1, start, end)

                left_char = filtered_s[left][1]
                window_count[left_char] -= 1
                if window_count[left_char] < t_count[left_char]:
                    formed -= 1
                left += 1

        return "" if result[0] == float('inf') else
s[result[1]:result[2] + 1]
```

Time Complexity: O(|S| + |T|)
Space Complexity: O(|S| + |T|)

209. Minimum Size Subarray Sum

Given an array of positive integers nums and a positive integer target, return *the* ***minimal length*** *of a subarray whose sum is greater than or equal to* target. If there is no such subarray, return 0 instead.

Example 1:

Input: target = 7, nums = [2,3,1,2,4,3]
Output: 2
Explanation: The subarray [4,3] has the minimal length under the problem constraint.

Example 2:

Input: target = 4, nums = [1,4,4]
Output: 1

Example 3:

Input: target = 11, nums = [1,1,1,1,1,1,1,1]
Output: 0

Approach 1: Sliding Window

```
class Solution:
    def minSubArrayLen(self, target: int, nums: List[int]) ->
int:
        left = 0
        current_sum = 0
        min_length = float('inf')

        for right in range(len(nums)):
            current_sum += nums[right]

            while current_sum >= target:
                min_length = min(min_length, right - left + 1)
                current_sum -= nums[left]
                left += 1

        return 0 if min_length == float('inf') else min_length
```

Time Complexity: O(n) - Each element visited at most twice
Space Complexity: O(1)

Approach 2: Binary Search on Prefix Sum

```
class Solution:
    def minSubArrayLen(self, target: int, nums: List[int]) ->
int:
        n = len(nums)
        if n == 0:
            return 0

        # Build prefix sum array
        prefix_sum = [0] * (n + 1)
        for i in range(n):
            prefix_sum[i + 1] = prefix_sum[i] + nums[i]

        min_length = float('inf')

        for i in range(n):
            # Find the smallest end such that prefix_sum[end]
- prefix_sum[i] >= target
            left, right = i + 1, n
            while left <= right:
                mid = left + (right - left) // 2
                if prefix_sum[mid] - prefix_sum[i] >= target:
                    min_length = min(min_length, mid - i)
                    right = mid - 1
                else:
                    left = mid + 1

        return 0 if min_length == float('inf') else min_length
```

Time Complexity: O(n log n) - Binary search for each starting index
Space Complexity: O(n) - Prefix sum array

Approach 3: Two Pointers with Early Exit

```
class Solution:
    def minSubArrayLen(self, target: int, nums: List[int]) ->
int:
        n = len(nums)
        if sum(nums) < target:
            return 0

        left = 0
        current_sum = 0
        min_length = n  # Initialize with max possible

        for right in range(n):
            current_sum += nums[right]

            # Try to shrink window as much as possible
            while current_sum - nums[left] >= target:
                current_sum -= nums[left]
                left += 1

            if current_sum >= target:
                min_length = min(min_length, right - left + 1)

        return min_length
```

Time Complexity: O(n)
Space Complexity: O(1)

424. Longest Repeating Character Replacement

You are given a string s and an integer k. You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most k times.

Return *the length of the longest substring containing the same letter you can get after performing the above operations.*

Example 1:

Input: s = "ABAB", k = 2
Output: 4
Explanation: Replace the two 'A's with two 'B's or vice versa.

Example 2:

Input: s = "AABABBA", k = 1
Output: 4
Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA".
The substring "BBBB" has the longest repeating letters, which is 4.
There may exists other ways to achieve this answer too.

Approach 1: Sliding Window with Frequency Count

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        char_count = [0] * 26
        max_count = 0
        left = 0
        max_length = 0

        for right in range(len(s)):
            char_count[ord(s[right]) - ord('A')] += 1
            max_count = max(max_count,
char_count[ord(s[right]) - ord('A')])

            # If window size - max_count > k, we need to
shrink
            while (right - left + 1) - max_count > k:
                char_count[ord(s[left]) - ord('A')] -= 1
                left += 1

            max_length = max(max_length, right - left + 1)

        return max_length
```

Time Complexity: O(n) - Each character visited at most twice
Space Complexity: O(1) - Fixed size array of 26

Approach 2: Optimized Sliding Window

Time Complexity: O(n)
Space Complexity: O(1)

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        char_count = [0] * 26
        max_count = 0
        left = 0

        for right in range(len(s)):
            char_count[ord(s[right]) - ord('A')] += 1
            max_count = max(max_count,
char_count[ord(s[right]) - ord('A')])

            # Only move left pointer if current window is invalid
            if (right - left + 1) - max_count > k:
                char_count[ord(s[left]) - ord('A')] -= 1
                left += 1

        return len(s) - left  # Maximum window size
```

Approach 3: Binary Search on Window Size

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        def canMakeValid(window_size):
            char_count = [0] * 26
            max_count = 0
            left = 0

            for right in range(len(s)):
                char_count[ord(s[right]) - ord('A')] += 1
                if right - left + 1 > window_size:
                    char_count[ord(s[left]) - ord('A')] -= 1
                    left += 1

                max_count = max(max_count,
char_count[ord(s[right]) - ord('A')])

                if right >= window_size - 1:
                    if window_size - max_count <= k:
                        return True

            return False

        left, right = 1, len(s)
        result = 1

        while left <= right:
            mid = left + (right - left) // 2
            if canMakeValid(mid):
                result = mid
                left = mid + 1
            else:
                right = mid - 1

        return result
```

Time Complexity: O(n log n)
Space Complexity: O(1)

239. Sliding Window Maximum

You are given an array of integers nums, there is a sliding window of size k which is moving from the very left of the array to the very right. You can only see the k numbers in the window. Each time the sliding window moves right by one position.

Return *the max sliding window.*

Example 1:

Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]
Explanation:

Window position	Max

[1 3 -1] -3 5 3 6 7	**3**
1 [3 -1 -3] 5 3 6 7	**3**
1 3 [-1 -3 5] 3 6 7	**5**
1 3 -1 [-3 5 3] 6 7	**5**
1 3 -1 -3 [5 3 6] 7	**6**
1 3 -1 -3 5 [3 6 7]	**7**

Example 2:

Input: nums = [1], k = 1
Output: [1]

Approach 1: Deque (Monotonic Queue)

```
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) ->
List[int]:
        if not nums:
            return []

        deq = deque()
        result = []

        for i in range(len(nums)):
            # Remove indices outside of window
            if deq and deq[0] < i - k + 1:
                deq.popleft()

    # Remove elements smaller than current from the back
            while deq and nums[deq[-1]] < nums[i]:
                deq.pop()

            deq.append(i)

            # Add to result when window size is reached
            if i >= k - 1:
                result.append(nums[deq[0]])

        return result
```

Time Complexity: O(n) - Each element pushed and popped at most once
Space Complexity: O(k) - Deque size

Approach 2: Dynamic Programming (Left and Right Arrays)

```
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) ->
List[int]:
        if not nums:
            return []

        n = len(nums)
        left = [0] * n
        right = [0] * n

        # Build left array
        for i in range(n):
            if i % k == 0:
                left[i] = nums[i]
            else:
                left[i] = max(left[i-1], nums[i])

        # Build right array
        right[n-1] = nums[n-1]
        for i in range(n-2, -1, -1):
            if (i + 1) % k == 0:
                right[i] = nums[i]
            else:
                right[i] = max(right[i+1], nums[i])

        # Calculate result
        result = []
        for i in range(n - k + 1):
            result.append(max(right[i], left[i + k - 1]))

        return result
```

Time Complexity: O(n) - Three passes through array
Space Complexity: O(n) - Two additional arrays

Approach 3: Max Heap (Priority Queue)

```
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) ->
List[int]:
        if not nums:
            return []

        max_heap = []
        result = []

        for i in range(len(nums)):
            heapq.heappush(max_heap, (-nums[i], i))
```

```
        if i >= k - 1:
            # Remove elements that are outside the window
            while max_heap[0][1] <= i - k:
                heapq.heappop(max_heap)

            result.append(-max_heap[0][0])

    return result
```

Time Complexity: O(n log n) - Heap operations take O(log n)
Space Complexity: O(n) - Heap size

Approach 4: Segment Tree

```
class SegmentTree:
    def __init__(self, nums):
        self.n = len(nums)
        self.tree = [0] * (4 * self.n)
        self.build(nums, 0, 0, self.n - 1)

    def build(self, nums, node, start, end):
        if start == end:
            self.tree[node] = nums[start]
        else:
            mid = (start + end) // 2
            self.build(nums, 2*node + 1, start, mid)
            self.build(nums, 2*node + 2, mid + 1, end)
            self.tree[node] = max(self.tree[2*node + 1], self.tree[2*node + 2])

    def query(self, node, start, end, l, r):
        if r < start or end < l:
            return float('-inf')
        if l <= start and end <= r:
            return self.tree[node]

        mid = (start + end) // 2
        left_max = self.query(2*node + 1, start, mid, l, r)
        right_max = self.query(2*node + 2, mid + 1, end, l, r)
        return max(left_max, right_max)

class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        if not nums:
            return []

        seg_tree = SegmentTree(nums)
        result = []

        for i in range(len(nums) - k + 1):
            result.append(seg_tree.query(0, 0, len(nums)-1, i, i+k-1))

        return result
```

Time Complexity: O(n log n) for building, O(log n) per query
Space Complexity: O(n) - Segment tree storage

---

567. Permutation in String

Given two strings s1 and s2, return true if s2 contains a permutation of s1, or false otherwise.

In other words, return true if one of s1's permutations is the substring of s2.

Example 1:

Input: s1 = "ab", s2 = "eidbaooo"
Output: true
Explanation: s2 contains one permutation of s1 ("ba").

Example 2:

Input: s1 = "ab", s2 = "eidboaoo"
Output: false

Approach 1: Sliding Window with Frequency Array

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        if len(s1) > len(s2):
            return False

        s1_count = [0] * 26
        s2_count = [0] * 26

        # Count frequencies for first window
        for i in range(len(s1)):
            s1_count[ord(s1[i]) - ord('a')] += 1
            s2_count[ord(s2[i]) - ord('a')] += 1

        # Check if first window matches
        matches = 0
        for i in range(26):
            if s1_count[i] == s2_count[i]:
                matches += 1

        # Slide the window
        for i in range(len(s1), len(s2)):
            if matches == 26:
                return True

            # Remove left character
            left_char = ord(s2[i - len(s1)]) - ord('a')
            s2_count[left_char] -= 1
            if s2_count[left_char] == s1_count[left_char]:
                matches += 1
            elif s2_count[left_char] + 1 == s1_count[left_char]:
                matches -= 1

            # Add right character
            right_char = ord(s2[i]) - ord('a')
            s2_count[right_char] += 1
            if s2_count[right_char] == s1_count[right_char]:
                matches += 1
            elif s2_count[right_char] - 1 == s1_count[right_char]:
                matches -= 1

        return matches == 26
```

Time Complexity: O(n) - Single pass through s2
Space Complexity: O(1) - Fixed size arrays

Approach 2: Optimized Sliding Window with Hash Map

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        if len(s1) > len(s2):
            return False

        s1_count = Counter(s1)
        window_count = Counter()
        matches = 0

        for i in range(len(s2)):
            # Add right character
            right_char = s2[i]
            window_count[right_char] += 1
            if window_count[right_char] == s1_count.get(right_char, 0):
                matches += 1
            elif window_count[right_char] == s1_count.get(right_char, 0) + 1:
                matches -= 1

            # Remove left character when window size exceeds
            if i >= len(s1):
                left_char = s2[i - len(s1)]
                window_count[left_char] -= 1
                if window_count[left_char] == s1_count.get(left_char, 0):
                    matches += 1
                elif window_count[left_char] == s1_count.get(left_char, 0) - 1:
                    matches -= 1

            if matches == len(s1_count):
                return True

        return False
```

Time Complexity: O(n)
Space Complexity: O(1) - Limited to 26 characters

Approach 3: Sorting Windows (Brute Force)

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        s1_sorted = sorted(s1)
        n = len(s1)

        for i in range(len(s2) - n + 1):
            window = s2[i:i+n]
            if sorted(window) == s1_sorted:
                return True

        return False
```

Time Complexity: O(n * m log m) where m = len(s1)
Space Complexity: O(m)

Approach 4: Using Sliding Window with Character Matches

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        n1, n2 = len(s1), len(s2)
        if n1 > n2:
            return False

        count = [0] * 26
        for i in range(n1):
            count[ord(s1[i]) - ord('a')] += 1
            count[ord(s2[i]) - ord('a')] -= 1

        # Check if first window is permutation
        if all(c == 0 for c in count):
            return True

        # Slide window
        for i in range(n1, n2):
            count[ord(s2[i]) - ord('a')] -= 1
            count[ord(s2[i - n1]) - ord('a')] += 1

            if all(c == 0 for c in count):
                return True

        return False
```

Time Complexity: O(n * 26) = O(n)
Space Complexity: O(1)

438. Find All Anagrams in a String

Given two strings s and p, return an array of all the start indices of p's anagrams in s. You may return the answer in any order.

Example 1:

Input: s = "cbaebabacd", p = "abc"
Output: [0,6]
Explanation:
The substring with start index = 0 is "cba", which is an anagram of "abc".
The substring with start index = 6 is "bac", which is an anagram of "abc".

Example 2:

Input: s = "abab", p = "ab"
Output: [0,1,2]
Explanation:
The substring with start index = 0 is "ab", which is an anagram of "ab".
The substring with start index = 1 is "ba", which is an anagram of "ab".
The substring with start index = 2 is "ab", which is an anagram of "ab".

Approach 1: Sliding Window with Frequency Array

```
class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        if len(p) > len(s):
            return []

        p_count = [0] * 26
        s_count = [0] * 26

        # Count frequencies for pattern p
        for char in p:
            p_count[ord(char) - ord('a')] += 1

        result = []
        window_len = len(p)

        # Initialize first window
        for i in range(window_len):
            s_count[ord(s[i]) - ord('a')] += 1

        # Check first window
        if p_count == s_count:
            result.append(0)

        # Slide the window
        for i in range(window_len, len(s)):
            # Remove left character
            s_count[ord(s[i - window_len]) - ord('a')] -= 1
            # Add right character
            s_count[ord(s[i]) - ord('a')] += 1

            # Check if current window is an anagram
            if p_count == s_count:
                result.append(i - window_len + 1)

        return result
```

Time Complexity: O(n + m) where n = len(s), m = len(p)
Space Complexity: O(1) - Fixed size arrays of 26

Approach 2: Optimized with Matches Count

```
class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        if len(p) > len(s):
            return []

        p_count = [0] * 26
        s_count = [0] * 26

        for char in p:
            p_count[ord(char) - ord('a')] += 1

        result = []
        left = 0
        matches = 0

        for right in range(len(s)):
            # Add right character
            idx = ord(s[right]) - ord('a')
            s_count[idx] += 1

            if s_count[idx] == p_count[idx]:
                matches += 1
            elif s_count[idx] == p_count[idx] + 1:
                matches -= 1

        # Remove left character when window exceeds size
            if right >= len(p):
                left_idx = ord(s[left]) - ord('a')
                s_count[left_idx] -= 1

                if s_count[left_idx] == p_count[left_idx]:
                    matches += 1
                elif s_count[left_idx] == p_count[left_idx] - 1:
                    matches -= 1

                left += 1
            # Check if all 26 characters match
            if matches == 26:
                result.append(left)

        return result
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 3: Using Counter and Sliding Window

```
class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        if len(p) > len(s):
            return []

        p_counter = Counter(p)
        s_counter = Counter()
        result = []
        window_len = len(p)

        for i in range(len(s)):
            s_counter[s[i]] += 1

        # Remove left character when window size exceeds
            if i >= window_len:
                if s_counter[s[i - window_len]] == 1:
                    del s_counter[s[i - window_len]]
                else:
                    s_counter[s[i - window_len]] -= 1

            # Check if current window is an anagram
            if i >= window_len - 1 and s_counter == p_counter:
                result.append(i - window_len + 1)

        return result
```

Time Complexity: O(n) - But Counter equality check is O(26)
Space Complexity: O(1) - At most 26 characters

---

904. Fruit Into Baskets

Given an integer array nums that may contain duplicates, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

Input: nums = [1,2,2]
Output: [[],[1],[1,2],[1,2,2],[2],[2,2]]

Example 2:

Input: nums = [0]
Output: [[],[0]]

Approach 1: Sliding Window with Hash Map

```
class Solution:
    def totalFruit(self, fruits: List[int]) -> int:
        basket = {}
        left = 0
        max_fruits = 0

        for right in range(len(fruits)):
            basket[fruits[right]] = basket.get(fruits[right], 0) + 1

            # Shrink window if we have more than 2 types
            while len(basket) > 2:
                basket[fruits[left]] -= 1
                if basket[fruits[left]] == 0:
                    del basket[fruits[left]]
                left += 1

            max_fruits = max(max_fruits, right - left + 1)

        return max_fruits
```

Time Complexity: O(n) - Each fruit visited at most twice
Space Complexity: O(1) - At most 3 entries in map

Approach 2: Optimized Sliding Window

```
class Solution:
    def totalFruit(self, fruits: List[int]) -> int:
        last_fruit = -1
        second_last_fruit = -1
        last_fruit_count = 0
        current_max = 0
        max_fruits = 0

        for fruit in fruits:
            if fruit == last_fruit or fruit == second_last_fruit:
                current_max += 1
            else:
                current_max = last_fruit_count + 1

            if fruit == last_fruit:
                last_fruit_count += 1
            else:
                last_fruit_count = 1
                second_last_fruit = last_fruit
                last_fruit = fruit

            max_fruits = max(max_fruits, current_max)

        return max_fruits
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 3: Two Pointers with Last Occurrence

```
class Solution:
    def totalFruit(self, fruits: List[int]) -> int:
        if len(fruits) <= 2:
            return len(fruits)

        fruit_types = {}
        left = 0
        max_fruits = 0

        for right in range(len(fruits)):
            fruit_types[fruits[right]] = right

            if len(fruit_types) > 2:
                # Find the fruit type to remove (the one with
smallest index)
                min_index = min(fruit_types.values())
                fruit_to_remove = fruits[min_index]
                del fruit_types[fruit_to_remove]
                left = min_index + 1

            max_fruits = max(max_fruits, right - left + 1)

        return max_fruits
```

Time Complexity: O(n)
Space Complexity: O(1)

1004. Max Consecutive Ones III

Given a binary array nums and an integer k, return *the maximum number of consecutive* 1*'s in the array if you can flip at most* k 0's.

Example 1:

Input: nums = [1,1,1,0,0,0,1,1,1,1,0], k = 2
Output: 6
Explanation: [1,1,1,0,0,1,1,1,1,1,1]
Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.
Example 2:

Input: nums = [0,0,1,1,0,0,1,1,1,0,1,1,0,0,0,1,1,1,1], k = 3
Output: 10
Explanation: [0,0,1,1,1,1,1,1,1,1,1,0,0,0,1,1,1,1]
Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.

Approach 1: Sliding Window

```
class Solution:
    def longestOnes(self, nums: List[int], k: int) -> int:
        left = 0
        zero_count = 0
        max_length = 0

        for right in range(len(nums)):
            if nums[right] == 0:
                zero_count += 1

            # Shrink window if zero_count exceeds k
            while zero_count > k:
                if nums[left] == 0:
                    zero_count -= 1
                left += 1

            max_length = max(max_length, right - left + 1)

        return max_length
```

Time Complexity: O(n) - Each element visited at most twice
Space Complexity: O(1)

Approach 2: Optimized Sliding Window (No Shrinking)

```
class Solution:
    def longestOnes(self, nums: List[int], k: int) -> int:
        left = 0
        right = 0

        for right in range(len(nums)):
            # If we encounter a 0, we reduce k
            if nums[right] == 0:
                k -= 1

            # If k becomes negative, move left pointer
            if k < 0:
                # If left pointer is pointing to 0, we increase k
                if nums[left] == 0:
                    k += 1
                left += 1

        return right - left + 1
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 3: Binary Search on Window Size

```
class Solution:
    def longestOnes(self, nums: List[int], k: int) -> int:
        def canMakeWindow(window_size):
            zero_count = 0

            # Check first window
            for i in range(window_size):
                if nums[i] == 0:
                    zero_count += 1

            if zero_count <= k:
                return True

            # Slide window
            for i in range(window_size, len(nums)):
                if nums[i - window_size] == 0:
                    zero_count -= 1
                if nums[i] == 0:
                    zero_count += 1

                if zero_count <= k:
                    return True

            return False

        left, right = 1, len(nums)
        result = 0

        while left <= right:
            mid = left + (right - left) // 2
            if canMakeWindow(mid):
                result = mid
                left = mid + 1
            else:
                right = mid - 1

        return result
```

Time Complexity: O(n log n)
Space Complexity: O(1)

Approach 4: Prefix Sum with Two Pointers

```
class Solution:
    def longestOnes(self, nums: List[int], k: int) -> int:
        n = len(nums)
        prefix_zeros = [0] * (n + 1)

        # Build prefix sum of zeros
        for i in range(1, n + 1):
            prefix_zeros[i] = prefix_zeros[i - 1] + (1 if nums[i -
1] == 0 else 0)

        left = 0
        max_length = 0

        for right in range(n):
            # Calculate zeros in current window
            zeros_in_window = prefix_zeros[right + 1] -
prefix_zeros[left]

            # If zeros exceed k, move left pointer
            if zeros_in_window > k:
                while left <= right and prefix_zeros[right + 1] -
prefix_zeros[left] > k:
                    left += 1

            max_length = max(max_length, right - left + 1)

        return max_length
```

Time Complexity: O(n)
Space Complexity: O(n)

1456. Maximum Number of Vowels in Substring

Given a string s and an integer k, return *the maximum number of vowel letters in any substring of* s *with length* k.

Vowel letters in English are 'a', 'e', 'i', 'o', and 'u'.

Example 1:

Input: s = "abciiidef", k = 3
Output: 3
Explanation: The substring "iii" contains 3 vowel letters.

Example 2:

Input: s = "aeiou", k = 2
Output: 2
Explanation: Any substring of length 2 contains 2 vowels.

Example 3:

Input: s = "leetcode", k = 3
Output: 2
Explanation: "lee", "eet" and "ode" contain 2 vowels.

Approach 1: Sliding Window

```
class Solution:
    def maxVowels(self, s: str, k: int) -> int:
        vowels = set(['a', 'e', 'i', 'o', 'u'])

        # Count vowels in first window
        current_vowels = 0
        for i in range(k):
            if s[i] in vowels:
                current_vowels += 1

        max_vowels = current_vowels
```

```
    # Slide the window
    for i in range(k, len(s)):
        # Remove left character
        if s[i - k] in vowels:
            current_vowels -= 1
        # Add right character
        if s[i] in vowels:
            current_vowels += 1

        max_vowels = max(max_vowels, current_vowels)

    return max_vowels
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 2: Preprocessed Vowel Array

```
class Solution:
    def maxVowels(self, s: str, k: int) -> int:
        n = len(s)
        is_vowel = [0] * n

        # Preprocess vowel positions
        vowels = set('aeiou')
        for i in range(n):
            if s[i] in vowels:
                is_vowel[i] = 1

        # Calculate first window sum
        current_sum = sum(is_vowel[:k])
        max_sum = current_sum

        # Slide window using prefix sum concept
        for i in range(k, n):
            current_sum += is_vowel[i] - is_vowel[i - k]
            max_sum = max(max_sum, current_sum)

        return max_sum
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 3: Optimized with Early Exit

```
class Solution:
    def maxVowels(self, s: str, k: int) -> int:
        vowels = set('aeiou')
        n = len(s)

        # Early exit if k == n
        if k == n:
            return sum(1 for char in s if char in vowels)

        current_vowels = 0
        max_vowels = 0

        for i in range(n):
            # Add current character
            if s[i] in vowels:
                current_vowels += 1

            # Remove character when window exceeds size
            if i >= k:
                if s[i - k] in vowels:
                    current_vowels -= 1

            # Update max if window is valid size
            if i >= k - 1:
                max_vowels = max(max_vowels,
current_vowels)
                # Early exit if max equals k (perfect window)
                if max_vowels == k:
                    return k

        return max_vowels
```

Time Complexity: O(n) with potential early exit
Space Complexity: O(1)

1493. Longest Subarray After Deleting One Element

Given a binary array nums, you should delete one element from it.

Return *the size of the longest non-empty subarray containing only 1's in the resulting array*. Return 0 if there is no such subarray.

Example 1:

Input: nums = [1,1,0,1]
Output: 3
Explanation: After deleting the number in position 2, [1,1,1] contains 3 numbers with value of 1's.

Example 2:

Input: nums = [0,1,1,1,0,1,1,0,1]
Output: 5
Explanation: After deleting the number in position 4, [0,1,1,1,1,1,0,1] longest subarray with value of 1's is [1,1,1,1,1].

Example 3:

Input: nums = [1,1,1]
Output: 2
Explanation: You must delete one element.

Approach 1: Sliding Window with Zero Count

```
class Solution:
    def longestSubarray(self, nums: List[int]) -> int:
        left = 0
        zero_count = 0
        max_length = 0

        for right in range(len(nums)):
            if nums[right] == 0:
                zero_count += 1

            # Shrink window if we have more than 1 zero
            while zero_count > 1:
                if nums[left] == 0:
                    zero_count -= 1
                left += 1

            max_length = max(max_length, right - left)

        return max_length
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 2: Count Consecutive Ones

```
class Solution:
    def longestSubarray(self, nums: List[int]) -> int:
        # Count consecutive ones separated by single zeros
        prev_ones = 0
        curr_ones = 0
        max_length = 0
        has_zero = False

        for num in nums:
            if num == 1:
                curr_ones += 1
            else:
                max_length = max(max_length, prev_ones +
curr_ones)
                prev_ones = curr_ones
                curr_ones = 0
                has_zero = True

        # Handle last segment
        max_length = max(max_length, prev_ones +
curr_ones)

        # If all ones, we must delete one element
        if not has_zero:
            return len(nums) - 1

        return max_length
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 3: Two Pointer with Zero Tracking

```
class Solution:
    def longestSubarray(self, nums: List[int]) -> int:
        left = 0
        last_zero = -1
        max_length = 0

        for right in range(len(nums)):
            if nums[right] == 0:
                left = last_zero + 1
                last_zero = right

            max_length = max(max_length, right - left)

        return max_length
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 4: Dynamic Programming

```
class Solution:
    def longestSubarray(self, nums: List[int]) -> int:
        n = len(nums)
        # dp0[i]: longest subarray ending at i with 0 deletions
        # dp1[i]: longest subarray ending at i with 1 deletion
        dp0 = [0] * n
        dp1 = [0] * n

        dp0[0] = 1 if nums[0] == 1 else 0
        dp1[0] = 0
        max_len = 0

        for i in range(1, n):
            if nums[i] == 1:
                dp0[i] = dp0[i-1] + 1
                dp1[i] = dp1[i-1] + 1
            else:
                dp0[i] = 0
                dp1[i] = dp0[i-1]

            max_len = max(max_len, dp1[i])

        # If array is all ones, we need to delete one
        if max_len == n:
            return n - 1

        return max_len
```

Time Complexity: O(n)
Space Complexity: O(n) (can be optimized to O(1))

930. Binary Subarrays With Sum

Given a binary array nums and an integer goal, return *the number of non-empty **subarrays** with a sum* goal.

A **subarray** is a contiguous part of the array.

Example 1:

Input: nums = [1,0,1,0,1], goal = 2
Output: 4
Explanation: The 4 subarrays are bolded and underlined below:
[**1,0,1**,0,1]
[**1,0,1,0**,1]
[1,**0,1,0,1**]
[1,0,**1,0,1**]

Example 2:

Input: nums = [0,0,0,0,0], goal = 0
Output: 15

Approach 1: Prefix Sum with Hash Map

```
class Solution:
    def numSubarraysWithSum(self, nums: List[int], goal: int) -> int:
        prefix_sum = 0
        count_map = {0: 1}  # Initialize with prefix sum 0
        result = 0

        for num in nums:
            prefix_sum += num
            # Check if (prefix_sum - goal) exists in map
            if prefix_sum - goal in count_map:
                result += count_map[prefix_sum - goal]
            # Add current prefix sum to map
            count_map[prefix_sum] = count_map.get(prefix_sum, 0) + 1

        return result
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: Sliding Window for At Most Goal

```
class Solution:
    def numSubarraysWithSum(self, nums: List[int], goal: int) -> int:
        def atMost(goal):
            if goal < 0:
                return 0

            left = 0
            current_sum = 0
            count = 0

            for right in range(len(nums)):
                current_sum += nums[right]

                while current_sum > goal:
                    current_sum -= nums[left]
                    left += 1

                # Number of subarrays ending at right
                count += right - left + 1

            return count

        # Subarrays with sum exactly goal = atMost(goal) - atMost(goal-1)
        return atMost(goal) - atMost(goal - 1)
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 3: Three Pointers

```
class Solution:
    def numSubarraysWithSum(self, nums: List[int], goal: int) -> int:
        i_lo = i_hi = 0
        sum_lo = sum_hi = 0
        result = 0

        for j in range(len(nums)):
            # Update i_lo and sum_lo (for atMost goal)
            sum_lo += nums[j]
            while i_lo < j and sum_lo > goal:
                sum_lo -= nums[i_lo]
                i_lo += 1

            # Update i_hi and sum_hi (for atMost goal-1)
            sum_hi += nums[j]
            while i_hi < j and sum_hi > goal - 1:
                sum_hi -= nums[i_hi]
                i_hi += 1

            # Add count of valid subarrays ending at j
            result += i_hi - i_lo

        return result
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 4: Count Zero Groups

```
class Solution:
    def numSubarraysWithSum(self, nums: List[int], goal: int) -> int:
        # Transform problem: count subarrays with exactly goal ones
        # Count zeros between ones
        zeros_between = []
        zero_count = 0

        for num in nums:
            if num == 0:
                zero_count += 1
            else:
                zeros_between.append(zero_count)
                zero_count = 0

        zeros_between.append(zero_count)  # trailing zeros

        if goal == 0:
            # Special case: sum of (n*(n+1)/2) for each zero group
            result = 0
            for z in zeros_between:
                result += z * (z + 1) // 2
            return result

        # For goal > 0
        result = 0
        for i in range(len(zeros_between) - goal):
            # Number of subarrays starting with zeros_between[i] + 1 choices
            # and ending with zeros_between[i + goal] + 1 choices
            result += (zeros_between[i] + 1) * (zeros_between[i + goal] + 1)

        return result
```

Time Complexity: O(n)
Space Complexity: O(n) for storing zero counts

BINARY SEARCH

704. Binary Search

Given an array of integers nums which is sorted in ascending order, and an integer target, write a function to search target in nums. If target exists, then return its index. Otherwise, return -1.

You must write an algorithm with O(log n) runtime complexity.

Example 1:

Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4

Example 2:

Input: nums = [-1,0,3,5,9,12], target = 2
Output: -1
Explanation: 2 does not exist in nums so return -1

Approach 1: Standard Binary Search (Iterative)

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1

        while left <= right:
            mid = left + (right - left) // 2

            if nums[mid] == target:
                return mid
            elif nums[mid] < target:
                left = mid + 1
            else:
                right = mid - 1

        return -1
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 2: Binary Search (Recursive)

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        def binary_search(left, right):
            if left > right:
                return -1

            mid = left + (right - left) // 2

            if nums[mid] == target:
                return mid
            elif nums[mid] < target:
                return binary_search(mid + 1, right)
            else:
                return binary_search(left, mid - 1)

        return binary_search(0, len(nums) - 1)
```

Time Complexity: O(log n)
Space Complexity: O(log n) - Recursion stack

Approach 3: Using bisect module

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        index = bisect.bisect_left(nums, target)
        if index < len(nums) and nums[index] == target:
            return index
        return -1
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 4: Template for finding first occurrence

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1

        while left < right:
            mid = left + (right - left) // 2

            if nums[mid] < target:
                left = mid + 1
            else:
                right = mid

        return left if nums[left] == target else -1
```

Time Complexity: O(log n)
Space Complexity: O(1)

---

35. Search Insert Position

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with O(log n) runtime complexity.

Example 1:

Input: nums = [1,3,5,6], target = 5
Output: 2

Example 2:

Input: nums = [1,3,5,6], target = 2
Output: 1

Example 3:

Input: nums = [1,3,5,6], target = 7
Output: 4

Approach 1: Standard Binary Search

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1

        while left <= right:
            mid = left + (right - left) // 2

            if nums[mid] == target:
                return mid
            elif nums[mid] < target:
                left = mid + 1
            else:
                right = mid - 1

        return left  # Insert position
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 2: Using bisect module

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        return bisect.bisect_left(nums, target)
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 3: Find lower bound

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums)

        while left < right:
            mid = left + (right - left) // 2

            if nums[mid] < target:
                left = mid + 1
            else:
                right = mid

        return left
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 4: Linear Search (for comparison)

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        for i in range(len(nums)):
            if nums[i] >= target:
                return i
        return len(nums)
```

Time Complexity: O(n)
Space Complexity: O(1)

---

33. Search in Rotated Sorted Array

There is an integer array nums sorted in ascending order (with **distinct** values).

Prior to being passed to your function, nums is **possibly left rotated** at an unknown index k (1 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (**0-indexed**). For example, [0,1,2,4,5,6,7] might be left rotated by 3 indices and become [4,5,6,7,0,1,2].

Given the array nums **after** the possible rotation and an integer target, return *the index of* target *if it is in* nums*, or* -1 *if it is not in* nums.

You must write an algorithm with O(log n) runtime complexity.

Example 1:

Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4

Example 2:

Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1

Example 3:

Input: nums = [1], target = 0
Output: -1

Approach 1: Modified Binary Search

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1

        while left <= right:
            mid = left + (right - left) // 2

            if nums[mid] == target:
                return mid

            # Check which side is sorted
            if nums[left] <= nums[mid]:  # Left side is sorted
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            else:  # Right side is sorted
                if nums[mid] < target <= nums[right]:
                    left = mid + 1
                else:
                    right = mid - 1

        return -1
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 2: Find Pivot + Binary Search

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        def find_pivot():
            left, right = 0, len(nums) - 1

            while left < right:
                mid = left + (right - left) // 2
                if nums[mid] > nums[right]:
                    left = mid + 1
                else:
                    right = mid
            return left

        pivot = find_pivot()
```

```
    # Binary search in appropriate segment
    def binary_search(left, right):
        while left <= right:
            mid = left + (right - left) // 2
            if nums[mid] == target:
                return mid
            elif nums[mid] < target:
                left = mid + 1
            else:
                right = mid - 1
        return -1

    # Determine which segment to search
    if target >= nums[pivot] and target <= nums[-1]:
        return binary_search(pivot, len(nums) - 1)
    else:
        return binary_search(0, pivot - 1)
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 3: One-pass with pivot detection

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1

        while left <= right:
            mid = left + (right - left) // 2

            if nums[mid] == target:
                return mid

            # Determine which part is sorted and where target lies
            if nums[mid] >= nums[left]:  # Left part is sorted
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            else:  # Right part is sorted
                if nums[mid] < target <= nums[right]:
                    left = mid + 1
                else:
                    right = mid - 1

        return -1
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 4: Find rotation point then binary search

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        n = len(nums)

        # Find rotation index (smallest element)
        def find_rotation():
            left, right = 0, n - 1
            while left < right:
                mid = left + (right - left) // 2
                if nums[mid] > nums[right]:
                    left = mid + 1
                else:
                    right = mid
            return left

        rotation = find_rotation()

        # Perform binary search with rotation offset
        left, right = 0, n - 1
        while left <= right:
            mid = left + (right - left) // 2
            real_mid = (mid + rotation) % n

            if nums[real_mid] == target:
                return real_mid
            elif nums[real_mid] < target:
                left = mid + 1
            else:
                right = mid - 1

        return -1
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 5: Linear Search (for comparison)

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        for i in range(len(nums)):
            if nums[i] == target:
                return i
        return -1
```

Time Complexity: O(n)
Space Complexity: O(1)

---

81. Search in Rotated Sorted Array II

There is an integer array nums sorted in non-decreasing order (not necessarily with **distinct** values).

Before being passed to your function, nums is **rotated** at an unknown pivot index k (0 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (**0-indexed**). For example, [0,1,2,4,4,4,5,6,6,7] might be rotated at pivot index 5 and become [4,5,6,6,7,0,1,2,4,4].

Given the array nums **after** the rotation and an integer target, return true *if* target *is in* nums*, or* false *if it is not in* nums.

You must decrease the overall operation steps as much as possible.

Example 1:

Input: nums = [2,5,6,0,0,1,2], target = 0
Output: true

Example 2:

Input: nums = [2,5,6,0,0,1,2], target = 3
Output: false

Approach 1: Modified Binary Search (Handling Duplicates)

```
class Solution:
    def search(self, nums: List[int], target: int) -> bool:
        left, right = 0, len(nums) - 1

        while left <= right:
            mid = left + (right - left) // 2

            if nums[mid] == target:
                return True

            # Handle duplicates: when nums[left] == nums[mid] == nums[right]
            if nums[left] == nums[mid] == nums[right]:
                left += 1
                right -= 1
            elif nums[left] <= nums[mid]:  # Left side is sorted
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            else:  # Right side is sorted
                if nums[mid] < target <= nums[right]:
                    left = mid + 1
                else:
                    right = mid - 1

        return False
```

Time Complexity: O(log n) average, O(n) worst-case (when many duplicates)
Space Complexity: O(1)

Approach 2: Find Pivot + Binary Search

```
class Solution:
    def search(self, nums: List[int], target: int) -> bool:
        def find_pivot():
            left, right = 0, len(nums) - 1

            while left < right:
                mid = left + (right - left) // 2

                if nums[mid] > nums[right]:
                    left = mid + 1
                elif nums[mid] < nums[right]:
                    right = mid
                else:  # nums[mid] == nums[right]
                    # Can't determine which side, reduce search space
                    right -= 1

            return left

        if not nums:
            return False

        pivot = find_pivot()

        # Binary search helper
        def binary_search(left, right):
            while left <= right:
                mid = left + (right - left) // 2
                if nums[mid] == target:
                    return True
                elif nums[mid] < target:
                    left = mid + 1
                else:
                    right = mid - 1
            return False

        # Search in both halves if needed
        if binary_search(0, pivot - 1) or binary_search(pivot, len(nums) - 1):
            return True

        return False
```

Time Complexity: O(log n) average, O(n) worst-case
Space Complexity: O(1)

Approach 3: Linear Search (for worst-case scenarios)

```
class Solution:
    def search(self, nums: List[int], target: int) -> bool:
        return target in nums
```

Time Complexity: O(n)
Space Complexity: O(1)

---

153. Find Minimum in Rotated Sorted Array

Suppose an array of length n sorted in ascending order is **rotated** between 1 and n times. For example, the array nums = [0,1,2,4,5,6,7] might become:

- [4,5,6,7,0,1,2] if it was rotated 4 times.
- [0,1,2,4,5,6,7] if it was rotated 7 times.

Notice that **rotating** an array [a[0], a[1], a[2], ..., a[n-1]] 1 time results in the array [a[n-1], a[0], a[1], a[2], ..., a[n-2]].

Given the sorted rotated array nums of **unique** elements, return *the minimum element of this array*.

You must write an algorithm that runs in O(log n) time.

Example 1:

Input: nums = [3,4,5,1,2]
Output: 1
Explanation: The original array was [1,2,3,4,5] rotated 3 times.

Example 2:

Input: nums = [4,5,6,7,0,1,2]
Output: 0
Explanation: The original array was [0,1,2,4,5,6,7] and it was rotated 4 times.

Example 3:

Input: nums = [11,13,15,17]
Output: 11
Explanation: The original array was [11,13,15,17] and it was rotated 4 times.

Approach 1: Binary Search

Time Complexity: O(log n)
Space Complexity: O(1)

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1

        while left < right:
            mid = left + (right - left) // 2

            if nums[mid] > nums[right]:
                # Minimum is in the right half
                left = mid + 1
            else:
                # Minimum is in the left half (including mid)
                right = mid

        return nums[left]
```

Approach 2: Find Inflection Point

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        if len(nums) == 1:
            return nums[0]

        left, right = 0, len(nums) - 1

        # If array is not rotated
        if nums[right] > nums[0]:
            return nums[0]

        while left <= right:
            mid = left + (right - left) // 2

            # Check if mid is the inflection point
            if nums[mid] > nums[mid + 1]:
                return nums[mid + 1]
            if nums[mid - 1] > nums[mid]:
                return nums[mid]

            # Decide which half to search
            if nums[mid] > nums[0]:
                left = mid + 1
            else:
                right = mid - 1

        return nums[0]
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 3: Using min() function (for comparison)

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        return min(nums)
```

Time Complexity: O(n)
Space Complexity: O(1)

154. Find Minimum in Rotated Sorted Array II

Suppose an array of length n sorted in ascending order is **rotated** between 1 and n times. For example, the array nums = [0,1,4,4,5,6,7] might become:

- [4,5,6,7,0,1,4] if it was rotated 4 times.
- [0,1,4,4,5,6,7] if it was rotated 7 times.

Notice that **rotating** an array [a[0], a[1], a[2], ..., a[n-1]] 1 time results in the array [a[n-1], a[0], a[1], a[2], ..., a[n-2]].

Given the sorted rotated array nums that may contain **duplicates**, return *the minimum element of this array*.

You must decrease the overall operation steps as much as possible.

Example 1:

Input: nums = [1,3,5]
Output: 1

Example 2:

Input: nums = [2,2,2,0,1]
Output: 0

Approach 1: Binary Search with Duplicate Handling

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1

        while left < right:
            mid = left + (right - left) // 2

            if nums[mid] > nums[right]:
                # Minimum is in the right half
                left = mid + 1
            elif nums[mid] < nums[right]:
                # Minimum is in the left half (including mid)
                right = mid
            else:  # nums[mid] == nums[right]
                # Can't determine which side, reduce search
space
                right -= 1

        return nums[left]
```

Time Complexity: O(log n) average, O(n) worst-case (when many duplicates)
Space Complexity: O(1)

Approach 2: Recursive Binary Search

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        def binary_search(left, right):
            if left == right:
                return nums[left]

            mid = left + (right - left) // 2

            if nums[mid] > nums[right]:
                return binary_search(mid + 1, right)
            elif nums[mid] < nums[right]:
                return binary_search(left, mid)
            else:  # nums[mid] == nums[right]
                # Search both halves
                left_min = binary_search(left, mid)
                right_min = binary_search(mid + 1, right)
                return min(left_min, right_min)

        return binary_search(0, len(nums) - 1)
```

Time Complexity: O(log n) average, O(n) worst-case
Space Complexity: O(log n) for recursion stack

Approach 3: Early Exit Optimization

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1

        # Early exit if array is not rotated
        if nums[right] > nums[left]:
            return nums[left]

        while left < right:
            # Skip duplicates from left
            while left < right and nums[left] == nums[left + 1]:
                left += 1
            # Skip duplicates from right
            while left < right and nums[right] == nums[right -
1]:
                right -= 1

            mid = left + (right - left) // 2

            if nums[mid] > nums[right]:
                left = mid + 1
            else:
                right = mid

        return nums[left]
```

Time Complexity: O(n) worst-case (when all elements are same)
Space Complexity: O(1)

Approach 4: Linear Scan (for worst-case)

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        min_val = nums[0]
        for i in range(1, len(nums)):
            if nums[i] < min_val:
                min_val = nums[i]
        return min_val
```

Time Complexity: O(n)
Space Complexity: O(1)

---

34. Find First and Last Position of Element

Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value.

If target is not found in the array, return [-1, -1].

You must write an algorithm with O(log n) runtime complexity.

Example 1:

Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]

Example 2:

Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]

Example 3:

Input: nums = [], target = 0
Output: [-1,-1]

Approach 1: Two Binary Searches

```
class Solution:
    def searchRange(self, nums: List[int], target: int) ->
List[int]:
        def find_first():
            left, right = 0, len(nums) - 1
            first = -1

            while left <= right:
                mid = left + (right - left) // 2

                if nums[mid] == target:
                    first = mid
                    right = mid - 1  # Continue searching left
                elif nums[mid] < target:
                    left = mid + 1
                else:
                    right = mid - 1

            return first

        def find_last():
            left, right = 0, len(nums) - 1
            last = -1

            while left <= right:
                mid = left + (right - left) // 2

                if nums[mid] == target:
                    last = mid
                    left = mid + 1  # Continue searching right
                elif nums[mid] < target:
                    left = mid + 1
                else:
                    right = mid - 1

            return last

        return [find_first(), find_last()]
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 2: Using bisect module

```
class Solution:
    def searchRange(self, nums: List[int], target: int) ->
List[int]:
        if not nums:
            return [-1, -1]

        left = bisect.bisect_left(nums, target)
        right = bisect.bisect_right(nums, target) - 1

        if left < len(nums) and nums[left] == target:
            return [left, right]
        return [-1, -1]
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 3: Single Binary Search + Linear Scan

```
class Solution:
    def searchRange(self, nums: List[int], target: int) ->
List[int]:
        left, right = 0, len(nums) - 1

        # Binary search to find target
        while left <= right:
            mid = left + (right - left) // 2

            if nums[mid] == target:
                # Found target, expand linearly
                start = end = mid
                while start > 0 and nums[start - 1] == target:
                    start -= 1
                while end < len(nums) - 1 and nums[end + 1] ==
target:
                    end += 1
                return [start, end]
            elif nums[mid] < target:
                left = mid + 1
            else:
                right = mid - 1

        return [-1, -1]
```

Time Complexity: O(log n + k) where k is number of target occurrences (worst O(n))
Space Complexity: O(1)

Approach 4: Template for first and last

```
class Solution:
    def searchRange(self, nums: List[int], target: int) ->
List[int]:
        def binary_search_left():
            left, right = 0, len(nums)
            while left < right:
                mid = left + (right - left) // 2
                if nums[mid] < target:
                    left = mid + 1
                else:
                    right = mid
            return left

        def binary_search_right():
            left, right = 0, len(nums)
            while left < right:
                mid = left + (right - left) // 2
                if nums[mid] <= target:
                    left = mid + 1
                else:
                    right = mid
            return left - 1

        first = binary_search_left()
        if first == len(nums) or nums[first] != target:
            return [-1, -1]

        last = binary_search_right()
        return [first, last]
```

Time Complexity: O(log n)
Space Complexity: O(1)

---

74. Search a 2D Matrix

You are given an m x n integer matrix matrix with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer target, return true *if* target *is in* matrix *or* false *otherwise*.

You must write a solution in O(log(m * n)) time complexity.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true

Example 2:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: false

Approach 1: Treat as 1D Array

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False

        m, n = len(matrix), len(matrix[0])
        left, right = 0, m * n - 1

        while left <= right:
            mid = left + (right - left) // 2
            # Convert 1D index to 2D coordinates
            row = mid // n
            col = mid % n

            if matrix[row][col] == target:
                return True
            elif matrix[row][col] < target:
                left = mid + 1
            else:
                right = mid - 1

        return False
```

Time Complexity: O(log(m*n)) = O(log m + log n)
Space Complexity: O(1)

Approach 2: Two Binary Searches

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False

        # Binary search to find correct row
        top, bottom = 0, len(matrix) - 1
        target_row = -1

        while top <= bottom:
            mid = top + (bottom - top) // 2
            if matrix[mid][0] <= target <= matrix[mid][-1]:
                target_row = mid
                break
            elif matrix[mid][0] > target:
                bottom = mid - 1
            else:
                top = mid + 1

        if target_row == -1:
            return False

        # Binary search within the row
        left, right = 0, len(matrix[0]) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if matrix[target_row][mid] == target:
                return True
            elif matrix[target_row][mid] < target:
                left = mid + 1
            else:
                right = mid - 1

        return False
```

Time Complexity: O(log m + log n)
Space Complexity: O(1)

Approach 3: Search from top-right corner

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False

        m, n = len(matrix), len(matrix[0])
        row, col = 0, n - 1  # Start from top-right corner

        while row < m and col >= 0:
            if matrix[row][col] == target:
                return True
            elif matrix[row][col] < target:
                # Move down (all elements in this row are smaller)
                row += 1
            else:
                # Move left (all elements in this column are larger)
                col -= 1

        return False
```

Time Complexity: O(m + n)
Space Complexity: O(1)

---

240. Search a 2D Matrix II

Write an efficient algorithm that searches for a value target in an m x n integer matrix matrix. This matrix has the following properties:

- Integers in each row are sorted in ascending from left to right.
- Integers in each column are sorted in ascending from top to bottom.

Example 1:

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

Input: matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 5
Output: true

Example 2:

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

Input: matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 20
Output: false

Approach 1: Search from top-right corner (optimal)

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False

        m, n = len(matrix), len(matrix[0])
        row, col = 0, n - 1  # Start from top-right

        while row < m and col >= 0:
            if matrix[row][col] == target:
                return True
            elif matrix[row][col] < target:
                row += 1  # Move down
            else:
                col -= 1  # Move left

        return False
```

Time Complexity: O(m + n)
Space Complexity: O(1)

Approach 2: Search from bottom-left corner

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False

        m, n = len(matrix), len(matrix[0])
        row, col = m - 1, 0  # Start from bottom-left

        while row >= 0 and col < n:
            if matrix[row][col] == target:
                return True
            elif matrix[row][col] < target:
                col += 1  # Move right
            else:
                row -= 1  # Move up

        return False
```

Time Complexity: O(m + n)
Space Complexity: O(1)

Approach 3: Binary Search each row

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False

        def binary_search(row):
            left, right = 0, len(matrix[row]) - 1
            while left <= right:
                mid = left + (right - left) // 2
                if matrix[row][mid] == target:
                    return True
                elif matrix[row][mid] < target:
                    left = mid + 1
                else:
                    right = mid - 1
            return False

        for i in range(len(matrix)):
            if matrix[i][0] <= target <= matrix[i][-1]:
                if binary_search(i):
                    return True

        return False
```

Time Complexity: O(m log n)
Space Complexity: O(1)

Approach 4: Divide and Conquer

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False

        def search_recursive(top, bottom, left, right):
            # Base cases
            if top > bottom or left > right:
                return False

            # Find middle row and column
            mid_row = top + (bottom - top) // 2
            mid_col = left + (right - left) // 2

            if matrix[mid_row][mid_col] == target:
                return True
            # If target is smaller, search top-left and adjacent quadrants
            if matrix[mid_row][mid_col] > target:
                return (search_recursive(top, mid_row - 1, left, right) or
                        search_recursive(mid_row, bottom, left, mid_col - 1))
            # If target is larger, search bottom-right and adjacent quadrants
            else:
                return (search_recursive(top, mid_row, mid_col + 1, right) or
                        search_recursive(mid_row + 1, bottom, left, right))
        return search_recursive(0, len(matrix) - 1, 0, len(matrix[0]) - 1)
```

Time Complexity: $O(n^{\log_3 4}) \approx O(n^{1.26})$
Space Complexity: O(log(min(m, n))) for recursion stack

---

162. Find Peak Element

A peak element is an element that is strictly greater than its neighbors.

Given a **0-indexed** integer array nums, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that nums[-1] = nums[n] = -∞. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in O(log n) time.

Example 1:

Input: nums = [1,2,3,1]
Output: 2
Explanation: 3 is a peak element and your function should return the index number 2.
Example 2:

Input: nums = [1,2,1,3,5,6,4]
Output: 5
Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

Approach 1: Binary Search

```
class Solution:
    def findPeakElement(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1

        while left < right:
            mid = left + (right - left) // 2

            if nums[mid] > nums[mid + 1]:
                # Peak is in left half (including mid)
                right = mid
            else:
                # Peak is in right half
                left = mid + 1

        return left
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 2: Linear Scan

```
class Solution:
    def findPeakElement(self, nums: List[int]) -> int:
        for i in range(len(nums) - 1):
            if nums[i] > nums[i + 1]:
                return i
        return len(nums) - 1  # Last element is peak
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 3: Recursive Binary Search

```
class Solution:
    def findPeakElement(self, nums: List[int]) -> int:
        def binary_search(left, right):
            if left == right:
                return left

            mid = left + (right - left) // 2

            if nums[mid] > nums[mid + 1]:
                return binary_search(left, mid)
            else:
                return binary_search(mid + 1, right)

        return binary_search(0, len(nums) - 1)
```

Time Complexity: O(log n)
Space Complexity: O(log n) for recursion stack

Approach 4: Compare with neighbors

```
class Solution:
    def findPeakElement(self, nums: List[int]) -> int:
        n = len(nums)

        # Check first element
        if n == 1 or nums[0] > nums[1]:
            return 0

        # Check last element
        if nums[-1] > nums[-2]:
            return n - 1

        # Check middle elements
        for i in range(1, n - 1):
            if nums[i - 1] < nums[i] > nums[i + 1]:
                return i

        return 0  # Should never reach here for valid input
```

Time Complexity: O(n)
Space Complexity: O(1)

278. First Bad Version

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have n versions [1, 2, ..., n] and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API bool isBadVersion(version) which returns whether version is bad. Implement a function to find the first bad version. You should minimize the number of calls to the API.

Example 1:

Input: n = 5, bad = 4
Output: 4
Explanation:
call isBadVersion(3) -> false
call isBadVersion(5) -> true
call isBadVersion(4) -> true
Then 4 is the first bad version.

Example 2:

Input: n = 1, bad = 1
Output: 1

Approach 1: Standard Binary Search

```
# The isBadVersion API is already defined for you.
# def isBadVersion(version: int) -> bool:

class Solution:
    def firstBadVersion(self, n: int) -> int:
        left, right = 1, n

        while left < right:
            mid = left + (right - left) // 2

            if isBadVersion(mid):
                # First bad version is in left half (including mid)
                right = mid
            else:
                # First bad version is in right half
                left = mid + 1

        return left
```

Time Complexity: O(log n)
Space Complexity: O(1)

Approach 2: Recursive Binary Search

```
class Solution:
    def firstBadVersion(self, n: int) -> int:
        def binary_search(left, right):
            if left == right:
                return left

            mid = left + (right - left) // 2

            if isBadVersion(mid):
                return binary_search(left, mid)
            else:
                return binary_search(mid + 1, right)

        return binary_search(1, n)
```

Time Complexity: O(log n)
Space Complexity: O(log n) for recursion stack

Approach 3: Using bisect module (simulated)

```
class Solution:
    def firstBadVersion(self, n: int) -> int:
        # Simulating bisect_left for custom condition
        left, right = 1, n

        while left < right:
            mid = left + (right - left) // 2

            if isBadVersion(mid):
                right = mid
            else:
                left = mid + 1

        return left
```

Time Complexity: O(log n)
Space Complexity: O(1)

---

69. Sqrt(x)

Given a non-negative integer x, return *the square root of* x *rounded down to the nearest integer*. The returned integer should be **non-negative** as well.

You **must not use** any built-in exponent function or operator.

- For example, do not use pow(x, 0.5) in c++ or x 0.5 in python.

Example 1:

Input: x = 4
Output: 2
Explanation: The square root of 4 is 2, so we return 2.

Example 2:

Input: x = 8
Output: 2
Explanation: The square root of 8 is 2.82842..., and since we round it down to the nearest integer, 2 is returned.

Approach 1: Binary Search

```
class Solution:
    def mySqrt(self, x: int) -> int:
        if x < 2:
            return x

        left, right = 2, x // 2

        while left <= right:
            mid = left + (right - left) // 2
            square = mid * mid

            if square == x:
                return mid
            elif square < x:
                left = mid + 1
            else:
                right = mid - 1

        return right  # Return floor value
```

Time Complexity: O(log x)
Space Complexity: O(1)

Approach 2: Newton's Method

```
class Solution:
    def mySqrt(self, x: int) -> int:
        if x < 2:
            return x

        guess = x // 2
        while guess * guess > x:
            guess = (guess + x // guess) // 2

        return guess
```

Time Complexity: O(log x) - converges quadratically
Space Complexity: O(1)

Approach 3: Bit Manipulation

```
class Solution:
    def mySqrt(self, x: int) -> int:
        if x < 2:
            return x

        # Find the most significant bit
        left = self.bit_length(x) // 2
        result = 0

        while left >= 0:
            candidate = result + (1 << left)
            if candidate * candidate <= x:
                result = candidate
            left -= 1

        return result

    def bit_length(self, n: int) -> int:
        length = 0
        while n:
            length += 1
            n >>= 1
        return length
```

Time Complexity: O(log x)
Space Complexity: O(1)

Approach 4: Built-in function

```
class Solution:
    def mySqrt(self, x: int) -> int:
        import math
        return int(math.sqrt(x))
```

Time Complexity: O(1) typically
Space Complexity: O(1)

---

875. Koko Eating Bananas

Koko loves to eat bananas. There are n piles of bananas, the i_{th} pile has piles[i] bananas. The guards have gone and will come back in h hours.

Koko can decide her bananas-per-hour eating speed of k. Each hour, she chooses some pile of bananas and eats k bananas from that pile. If the pile has less than k bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return.

Return *the minimum integer* k *such that she can eat all the bananas within* h *hours*.

Example 1:

Input: piles = [3,6,7,11], h = 8
Output: 4

Example 2:

Input: piles = [30,11,23,4,20], h = 5
Output: 30

Example 3:

Input: piles = [30,11,23,4,20], h = 6
Output: 23

Approach 1: Binary Search on Eating Speed

```
class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        def can_finish(speed):
            hours = 0
            for bananas in piles:
                hours += (bananas + speed - 1) // speed  #
Ceiling division
                if hours > h:
                    return False
            return True

        left, right = 1, max(piles)

        while left < right:
            mid = left + (right - left) // 2

            if can_finish(mid):
                # Try slower speed
                right = mid
            else:
                # Need faster speed
                left = mid + 1

        return left
```

Time Complexity: O(n log m) where n = len(piles), m = max(piles)
Space Complexity: O(1)

Approach 2: Optimized with early checks

```
class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        if len(piles) == h:
            return max(piles)

        # Calculate initial bounds
        total = sum(piles)
        left = (total + h - 1) // h  # Ceiling division
        right = max(piles)

        def can_finish(speed):
            hours = 0
            for bananas in piles:
                hours += (bananas + speed - 1) // speed
                if hours > h:
                    return False
            return True

        while left < right:
            mid = left + (right - left) // 2

            if can_finish(mid):
                right = mid
            else:
                left = mid + 1

        return left
```

Time Complexity: O(n log m)
Space Complexity: O(1)

Approach 3: Using math.ceil

```
class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        def can_finish(speed):
            return sum(math.ceil(bananas / speed) for
bananas in piles) <= h

        left, right = 1, max(piles)

        while left < right:
            mid = left + (right - left) // 2

            if can_finish(mid):
                right = mid
            else:
                left = mid + 1

        return left
```

Time Complexity: O(n log m)
Space Complexity: O(1)

Approach 4: Linear Search (for comparison)

```
class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        max_pile = max(piles)

        for speed in range(1, max_pile + 1):
            hours = 0
            for bananas in piles:
                hours += (bananas + speed - 1) // speed

            if hours <= h:
                return speed

        return max_pile
```

Time Complexity: O(n * m) where m = max(piles)
Space Complexity: O(1)

MONOTONIC STACK

739. Daily Temperatures

Given an array of integers temperatures represents the daily temperatures, return *an array* answer *such that* answer[i] *is the number of days you have to wait after the* ith *day to get a warmer temperature*. If there is no future day for which this is possible, keep answer[i] == 0 instead.

Example 1:

Input: temperatures = [73,74,75,71,69,72,76,73]
Output: [1,1,4,2,1,1,0,0]

Example 2:

Input: temperatures = [30,40,50,60]
Output: [1,1,1,0]

Example 3:

Input: temperatures = [30,60,90]
Output: [1,1,0]

Approach 1: Monotonic Decreasing Stack

```
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) ->
List[int]:
        n = len(temperatures)
        answer = [0] * n
        stack = []  # Store indices of temperatures in
decreasing order

        for i in range(n):
            # While current temperature is warmer than stack
top
            while stack and temperatures[i] >
temperatures[stack[-1]]:
                prev_index = stack.pop()
                answer[prev_index] = i - prev_index

            stack.append(i)

        return answer
```

Time Complexity: O(n) - Each element pushed and popped at most once
Space Complexity: O(n) - Stack and answer array

Approach 2: Right-to-Left Traversal

```
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) ->
List[int]:
        n = len(temperatures)
        answer = [0] * n
        hottest = 0  # Track hottest temperature seen so far

        for i in range(n-1, -1, -1):
            current_temp = temperatures[i]

            # If current is hottest, no warmer day exists
            if current_temp >= hottest:
                hottest = current_temp
                continue

            # Otherwise, find next warmer day
            days = 1
            while temperatures[i + days] <= current_temp:
                # Use answer to skip days (DP-like)
                days += answer[i + days]

            answer[i] = days

        return answer
```

Time Complexity: O(n) - Each day visited at most twice
Space Complexity: O(1) additional space (answer array is required output)

Approach 3: Brute Force (for comparison)

```
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) ->
List[int]:
        n = len(temperatures)
        answer = [0] * n

        for i in range(n):
            for j in range(i+1, n):
                if temperatures[j] > temperatures[i]:
                    answer[i] = j - i
                    break

        return answer
```

Time Complexity: $O(n^2)$
Space Complexity: O(1) additional space

496. Next Greater Element I

The **next greater element** of some element x in an array is the **first greater** element that is **to the right** of x in the same array.

You are given two **distinct 0-indexed** integer arrays nums1 and nums2, where nums1 is a subset of nums2.

For each 0 <= i < nums1.length, find the index j such that nums1[i] == nums2[j] and determine the **next greater element** of nums2[j] in nums2. If there is no next greater element, then the answer for this query is -1.

Return *an array* ans *of length* nums1.length *such that* ans[i] *is the* ***next greater element*** *as described above.*

Example 1:

Input: nums1 = [4,1,2], nums2 = [1,3,4,2]
Output: [-1,3,-1]
Explanation: The next greater element for each value of nums1 is as follows:
- 4 is underlined in nums2 = [1,3,4,2]. There is no next greater element, so the answer is -1.
- 1 is underlined in nums2 = [1,3,4,2]. The next greater element is 3.
- 2 is underlined in nums2 = [1,3,4,2]. There is no next greater element, so the answer is -1.

Example 2:

Input: nums1 = [2,4], nums2 = [1,2,3,4]
Output: [3,-1]
Explanation: The next greater element for each value of nums1 is as follows:
- 2 is underlined in nums2 = [1,2,3,4]. The next greater element is 3.
- 4 is underlined in nums2 = [1,2,3,4]. There is no next greater element, so the answer is -1.

Approach 1: Monotonic Stack + Hash Map

```
class Solution:
    def nextGreaterElement(self, nums1: List[int], nums2: List[int]) -> List[int]:
        next_greater = {}
        stack = []

        # Find next greater for each element in nums2
        for num in nums2:
            while stack and num > stack[-1]:
                next_greater[stack.pop()] = num
            stack.append(num)

        # Remaining elements have no next greater
        while stack:
            next_greater[stack.pop()] = -1

        # Build result for nums1
        result = []
        for num in nums1:
            result.append(next_greater[num])

        return result
```

Time Complexity: O(n + m) where n = len(nums2), m = len(nums1)
Space Complexity: O(n) for stack and hash map

Approach 2: Optimized with direct array processing

```
class Solution:
    def nextGreaterElement(self, nums1: List[int], nums2: List[int]) -> List[int]:
        # Store indices of nums1 elements in nums2
        index_map = {}
        for i, num in enumerate(nums2):
            index_map[num] = i

        result = []
        for num in nums1:
            idx = index_map[num]
            found = -1

            # Search for next greater in nums2
            for j in range(idx + 1, len(nums2)):
                if nums2[j] > num:
                    found = nums2[j]
                    break

            result.append(found)

        return result
```

Time Complexity: O(n * m) in worst case
Space Complexity: O(n)

Approach 3: Using Stack with Array

```
class Solution:
    def nextGreaterElement(self, nums1: List[int], nums2: List[int]) -> List[int]:
        # Find all next greater elements for nums2
        n = len(nums2)
        next_greater = [-1] * n
        stack = []

        for i in range(n):
            while stack and nums2[i] > nums2[stack[-1]]:
                prev_idx = stack.pop()
                next_greater[prev_idx] = nums2[i]
            stack.append(i)

        # Map nums2 elements to their next greater
        num_to_next = {}
        for i in range(n):
            num_to_next[nums2[i]] = next_greater[i]

        # Get results for nums1
        return [num_to_next[num] for num in nums1]
```

Time Complexity: O(n + m)
Space Complexity: O(n)

503. Next Greater Element II

Given a circular integer array nums (i.e., the next element of nums[nums.length - 1] is nums[0]), return *the* ***next greater number*** *for every element in* nums.

The **next greater number** of a number x is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return -1 for this number.

Example 1:

Input: nums = [1,2,1]
Output: [2,-1,2]
Explanation: The first 1's next greater number is 2;
The number 2 can't find next greater number.
The second 1's next greater number needs to search circularly, which is also 2.

Example 2:

Input: nums = [1,2,3,4,3]
Output: [2,3,4,-1,4]

Approach 1: Monotonic Stack with Circular Array (Double Pass)

```
class Solution:
    def nextGreaterElements(self, nums: List[int]) -> List[int]:
        n = len(nums)
        result = [-1] * n
        stack = []  # Store indices

        # Process circular array by going through it twice
        for i in range(2 * n):
            idx = i % n
            while stack and nums[idx] > nums[stack[-1]]:
                prev_idx = stack.pop()
                result[prev_idx] = nums[idx]

            # Only push indices from first pass
            if i < n:
                stack.append(idx)

        return result
```

Time Complexity: O(n) - Each element pushed and popped at most once
Space Complexity: O(n) for stack and result

Approach 2: Brute Force Circular

```
class Solution:
    def nextGreaterElements(self, nums: List[int]) -> List[int]:
        n = len(nums)
        result = [-1] * n

        for i in range(n):
            for j in range(1, n):  # Check n-1 elements
                idx = (i + j) % n
                if nums[idx] > nums[i]:
                    result[i] = nums[idx]
                    break

        return result
```

Time Complexity: $O(n^2)$
Space Complexity: O(1) additional space

Approach 3: Using Extended Array

```
class Solution:
    def nextGreaterElements(self, nums: List[int]) ->
List[int]:
        n = len(nums)
        # Create extended array: nums + nums
        extended_nums = nums + nums
        result = [-1] * n
        stack = []

        for i in range(2 * n):
            while stack and extended_nums[i] >
extended_nums[stack[-1]]:
                idx = stack.pop()
                if idx < n:  # Only update for original indices
                    result[idx] = extended_nums[i]
            stack.append(i)

        return result
```

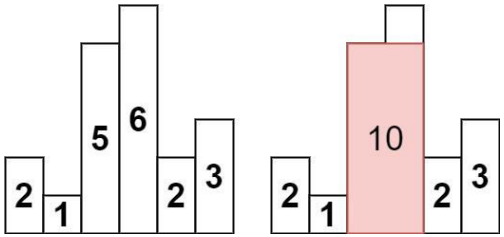
Time Complexity: O(n)
Space Complexity: O(n) for extended array

---

84. Largest Rectangle in Histogram

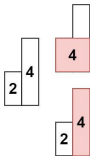
Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return *the area of the largest rectangle in the histogram*.

Example 1:



Input: heights = [2,1,5,6,2,3]
Output: 10
Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 units.

Example 2:



Input: heights = [2,4]
Output: 4

Approach 1: Monotonic Stack

```
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        stack = []  # Store indices in increasing height order
        max_area = 0
        n = len(heights)

        for i in range(n + 1):
            # Add sentinel height 0 at the end
            current_height = heights[i] if i < n else 0

            while stack and current_height < heights[stack[-1]]:
                height = heights[stack.pop()]
                # Width extends from popped index to current
index
                width = i if not stack else i - stack[-1] - 1
                max_area = max(max_area, height * width)

            stack.append(i)

        return max_area
```

Time Complexity: O(n) - Each bar pushed and popped once
Space Complexity: O(n) for stack

Approach 2: Divide and Conquer

```
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        def divide_and_conquer(left, right):
            if left > right:
                return 0

            # Find minimum height index in range
            min_idx = left
            for i in range(left, right + 1):
                if heights[i] < heights[min_idx]:
                    min_idx = i

            # Calculate area with minimum height
            current_area = heights[min_idx] * (right - left + 1)

            # Recursively calculate left and right areas
            left_area = divide_and_conquer(left, min_idx - 1)
            right_area = divide_and_conquer(min_idx + 1,
right)

            return max(current_area, left_area, right_area)

        return divide_and_conquer(0, len(heights) - 1)
```

Time Complexity: O(n log n) average, O(n²) worst-case
Space Complexity: O(log n) for recursion stack

Approach 3: Dynamic Programming with Left/Right Boundaries

```
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        if not heights:
            return 0

        n = len(heights)
        left_bound = [0] * n  # Index of first bar lower than
current on left
        right_bound = [0] * n  # Index of first bar lower than
current on right

        # Calculate left boundaries
        left_bound[0] = -1
        for i in range(1, n):
            p = i - 1
            while p >= 0 and heights[p] >= heights[i]:
                p = left_bound[p]
            left_bound[i] = p

        # Calculate right boundaries
        right_bound[n-1] = n
        for i in range(n-2, -1, -1):
            p = i + 1
            while p < n and heights[p] >= heights[i]:
                p = right_bound[p]
            right_bound[i] = p

        # Calculate max area
        max_area = 0
        for i in range(n):
            width = right_bound[i] - left_bound[i] - 1
            max_area = max(max_area, heights[i] * width)

        return max_area
```

Time Complexity: O(n) - Each bar visited at most twice
Space Complexity: O(n) for boundary arrays

Approach 4: Brute Force

```
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        n = len(heights)
        max_area = 0

        for i in range(n):
            min_height = float('inf')
            for j in range(i, n):
                min_height = min(min_height, heights[j])
                width = j - i + 1
                max_area = max(max_area, min_height * width)

        return max_area
```

Time Complexity: O(n²)
Space Complexity: O(1)

85. Maximal Rectangle

Given a rows x cols binary matrix filled with 0's and 1's, find the largest rectangle containing only 1's and return *its area*.

Example 1:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

Input: matrix = [["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]
Output: 6
Explanation: The maximal rectangle is shown in the above picture.

Example 2:

Input: matrix = [["0"]]
Output: 0

Example 3:

Input: matrix = [["1"]]
Output: 1

Approach 1: Dynamic Programming with Histogram Method

```
class Solution:
    def maximalRectangle(self, matrix: List[List[str]]) -> int:
        if not matrix or not matrix[0]:
            return 0

        rows, cols = len(matrix), len(matrix[0])
        # Heights array for histogram
        heights = [0] * (cols + 1)  # +1 for sentinel
        max_area = 0

        for i in range(rows):
            # Update heights for current row
            for j in range(cols):
                if matrix[i][j] == '1':
                    heights[j] += 1
                else:
                    heights[j] = 0

            # Calculate largest rectangle in histogram for current heights
            stack = []
            for j in range(cols + 1):
                while stack and heights[j] < heights[stack[-1]]:
                    height = heights[stack.pop()]
                    width = j if not stack else j - stack[-1] - 1
                    max_area = max(max_area, height * width)
                stack.append(j)

        return max_area
```

Time Complexity: O(rows × cols) - Each cell processed once
Space Complexity: O(cols) for heights array and stack

Approach 2: Dynamic Programming with Left/Right Boundaries

```
class Solution:
    def maximalRectangle(self, matrix: List[List[str]]) -> int:
        if not matrix or not matrix[0]:
            return 0

        rows, cols = len(matrix), len(matrix[0])
        max_area = 0

        # DP arrays
        heights = [0] * cols
        left = [0] * cols
        right = [cols] * cols

        for i in range(rows):
            # Update heights
            for j in range(cols):
                heights[j] = heights[j] + 1 if matrix[i][j] == '1' else 0

            # Update left boundaries
            current_left = 0
            for j in range(cols):
                if matrix[i][j] == '1':
                    left[j] = max(left[j], current_left)
                else:
                    left[j] = 0
                    current_left = j + 1

            # Update right boundaries
            current_right = cols
            for j in range(cols-1, -1, -1):
                if matrix[i][j] == '1':
                    right[j] = min(right[j], current_right)
                else:
                    right[j] = cols
                    current_right = j

            # Calculate max area
            for j in range(cols):
                max_area = max(max_area, heights[j] * (right[j] - left[j]))

        return max_area
```

Time Complexity: O(rows × cols)
Space Complexity: O(cols)

Approach 3: Brute Force with Optimization

```
class Solution:
    def maximalRectangle(self, matrix: List[List[str]]) -> int:
        if not matrix or not matrix[0]:
            return 0

        rows, cols = len(matrix), len(matrix[0])
        max_area = 0

        # Preprocess: convert to heights
        heights = [[0] * cols for _ in range(rows)]

        # Build heights matrix
        for j in range(cols):
            consecutive = 0
            for i in range(rows):
                if matrix[i][j] == '1':
                    consecutive += 1
                    heights[i][j] = consecutive
                else:
                    consecutive = 0

        # For each cell, expand left to find max rectangle
        for i in range(rows):
            for j in range(cols):
                if heights[i][j] > 0:
                    height = heights[i][j]
                    width = 1

                    # Expand left
                    k = j - 1
                    while k >= 0 and heights[i][k] >= height:
                        width += 1
                        k -= 1

                    # Expand right
                    k = j + 1
                    while k < cols and heights[i][k] >= height:
                        width += 1
                        k += 1

                    max_area = max(max_area, height * width)

        return max_area
```

Time Complexity: O(rows × cols²) in worst case
Space Complexity: O(rows × cols)

901. Online Stock Span

Design an algorithm that collects daily price quotes for some stock and returns **the span** of that stock's price for the current day.

The **span** of the stock's price in one day is the maximum number of consecutive days (starting from that day and going backward) for which the stock price was less than or equal to the price of that day.

- For example, if the prices of the stock in the last four days is [7,2,1,2] and the price of the stock today is 2, then the span of today is 4 because starting from today, the price of the stock was less than or equal 2 for 4 consecutive days.
- Also, if the prices of the stock in the last four days is [7,34,1,2] and the price of the stock today is 8, then the span of today is 3 because starting from today, the price of the stock was less than or equal 8 for 3 consecutive days.

Implement the StockSpanner class:

- StockSpanner() Initializes the object of the class.
- int next(int price) Returns the **span** of the stock's price given that today's price is price.

Example 1:

Input
["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]
[[], [100], [80], [60], [70], [60], [75], [85]]
Output
[null, 1, 1, 1, 2, 1, 4, 6]

Explanation
StockSpanner stockSpanner = new StockSpanner();
stockSpanner.next(100); // return 1
stockSpanner.next(80); // return 1
stockSpanner.next(60); // return 1
stockSpanner.next(70); // return 2
stockSpanner.next(60); // return 1
stockSpanner.next(75); // return 4, because the last 4 prices (including today's price of 75) were less than or equal to today's price.
stockSpanner.next(85); // return 6

Approach 1: Monotonic Stack

```
class StockSpanner:
    def __init__(self):
        self.stack = []  # Store (price, span) pairs
        # Stack maintains decreasing prices

    def next(self, price: int) -> int:
        span = 1

        # While stack not empty and current price >= stack top price
        while self.stack and price >= self.stack[-1][0]:
            span += self.stack.pop()[1]

        self.stack.append((price, span))
        return span
```

Time Complexity: Amortized O(1) per call - Each price pushed and popped at most once
Space Complexity: O(n) where n is number of calls

Approach 2: Using Index-based Stack

```
class StockSpanner:
    def __init__(self):
        self.prices = []
        self.stack = []  # Store indices

    def next(self, price: int) -> int:
        self.prices.append(price)
        idx = len(self.prices) - 1

        # Remove smaller or equal prices from stack
        while self.stack and self.prices[self.stack[-1]] <= price:
            self.stack.pop()

        # Calculate span
        if not self.stack:
            span = idx + 1
        else:
            span = idx - self.stack[-1]

        self.stack.append(idx)
        return span
```

Time Complexity: Amortized O(1) per call
Space Complexity: O(n)

Approach 3: DP-like approach

```
class StockSpanner:
    def __init__(self):
        self.prices = []
        self.spans = []

    def next(self, price: int) -> int:
        self.prices.append(price)

        span = 1
        i = len(self.prices) - 2

        # Jump back using spans
        while i >= 0 and self.prices[i] <= price:
            span += self.spans[i]
            i -= self.spans[i]

        self.spans.append(span)
        return span
```

Time Complexity: Amortized O(1) per call
Space Complexity: O(n)

---

402. Remove K Digits

Given string num representing a non-negative integer num, and an integer k, return *the smallest possible integer after removing* k *digits from* num.

Example 1:

Input: num = "1432219", k = 3
Output: "1219"
Explanation: Remove the three digits 4, 3, and 2 to form the new number 1219 which is the smallest.

Example 2:

Input: num = "10200", k = 1
Output: "200"
Explanation: Remove the leading 1 and the number is 200. Note that the output must not contain leading zeroes.

Example 3:

Input: num = "10", k = 2
Output: "0"
Explanation: Remove all the digits from the number and it is left with nothing which is 0.

Approach 1: Monotonic Increasing Stack

```
class Solution:
    def removeKdigits(self, num: str, k: int) -> str:
        if k >= len(num):
            return "0"

        stack = []
        removed = 0

        for digit in num:
            while stack and removed < k and stack[-1] > digit:
                stack.pop()
                removed += 1
            stack.append(digit)

        # If still need to remove digits (case when digits are increasing)
        while removed < k:
            stack.pop()
            removed += 1

        # Remove leading zeros
        result = ''.join(stack).lstrip('0')

        return result if result else "0"
```

Time Complexity: O(n) - Each digit processed once
Space Complexity: O(n) for stack

Approach 2: Using deque for efficiency

```
class Solution:
    def removeKdigits(self, num: str, k: int) -> str:
        if k >= len(num):
            return "0"

        dq = deque()
        removed = 0

        for digit in num:
            while dq and removed < k and dq[-1] > digit:
                dq.pop()
                removed += 1
            dq.append(digit)

        # Remove remaining if needed
        for _ in range(k - removed):
            dq.pop()

        # Remove leading zeros
        while dq and dq[0] == '0':
            dq.popleft()

        result = ''.join(dq)
        return result if result else "0"
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 3: Recursive (Brute Force)

```
class Solution:
    def removeKdigits(self, num: str, k: int) -> str:
        if k == 0:
            return num.lstrip('0') or "0"

        if len(num) <= k:
            return "0"

        # Try removing each digit, take minimum result
        min_num = None
        for i in range(len(num)):
            new_num = num[:i] + num[i+1:]
            result = self.removeKdigits(new_num, k-1)
            if min_num is None or result < min_num:
                min_num = result

        return min_num
```

Time Complexity: O(C(n,k)) - Exponential
Space Complexity: O(n) for recursion stack

---

316. Remove Duplicate Letters

Given a string s, remove duplicate letters so that every letter appears once and only once. You must make sure your result is **the smallest in lexicographical order** among all possible results.

Example 1:

Input: s = "bcabc"
Output: "abc"

Example 2:

Input: s = "cbacdcbc"
Output: "acdb"

Approach 1: Monotonic Stack with Last Occurrence

```
class Solution:
    def removeDuplicateLetters(self, s: str) -> str:
        # Store last occurrence index of each character
        last_occurrence = {}
        for i, ch in enumerate(s):
            last_occurrence[ch] = i

        stack = []
        seen = set()  # Track characters in stack

        for i, ch in enumerate(s):
            if ch in seen:
                continue

            # While stack not empty, current char < stack top,
```

```
        # and stack top appears later in string
        while stack and ch < stack[-1] and
last_occurrence[stack[-1]] > i:
            seen.remove(stack.pop())

        stack.append(ch)
        seen.add(ch)

    return ''.join(stack)
```

Time Complexity: O(n) - Each character processed once
Space Complexity: O(1) for alphabet (26 letters)

Approach 2: Using Counter and Visited

```
class Solution:
  def removeDuplicateLetters(self, s: str) -> str:
    count = Counter(s)
    stack = []
    in_stack = [False] * 26

    for ch in s:
      count[ch] -= 1

      if in_stack[ord(ch) - ord('a')]:
        continue

# Remove characters that are greater and have remaining occurrences
      while stack and ch < stack[-1] and count[stack[-1]]
> 0:
        in_stack[ord(stack.pop()) - ord('a')] = False

      stack.append(ch)
      in_stack[ord(ch) - ord('a')] = True

    return ''.join(stack)
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 3: Recursive Greedy

```
class Solution:
  def removeDuplicateLetters(self, s: str) -> str:
    # Find the smallest character whose suffix contains
all unique characters
    for ch in sorted(set(s)):
      suffix = s[s.index(ch):]
      if set(suffix) == set(s):
        # Recursively process the remaining string
        return ch +
self.removeDuplicateLetters(suffix.replace(ch, ''))
    return ''
```

Time Complexity: O(n²) in worst case
Space Complexity: O(n) for recursion stack

Approach 4: Using deque with frequency count

```
class Solution:
  def removeDuplicateLetters(self, s: str) -> str:
    count = Counter(s)
    dq = deque()
    in_dq = [False] * 26

    for ch in s:
      count[ch] -= 1

      if in_dq[ord(ch) - ord('a')]:
        continue

      # Remove from back while possible
      while dq and ch < dq[-1] and count[dq[-1]] > 0:
        in_dq[ord(dq.pop()) - ord('a')] = False

      dq.append(ch)
      in_dq[ord(ch) - ord('a')] = True

    return ''.join(dq)
```

Time Complexity: O(n)
Space Complexity: O(1)

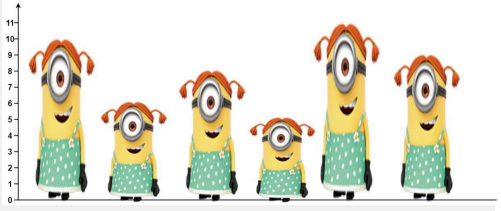
1944. Number of Visible People in a Queue

There are n people standing in a queue, and they numbered from 0 to n - 1 in **left to right** order. You are given an array heights of **distinct** integers where heights[i] represents the height of the i_{th} person.

A person can **see** another person to their right in the queue if everybody in between is **shorter** than both of them. More formally, the i_{th} person can see the j_{th} person if i < j and min(heights[i], heights[j]) > max(heights[i+1], heights[i+2], ..., heights[j-1]).

Return *an array* answer *of length* n *where* answer[i] *is the* ***number of people*** *the* i_{th} *person can* ***see*** *to their right in the queue*.

Example 1:



Input: heights = [10,6,8,5,11,9]
Output: [3,1,2,1,1,0]
Explanation:
Person 0 can see person 1, 2, and 4.
Person 1 can see person 2.
Person 2 can see person 3 and 4.
Person 3 can see person 4.
Person 4 can see person 5.
Person 5 can see no one since nobody is to the right of them.

Example 2:

Input: heights = [5,1,2,3,10]
Output: [4,1,1,1,0]

Approach 1: Monotonic Decreasing Stack

```
class Solution:
  def canSeePersonsCount(self, heights: List[int]) ->
List[int]:
    n = len(heights)
    result = [0] * n
    stack = []  # Monotonic decreasing stack of indices

    for i in range(n-1, -1, -1):
      visible = 0

      # Count people who are shorter than current
person
      while stack and heights[i] > heights[stack[-1]]:
        stack.pop()
        visible += 1

      # If there's someone taller or equal, count them
too
      if stack:
        visible += 1

      result[i] = visible
      stack.append(i)

    return result
```

Time Complexity: O(n) - Each person pushed and popped once
Space Complexity: O(n) for stack

Approach 2: Alternative Stack Implementation

```
class Solution:
  def canSeePersonsCount(self, heights: List[int]) ->
List[int]:
    n = len(heights)
    result = [0] * n
    stack = []  # Store heights in decreasing order

    for i in range(n-1, -1, -1):
      count = 0

      # Count all people that are visible
      while stack and heights[i] > stack[-1]:
        stack.pop()
        count += 1

      # If stack not empty, can see one more person
(taller or equal)
      if stack:
        count += 1

      result[i] = count
      # Maintain decreasing stack
      while stack and heights[i] == stack[-1]:
        stack.pop()
      stack.append(heights[i])

    return result
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 3: Using deque

```
class Solution:
  def canSeePersonsCount(self, heights: List[int]) ->
List[int]:
    n = len(heights)
    result = [0] * n
    dq = deque()  # Store indices

    for i in range(n-1, -1, -1):
      # Remove people shorter than current (can see
them)
      count = 0
      while dq and heights[i] > heights[dq[0]]:
        dq.popleft()
        count += 1

      # If there's a taller or equal person, can see them
too
      if dq:
        count += 1

      result[i] = count
```

```
        # Maintain decreasing order
        while dq and heights[i] >= heights[dq[-1]]:
            dq.pop()
        dq.appendleft(i)

    return result
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 4: Brute Force (for comparison)

```
class Solution:
    def canSeePersonsCount(self, heights: List[int]) -> List[int]:
        n = len(heights)
        result = [0] * n

        for i in range(n):
            max_height = 0
            for j in range(i+1, n):
                # Person j is visible if they're taller than max_height so far
                if heights[j] > max_height:
                    result[i] += 1
                    max_height = heights[j]

                # Stop if we encounter someone taller than or equal to person i
                if heights[j] >= heights[i]:
                    break

    return result
```

Time Complexity: O(n²) in worst case
Space Complexity: O(1) additional space

HEAPS

215. Kth Largest Element in an Array

Given an integer array nums and an integer k, return *the* k_{th} *largest element in the array*.

Note that it is the k_{th} largest element in the sorted order, not the k_{th} distinct element.

Can you solve it without sorting?

Example 1:

Input: nums = [3,2,1,5,6,4], k = 2
Output: 5

Example 2:

Input: nums = [3,2,3,1,2,4,5,5,6], k = 4
Output: 4

Approach 1. Sorting

```
def findKthLargest(nums, k):
    nums.sort()
    return nums[-k]
```

Time Complexity: O(n log n)
Space Complexity: O(1) or O(n) depending on sort implementation

---

Approach 2. Min-Heap (size k)

```
def findKthLargest(nums, k):
    heap = []
    for num in nums:
        heapq.heappush(heap, num)
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0]
```

Time Complexity: O(n log k)
Space Complexity: O(k)

Approach 3. Max-Heap

```
def findKthLargest(nums, k):
    max_heap = [-num for num in nums]
    heapq.heapify(max_heap)
    for _ in range(k-1):
        heapq.heappop(max_heap)
    return -heapq.heappop(max_heap)
```

Time Complexity: O(n + k log n) for heapify + k pops
Space Complexity: O(n)

Approach 4. QuickSelect (Hoare's Selection)

```
def findKthLargest(nums, k):
    def quickselect(left, right, k_smallest):
        if left == right:
            return nums[left]

        pivot_index = random.randint(left, right)
        pivot_index = partition(left, right, pivot_index)

        if k_smallest == pivot_index:
            return nums[k_smallest]
        elif k_smallest < pivot_index:
            return quickselect(left, pivot_index - 1, k_smallest)
        else:
            return quickselect(pivot_index + 1, right, k_smallest)

    def partition(left, right, pivot_index):
        pivot_value = nums[pivot_index]
        nums[pivot_index], nums[right] = nums[right], nums[pivot_index]
        store_index = left

        for i in range(left, right):
            if nums[i] < pivot_value:
                nums[store_index], nums[i] = nums[i], nums[store_index]
                store_index += 1

        nums[right], nums[store_index] = nums[store_index], nums[right]
        return store_index

    return quickselect(0, len(nums) - 1, len(nums) - k)
```

Time Complexity: O(n) average, O(n²) worst-case
Space Complexity: O(1) iterative, O(log n) recursive

---

703. Kth Largest Element in a Stream

You are part of a university admissions office and need to keep track of the kth highest test score from applicants in real-time. This helps to determine cut-off marks for interviews and admissions dynamically as new applicants submit their scores.

You are tasked to implement a class which, for a given integer k, maintains a stream of test scores and continuously returns the kth highest test score **after** a new score has been submitted. More specifically, we are looking for the kth highest score in the sorted list of all scores.

Implement the KthLargest class:

- KthLargest(int k, int[] nums) Initializes the object with the integer k and the stream of test scores nums.
- int add(int val) Adds a new test score val to the stream and returns the element representing the k_{th} largest element in the pool of test scores so far.

Example 1:

Input:
["KthLargest", "add", "add", "add", "add", "add"]
[[3, [4, 5, 8, 2]], [3], [5], [10], [9], [4]]

Output: [null, 4, 5, 5, 8, 8]

Explanation:

KthLargest kthLargest = new KthLargest(3, [4, 5, 8, 2]);
kthLargest.add(3); // return 4
kthLargest.add(5); // return 5
kthLargest.add(10); // return 5
kthLargest.add(9); // return 8
kthLargest.add(4); // return 8

Example 2:

Input:
["KthLargest", "add", "add", "add", "add"]
[[4, [7, 7, 7, 7, 8, 3]], [2], [10], [9], [9]]

Output: [null, 7, 7, 7, 8]

Explanation:

KthLargest kthLargest = new KthLargest(4, [7, 7, 7, 7, 8, 3]);
kthLargest.add(2); // return 7
kthLargest.add(10); // return 7
kthLargest.add(9); // return 7
kthLargest.add(9); // return 8

Approach 1. Min-Heap (Optimal)

```
class KthLargest:
    def __init__(self, k, nums):
        self.k = k
        self.heap = []
        for num in nums:
            self.add(num)

    def add(self, val):
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0]
```

Time Complexity: O(log k) per add operation
Space Complexity: O(k)

Approach 2. BST/Ordered Set

```
class KthLargest:
    def __init__(self, k, nums):
        self.k = k
        self.sorted_list = SortedList(nums)

    def add(self, val):
        self.sorted_list.add(val)
        return self.sorted_list[-self.k]
```

Time Complexity: O(log n) per add operation
Space Complexity: O(n)

---

23. Merge k Sorted Lists

You are given an array of k linked-lists lists, each linked-list is sorted in ascending order.

Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked-lists are:
[
  1->4->5,
  1->3->4,
  2->6
]
merging them into one sorted linked list:
1->1->2->3->4->4->5->6

Example 2:

Input: lists = []
Output: []

Example 3:

Input: lists = [[]]
Output: []

Approach 1. Min-Heap (Priority Queue)

```
def mergeKLists(lists):
    heap = []

    # Push first node of each list
    for i, lst in enumerate(lists):
        if lst:
            heapq.heappush(heap, (lst.val, i, lst))

    dummy = ListNode(0)
    curr = dummy

    while heap:
        val, i, node = heapq.heappop(heap)
        curr.next = node
        curr = curr.next

        if node.next:
            heapq.heappush(heap, (node.next.val, i,
node.next))

    return dummy.next
```

Time Complexity: O(N log k) where N is total nodes
Space Complexity: O(k) for heap

Approach 2. Divide and Conquer (Merge Sort)

```
def mergeKLists(lists):
    def mergeTwoLists(l1, l2):
        dummy = ListNode(0)
        curr = dummy

        while l1 and l2:
            if l1.val < l2.val:
                curr.next = l1
                l1 = l1.next
            else:
                curr.next = l2
                l2 = l2.next
            curr = curr.next

        curr.next = l1 if l1 else l2
        return dummy.next

    if not lists:
        return None

    interval = 1
    while interval < len(lists):
        for i in range(0, len(lists) - interval, interval * 2):
            lists[i] = mergeTwoLists(lists[i], lists[i + interval])
        interval *= 2

    return lists[0]
```

Time Complexity: O(N log k)
Space Complexity: O(1) or O(log k) for recursion

Approach 3. Brute Force

```
def mergeKLists(lists):
    all_nodes = []
    for lst in lists:
        while lst:
            all_nodes.append(lst)
            lst = lst.next

    all_nodes.sort(key=lambda x: x.val)

    dummy = ListNode(0)
    curr = dummy
    for node in all_nodes:
        curr.next = node
        curr = curr.next

    return dummy.next
```

Time Complexity: O(N log N)
Space Complexity: O(N)

---

295. Find Median from Data Stream

The **median** is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for arr = [2,3,4], the median is 3.
- For example, for arr = [2,3], the median is (2 + 3) / 2 = 2.5.

Implement the MedianFinder class:

- MedianFinder() initializes the MedianFinder object.
- void addNum(int num) adds the integer num from the data stream to the data structure.
- double findMedian() returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

Input
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
[[], [1], [2], [], [3], []]
Output
[null, null, null, 1.5, null, 2.0]

Explanation
MedianFinder medianFinder = new MedianFinder();
medianFinder.addNum(1); // arr = [1]
medianFinder.addNum(2); // arr = [1, 2]
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)
medianFinder.addNum(3); // arr[1, 2, 3]
medianFinder.findMedian(); // return 2.0

Approach 1. Two Heaps (Optimal)

```
class MedianFinder:
    def __init__(self):
        self.max_heap = []  # left half (invert values)
        self.min_heap = []  # right half

    def addNum(self, num):
        # Push to max heap first
        heapq.heappush(self.max_heap, -num)

        # Ensure max_heap <= min_heap
        if self.max_heap and self.min_heap and
(-self.max_heap[0] > self.min_heap[0]):
            heapq.heappush(self.min_heap,
-heapq.heappop(self.max_heap))

        # Balance sizes
        if len(self.max_heap) > len(self.min_heap) + 1:
            heapq.heappush(self.min_heap,
-heapq.heappop(self.max_heap))
        elif len(self.min_heap) > len(self.max_heap):
            heapq.heappush(self.max_heap,
-heapq.heappop(self.min_heap))

    def findMedian(self):
        if len(self.max_heap) > len(self.min_heap):
            return -self.max_heap[0]
        return (-self.max_heap[0] + self.min_heap[0]) / 2
```

Time Complexity: O(log n) per addNum, O(1) for findMedian
Space Complexity: O(n)

Approach 2. Insertion Sort

```
class MedianFinder:
    def __init__(self):
        self.nums = []

    def addNum(self, num):
        # Insert in sorted position
        import bisect
        bisect.insort(self.nums, num)

    def findMedian(self):
        n = len(self.nums)
        if n % 2 == 1:
            return self.nums[n // 2]
        return (self.nums[n // 2 - 1] + self.nums[n // 2]) / 2
```

Time Complexity: O(n) per addNum (insertion), O(1) for findMedian
Space Complexity: O(n)

---

347. Top K Frequent Elements

Given an integer array nums and an integer k, return *the* k *most frequent elements*. You may return the answer in **any order**.

Example 1:

Input: nums = [1,1,1,2,2,3], k = 2

Output: [1,2]

Example 2:

Input: nums = [1], k = 1

Output: [1]

Example 3:

Input: nums = [1,2,1,2,1,2,3,1,3,2], k = 2

Output: [1,2]

Approach 1. Bucket Sort

```
def topKFrequent(nums, k):
    freq_map = {}
    for num in nums:
        freq_map[num] = freq_map.get(num, 0) + 1

    buckets = [[] for _ in range(len(nums) + 1)]
    for num, freq in freq_map.items():
        buckets[freq].append(num)

    result = []
    for i in range(len(buckets) - 1, -1, -1):
        result.extend(buckets[i])
        if len(result) >= k:
            break

    return result[:k]
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2. Min-Heap

```
def topKFrequent(nums, k):
    freq = Counter(nums)
    heap = []

    for num, count in freq.items():
        heapq.heappush(heap, (count, num))
        if len(heap) > k:
            heapq.heappop(heap)

    return [num for _, num in heap]
```

Time Complexity: O(n log k)
Space Complexity: O(n + k)

Approach 3. QuickSelect

```
def topKFrequent(nums, k):
    freq = Counter(nums)
    unique = list(freq.keys())

    def partition(left, right, pivot_index):
        pivot_freq = freq[unique[pivot_index]]
        unique[pivot_index], unique[right] = unique[right],
unique[pivot_index]

        store_index = left
        for i in range(left, right):
            if freq[unique[i]] < pivot_freq:
                unique[store_index], unique[i] = unique[i],
unique[store_index]
                store_index += 1

        unique[right], unique[store_index] =
unique[store_index], unique[right]
        return store_index

    def quickselect(left, right, k_smallest):
        if left == right:
            return

        pivot_index = random.randint(left, right)
        pivot_index = partition(left, right, pivot_index)

        if k_smallest == pivot_index:
            return
        elif k_smallest < pivot_index:
            quickselect(left, pivot_index - 1, k_smallest)
        else:
            quickselect(pivot_index + 1, right, k_smallest)

    n = len(unique)
    quickselect(0, n - 1, n - k)
    return unique[n - k:]
```

Time Complexity: O(n) average, O(n²) worst-case
Space Complexity: O(n)

---

692. Top K Frequent Words

Given an array of strings words and an integer k, return *the* k *most frequent strings*.

Return the answer **sorted** by **the frequency** from highest to lowest. Sort the words with the same frequency by their **lexicographical order**.

Example 1:

Input: words = ["i","love","leetcode","i","love","coding"], k = 2
Output: ["i","love"]
Explanation: "i" and "love" are the two most frequent words. Note that "i" comes before "love" due to a lower alphabetical order.

Example 2:

Input: words = ["the","day","is","sunny","the","the","the","sunny","is","is"], k = 4
Output: ["the","is","sunny","day"]
Explanation: "the", "is", "sunny" and "day" are the four most frequent words, with the number of occurrence being 4, 3, 2 and 1 respectively.

Approach 1. Min-Heap with Custom Comparator

```
def topKFrequent(words, k):
    freq = Counter(words)

    heap = []
    for word, count in freq.items():
        heapq.heappush(heap, (-count, word))

    return [heapq.heappop(heap)[1] for _ in range(k)]
```

Time Complexity: O(n log k)
Space Complexity: O(n)

Approach 2. Bucket Sort + Trie/BST

```
def topKFrequent(words, k):
    freq = Counter(words)
    buckets = defaultdict(list)

    for word, count in freq.items():
        buckets[count].append(word)

    result = []
    for count in sorted(buckets.keys(), reverse=True):
        result.extend(sorted(buckets[count]))
        if len(result) >= k:
            break

    return result[:k]
```

Time Complexity: O(n + m log m) where m is unique frequencies
Space Complexity: O(n)

253. Meeting Rooms II

Given two arrays start[] and end[] such that start[i] is the starting time of ith meeting and end[i] is the ending time of ith meeting. Return the minimum number of rooms required to attend all meetings.

Note: A person can also attend a meeting if it's starting time is same as the previous meeting's ending time.

Examples:

Input: start[] = [1, 10, 7], end[] = [4, 15, 10]
Output: 1
Explanation: Since all the meetings are held at different times, it is possible to attend all the meetings in a single room.
Input: start[] = [2, 9, 6], end[] = [4, 12, 10]
Output: 2
Explanation: 1st and 2nd meetings at one room but for 3rd meeting one another room required.

Approach 1. Chronological Ordering

```
def minMeetingRooms(intervals):
    if not intervals:
        return 0

    start_times = sorted([i[0] for i in intervals])
    end_times = sorted([i[1] for i in intervals])

    rooms = 0
    end_ptr = 0

    for start in start_times:
        if start < end_times[end_ptr]:
            rooms += 1
        else:
            end_ptr += 1

    return rooms
```

Time Complexity: O(n log n)
Space Complexity: O(n)

Approach 2. Min-Heap

```
def minMeetingRooms(intervals):
    if not intervals:
        return 0

    intervals.sort(key=lambda x: x[0])
    heap = []  # stores end times

    for interval in intervals:
        if heap and interval[0] >= heap[0]:
            heapq.heappop(heap)
        heapq.heappush(heap, interval[1])

    return len(heap)
```

Time Complexity: O(n log n)
Space Complexity: O(n)

---

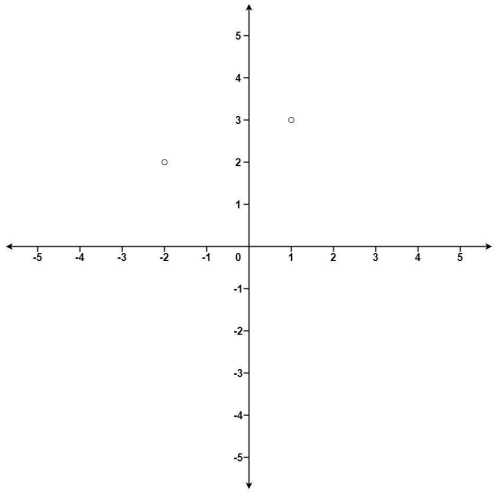
973. K Closest Points to Origin

Given an array of points where points[i] = [x_i, y_i] represents a point on the **X-Y** plane and an integer k, return the k closest points to the origin (0, 0).

The distance between two points on the **X-Y** plane is the Euclidean distance (i.e., $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$).

You may return the answer in **any order**. The answer is **guaranteed** to be **unique** (except for the order that it is in).

Example 1:



Input: points = [[1,3],[-2,2]], k = 1
Output: [[-2,2]]
Explanation:
The distance between (1, 3) and the origin is sqrt(10).
The distance between (-2, 2) and the origin is sqrt(8).
Since sqrt(8) < sqrt(10), (-2, 2) is closer to the origin.
We only want the closest k = 1 points from the origin, so the answer is just [[-2,2]].

Example 2:

Input: points = [[3,3],[5,-1],[-2,4]], k = 2
Output: [[3,3],[-2,4]]
Explanation: The answer [[-2,4],[3,3]] would also be accepted.

Approach 1. Max-Heap

```
def kClosest(points, k):
    heap = []

    for point in points:
        dist = point[0]2 + point[1]2
        heapq.heappush(heap, (-dist, point))
        if len(heap) > k:
            heapq.heappop(heap)

    return [point for _, point in heap]
```

Time Complexity: O(n log k)
Space Complexity: O(k)

Approach 2. QuickSelect

```
def kClosest(points, k):
    def dist(point):
        return point[0]2 + point[1]2

    def quickselect(left, right, k_smallest):
        if left == right:
            return points[:k]

        pivot_index = random.randint(left, right)
        pivot_index = partition(left, right, pivot_index)

        if k_smallest == pivot_index:
            return points[:k]
        elif k_smallest < pivot_index:
            return quickselect(left, pivot_index - 1, k_smallest)
        else:
            return quickselect(pivot_index + 1, right,
k_smallest)

    def partition(left, right, pivot_index):
        pivot_dist = dist(points[pivot_index])
        points[pivot_index], points[right] = points[right],
points[pivot_index]
        store_index = left

        for i in range(left, right):
            if dist(points[i]) < pivot_dist:
                points[store_index], points[i] = points[i],
points[store_index]
                store_index += 1

        points[right], points[store_index] =
points[store_index], points[right]
        return store_index

    return quickselect(0, len(points) - 1, k)
```

Time Complexity: O(n) average, O(n^2) worst
Space Complexity: O(1) or O(log n) recursive

767. Reorganize String

Given a string s, rearrange the characters of s so that any two adjacent characters are not the same.

Return *any possible rearrangement of* s *or return* "" *if not possible*.

Example 1:

Input: s = "aab"
Output: "aba"

Example 2:

Input: s = "aaab"
Output: ""

Approach 1. Max-Heap + Greedy

```
def reorganizeString(s):
    freq = Counter(s)
    heap = [(-count, char) for char, count in freq.items()]
    heapq.heapify(heap)

    result = []
    prev = None

    while heap or prev:
        if not heap and prev:
            return ""

        count, char = heapq.heappop(heap)
        result.append(char)

        if prev:
            heapq.heappush(heap, prev)
            prev = None

        if count + 1 < 0:
            prev = (count + 1, char)

    return "".join(result)
```

Time Complexity: O(n log A) where A is alphabet size (26)
Space Complexity: O(A)

Approach 2. Sort by Frequency

```
def reorganizeString(s):
    freq = Counter(s)
    sorted_chars = sorted(freq.keys(), key=lambda x:
-freq[x])

    if freq[sorted_chars[0]] > (len(s) + 1) // 2:
        return ""

    result = ['] * len(s)
    index = 0

    for char in sorted_chars:
        count = freq[char]
        for _ in range(count):
            if index >= len(s):
                index = 1
            result[index] = char
            index += 2

    return "".join(result)
```

Time Complexity: O(n + A log A)
Space Complexity: O(n)

---

621. Task Scheduler

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of **at least** n intervals between two tasks with the same label.

Return the **minimum** number of CPU intervals required to complete all tasks.

Example 1:

Input: tasks = ["A","A","A","B","B","B"], n = 2

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.

Example 2:

Input: tasks = ["A","C","A","B","D","B"], n = 1

Output: 6

Explanation: A possible sequence is: A -> B -> C -> D -> A -> B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: tasks = ["A","A","A", "B","B","B"], n = 3

Output: 10

Explanation: A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Approach 1. Max-Heap + Queue

```
def leastInterval(tasks, n):
    freq = Counter(tasks)
    heap = [-count for count in freq.values()]
    heapq.heapify(heap)

    time = 0
    queue = deque()  # (count, available_time)

    while heap or queue:
        time += 1

        if heap:
            count = heapq.heappop(heap) + 1
            if count < 0:
                queue.append((count, time + n))

        if queue and queue[0][1] == Time Complexity:
            heapq.heappush(heap, queue.popleft()[0])

    return time
```

Time Complexity: O(n * m) where m is idle time
Space Complexity: O(26) = O(1)

Approach 2. Mathematical Formula

```
def leastInterval(tasks, n):
    freq = Counter(tasks)
    max_freq = max(freq.values())

    max_count = sum(1 for count in freq.values() if count
== max_freq)

    return max(len(tasks), (max_freq - 1) * (n + 1) +
max_count)
```

Time Complexity: O(n)
Space Complexity: O(26) = O(1)

---

373. Find K Pairs with Smallest Sums

You are given two integer arrays nums1 and nums2 sorted in **non-decreasing order** and an integer k.

Define a pair (u, v) which consists of one element from the first array and one element from the second array.

Return *the* k *pairs* $(u_1, v_1), (u_2, v_2), \ldots, (u_k, v_k)$ *with the smallest sums*.

Example 1:

Input: nums1 = [1,7,11], nums2 = [2,4,6], k = 3
Output: [[1,2],[1,4],[1,6]]
Explanation: The first 3 pairs are returned from the sequence: [1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6],[11,4],[11,6]

Example 2:

Input: nums1 = [1,1,2], nums2 = [1,2,3], k = 2
Output: [[1,1],[1,1]]
Explanation: The first 2 pairs are returned from the sequence: [1,1],[1,1],[1,2],[2,1],[1,2],[2,2],[1,3],[1,3],[2,3]

Approach 1. Min-Heap

```
def kSmallestPairs(nums1, nums2, k):
    if not nums1 or not nums2:
        return []

    heap = []
    for i in range(min(k, len(nums1))):
        heapq.heappush(heap, (nums1[i] + nums2[0], i, 0))

    result = []
    while heap and len(result) < k:
        _, i, j = heapq.heappop(heap)
        result.append([nums1[i], nums2[j]])

        if j + 1 < len(nums2):
            heapq.heappush(heap, (nums1[i] + nums2[j + 1], i,
j + 1))

    return result
```

Time Complexity: O(k log k)
Space Complexity: O(k)

Approach 2. Binary Search on Sum Value

```
def kSmallestPairs(nums1, nums2, k):
    def count_pairs(target):
        count = 0
        j = len(nums2) - 1
        for num1 in nums1:
            while j >= 0 and num1 + nums2[j] > target:
                j -= 1
            count += j + 1
        return count

    low = nums1[0] + nums2[0]
    high = nums1[-1] + nums2[-1]

    while low < high:
        mid = (low + high) // 2
        if count_pairs(mid) < k:
            low = mid + 1
        else:
            high = mid

    result = []
    for num1 in nums1:
        for num2 in nums2:
            if num1 + num2 <= low:
                result.append([num1, num2])
                if len(result) == k:
                    return result
            else:
                break

    return result[:k]
```

Time Complexity: O((m+n) log D) where D is sum range
Space Complexity: O(k)

GREEDY

121. Best Time to Buy and Sell Stock

You are given an array prices where prices[i] is the price of a given stock on the i_{th} day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return 0.

Example 1:

Input: prices = [7,1,5,3,6,4]
Output: 5
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

Input: prices = [7,6,4,3,1]
Output: 0
Explanation: In this case, no transactions are done and the max profit = 0.

Solution 1: Brute Force

```
def maxProfit(prices):
    max_profit = 0
    n = len(prices)
    for i in range(n):
        for j in range(i + 1, n):
            profit = prices[j] - prices[i]
            max_profit = max(max_profit, profit)
    return max_profit
```

Time Complexity: O(n²) | Space Complexity: O(1)
Idea: Check every possible buy-sell pair.

Solution 2: One Pass

```
def maxProfit(prices):
    min_price = float('inf')
    max_profit = 0
    for price in prices:
        min_price = min(min_price, price)
        max_profit = max(max_profit, price - min_price)
    return max_profit
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Track minimum price seen so far and calculate profit if selling at current price

---

Solution 3: Kadane's Algorithm

```
def maxProfit(prices):
    max_current = 0
    max_so_far = 0
    for i in range(1, len(prices)):
        max_current = max(0, max_current + prices[i] - prices[i-1])
        max_so_far = max(max_so_far, max_current)
    return max_so_far
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Maximum subarray sum of price differences.

Solution 4: Dynamic Programming

```
def maxProfit(prices):
    n = len(prices)
    if n < 2: return 0

    dp = [0] * n
    min_price = prices[0]

    for i in range(1, n):
        min_price = min(min_price, prices[i])
        dp[i] = max(dp[i-1], prices[i] - min_price)

    return dp[-1]
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: DP where dp[i] = max profit up to day i.

---

122. Best Time to Buy and Sell Stock II

You are given an integer array prices where prices[i] is the price of a given stock on the i_{th} day.

On each day, you may decide to buy and/or sell the stock. You can only hold **at most one** share of the stock at any time. However, you can sell and buy the stock multiple times on the **same day**, ensuring you never hold more than one share of the stock.

Find and return *the **maximum** profit you can achieve*.

Example 1:

Input: prices = [7,1,5,3,6,4]
Output: 7
Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = 5-1 = 4.
Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = 6-3 = 3.
Total profit is 4 + 3 = 7.

Example 2:

Input: prices = [1,2,3,4,5]
Output: 4
Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.
Total profit is 4.

Example 3:

Input: prices = [7,6,4,3,1]
Output: 0
Explanation: There is no way to make a positive profit, so we never buy the stock to achieve the maximum profit of 0.

Solution 1: Valley-Peak Approach

```
def maxProfit(prices):
    i = 0
    n = len(prices)
    max_profit = 0

    while i < n - 1:
        # Find valley (local minimum)
        while i < n - 1 and prices[i] >= prices[i + 1]:
            i += 1
        valley = prices[i]

        # Find peak (local maximum)
        while i < n - 1 and prices[i] <= prices[i + 1]:
            i += 1
        peak = prices[i]

        max_profit += peak - valley

    return max_profit
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Buy at valleys and sell at peaks.

Solution 2: Simple One Pass

```
def maxProfit(prices):
    max_profit = 0
    for i in range(1, len(prices)):
        if prices[i] > prices[i-1]:
            max_profit += prices[i] - prices[i-1]
    return max_profit
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Sum all positive differences between consecutive days.

Solution 3: State Machine DP

```
def maxProfit(prices):
    cash = 0  # max profit with no stock
    hold = -float('inf')  # max profit with stock

    for price in prices:
        prev_cash = cash
        cash = max(cash, hold + price)  # sell or keep cash
        hold = max(hold, prev_cash - price)  # buy or keep
holding

    return cash
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Track best profit in two states: holding stock or not.

Solution 4: Recursion with Memoization

```
def maxProfit(prices):
    from functools import lru_cache

    @lru_cache(None)
    def dfs(i, holding):
        if i >= len(prices):
            return 0

        # Skip current day
        skip = dfs(i + 1, holding)

        if holding:
            # Sell today
            sell = dfs(i + 1, False) + prices[i]
            return max(skip, sell)
        else:
            # Buy today
            buy = dfs(i + 1, True) - prices[i]
            return max(skip, buy)

    return dfs(0, False)
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Memoized DFS exploring buy/sell/skip decisions.

---

55. Jump Game

You are given an integer array nums. You are initially positioned at the array's **first index**, and each element in the array represents your maximum jump length at that position.

Return true *if you can reach the last index, or* false *otherwise*.

Example 1:

Input: nums = [2,3,1,1,4]
Output: true
Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: nums = [3,2,1,0,4]
Output: false
Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.

Solution 1: Greedy

```
def canJump(nums):
    max_reach = 0
    for i in range(len(nums)):
        if i > max_reach:
            return False
        max_reach = max(max_reach, i + nums[i])
        if max_reach >= len(nums) - 1:
            return True
    return True
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Track furthest index reachable so far.

Solution 2: Backward Greedy

```
def canJump(nums):
    last_pos = len(nums) - 1
    for i in range(len(nums) - 2, -1, -1):
        if i + nums[i] >= last_pos:
            last_pos = i
    return last_pos == 0
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Work backwards checking if previous positions can reach current good position.

Solution 3: DP with Memoization

```
def canJump(nums):
    from functools import lru_cache

    @lru_cache(None)
    def dfs(i):
        if i >= len(nums) - 1:
            return True

        max_jump = min(i + nums[i], len(nums) - 1)
        for j in range(i + 1, max_jump + 1):
            if dfs(j):
                return True
        return False

    return dfs(0)
```

Time Complexity: O(n²) | Space Complexity: O(n)
Idea: Memoized DFS trying all possible jumps.

Solution 4: BFS

```
def canJump(nums):
    from collections import deque
    n = len(nums)
    visited = [False] * n
    queue = deque([0])
    visited[0] = True

    while queue:
        i = queue.popleft()
        if i == n - 1:
            return True

        max_jump = min(i + nums[i], n - 1)
        for j in range(i + 1, max_jump + 1):
            if not visited[j]:
                visited[j] = True
                queue.append(j)

    return False
```

Time Complexity: O(n²) | Space Complexity: O(n)
Idea: BFS exploring reachable indices.

---

45. Jump Game II

You are given a **0-indexed** array of integers nums of length n. You are initially positioned at index 0.

Each element nums[i] represents the maximum length of a forward jump from index i. In other words, if you are at index i, you can jump to any index (i + j) where:

- 0 <= j <= nums[i] and
- i + j < n

Return *the minimum number of jumps to reach index* n - 1. The test cases are generated such that you can reach index n - 1.

Example 1:

Input: nums = [2,3,1,1,4]
Output: 2
Explanation: The minimum number of jumps to reach the last index is 2. Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: nums = [2,3,0,1,4]
Output: 2

Solution 1: BFS/Level Order

```
def jump(nums):
    if len(nums) <= 1:
        return 0

    jumps = 0
    current_end = 0
    farthest = 0

    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])

        if i == current_end:
            jumps += 1
            current_end = farthest

            if current_end >= len(nums) - 1:
                break

    return jumps
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: BFS-like level traversal where each "level" is one jump's reach.

Solution 2: Dynamic Programming

```
def jump(nums):
    n = len(nums)
    dp = [float('inf')] * n
    dp[0] = 0

    for i in range(n):
        max_jump = min(i + nums[i], n - 1)
        for j in range(i + 1, max_jump + 1):
            dp[j] = min(dp[j], dp[i] + 1)

    return dp[-1]
```

Time Complexity: O(n²) | Space Complexity: O(n)
Idea: DP where dp[i] = min jumps to reach index i.

Solution 3: Recursion with Memoization

```
def jump(nums):
    from functools import lru_cache

    @lru_cache(None)
    def dfs(i):
        if i >= len(nums) - 1:
            return 0

        min_jumps = float('inf')
        max_jump = min(i + nums[i], len(nums) - 1)

        for j in range(i + 1, max_jump + 1):
            min_jumps = min(min_jumps, 1 + dfs(j))

        return min_jumps

    return dfs(0)
```

Time Complexity: O(n²) | Space Complexity: O(n)
Idea: Memoized DFS exploring all jump paths.

Solution 4: Bidirectional BFS

```
def jump(nums):
    from collections import deque

    n = len(nums)
    if n <= 1:
        return 0

    visited = [False] * n
    queue = deque([0])
    visited[0] = True
    jumps = 0

    while queue:
        size = len(queue)
        for _ in range(size):
            i = queue.popleft()
            if i == n - 1:
                return jumps

            max_jump = min(i + nums[i], n - 1)
            for j in range(max_jump, i, -1):  # Try larger jumps first
                if not visited[j]:
                    visited[j] = True
                    queue.append(j)

        jumps += 1

    return -1
```

Time Complexity: O(n²) | Space Complexity: O(n)
Idea: BFS that finds shortest path (minimum jumps).

134. Gas Station

There are n gas stations along a circular route, where the amount of gas at the i_{th} station is gas[i].

You have a car with an unlimited gas tank and it costs cost[i] of gas to travel from the i_{th} station to its next $(i + 1)_{th}$ station. You begin the journey with an empty tank at one of the gas stations.

Given two integer arrays gas and cost, return *the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return* -1. If there exists a solution, it is **guaranteed** to be **unique**.

Example 1:

Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]
Output: 3
Explanation:
Start at station 3 (index 3) and fill up with 4 unit of gas. Your tank = 0 + 4 = 4
Travel to station 4. Your tank = 4 - 1 + 5 = 8
Travel to station 0. Your tank = 8 - 2 + 1 = 7
Travel to station 1. Your tank = 7 - 3 + 2 = 6
Travel to station 2. Your tank = 6 - 4 + 3 = 5
Travel to station 3. The cost is 5. Your gas is just enough to travel back to station 3.
Therefore, return 3 as the starting index.

Example 2:

Input: gas = [2,3,4], cost = [3,4,3]
Output: -1
Explanation:
You can't start at station 0 or 1, as there is not enough gas to travel to the next station.
Let's start at station 2 and fill up with 4 unit of gas. Your tank = 0 + 4 = 4
Travel to station 0. Your tank = 4 - 3 + 2 = 3
Travel to station 1. Your tank = 3 - 3 + 3 = 3
You cannot travel back to station 2, as it requires 4 unit of gas but you only have 3.

Solution 1: Brute Force

```
def canCompleteCircuit(gas, cost):
    n = len(gas)

    for start in range(n):
        tank = 0
        for i in range(n):
            idx = (start + i) % n
            tank += gas[idx] - cost[idx]
            if tank < 0:
                break
        if tank >= 0:
            return start
    return -1
```

Time Complexity: O(n²) | Space Complexity: O(1)
Idea: Try every station as starting point, simulate full circuit.

Solution 2: Greedy (One Pass)

```
def canCompleteCircuit(gas, cost):
    total_tank = 0
    curr_tank = 0
    start = 0

    for i in range(len(gas)):
        total_tank += gas[i] - cost[i]
        curr_tank += gas[i] - cost[i]

        if curr_tank < 0:
            start = i + 1
            curr_tank = 0

    return start if total_tank >= 0 else -1
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: If total gas >= total cost, solution exists. Reset start when tank goes negative.

Solution 3: Two-Pointer Greedy

```
def canCompleteCircuit(gas, cost):
    n = len(gas)
    start, end = 0, 0
    tank = 0

    for _ in range(n):
        tank += gas[end] - cost[end]
        end = (end + 1) % n

        while tank < 0 and start != end:
            tank -= gas[start] - cost[start]
            start = (start + 1) % n

        if start == end and tank >= 0:
            return start

    return -1
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Expand end until tank negative, then shrink from start.

Solution 4: Prefix Sum

```
def canCompleteCircuit(gas, cost):
    n = len(gas)
    net = [gas[i] - cost[i] for i in range(n)]

    # Check if total is negative
    if sum(net) < 0:
        return -1

    # Find minimum prefix sum
    min_prefix = float('inf')
    prefix = 0
    start = 0

    for i in range(n):
        prefix += net[i]
        if prefix < min_prefix:
            min_prefix = prefix
            start = (i + 1) % n

    return start
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Start after minimum prefix sum point (where cumulative deficit is maximum).

---

135. Candy

There are n children standing in a line. Each child is assigned a rating value given in the integer array ratings.

You are giving candies to these children subjected to the following requirements:

- Each child must have at least one candy.
- Children with a higher rating get more candies than their neighbors.

Return *the minimum number of candies you need to have to distribute the candies to the children*.

Example 1:

Input: ratings = [1,0,2]
Output: 5
Explanation: You can allocate to the first, second and third child with 2, 1, 2 candies respectively.

Example 2:

Input: ratings = [1,2,2]
Output: 4
Explanation: You can allocate to the first, second and third child with 1, 2, 1 candies respectively.
The third child gets 1 candy because it satisfies the above two conditions.

Solution 1: Two-Pass Greedy

```
def candy(ratings):
    n = len(ratings)
    candies = [1] * n

    # Left to right
    for i in range(1, n):
        if ratings[i] > ratings[i-1]:
            candies[i] = candies[i-1] + 1

    # Right to left
    for i in range(n-2, -1, -1):
        if ratings[i] > ratings[i+1]:
            candies[i] = max(candies[i], candies[i+1] + 1)

    return sum(candies)
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: First pass ensures left neighbors satisfied, second pass ensures right neighbors.

Solution 2: One-Pass with Peaks and Valleys

```
def candy(ratings):
    if not ratings:
        return 0

    n = len(ratings)
    total = 1
    up = down = peak = 0

    for i in range(1, n):
        if ratings[i] > ratings[i-1]:
            up += 1
            down = 0
            peak = up
            total += 1 + up

        elif ratings[i] == ratings[i-1]:
            up = down = peak = 0
            total += 1

        else:
            down += 1
            up = 0
            total += 1 + down
            if peak >= down:
                total -= 1

    return total
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Track up/down sequences and adjust for peaks.

Solution 3: Slopes Counting

```
def candy(ratings):
    n = len(ratings)
    if n <= 1:
        return n

    def count(n):
        return n * (n + 1) // 2

    candies = 0
    up = down = 0
    old_slope = 0

    for i in range(1, n):
        new_slope = 0
        if ratings[i] > ratings[i-1]:
            new_slope = 1
        elif ratings[i] < ratings[i-1]:
            new_slope = -1

        if (old_slope > 0 and new_slope == 0) or (old_slope <
0 and new_slope >= 0):
            candies += count(up) + count(down) + max(up,
down)
            up = down = 0

        if new_slope > 0:
            up += 1
        elif new_slope < 0:
            down += 1
        else:
            candies += 1

        old_slope = new_slope

    candies += count(up) + count(down) + max(up, down) +
1
    return candies
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Mathematical approach counting arithmetic sequences for slopes.

Solution 4: BFS/Topological Sort

```
def candy(ratings):
    from collections import deque

    n = len(ratings)
    indegree = [0] * n
    candies = [1] * n

    # Build graph: higher rating points to lower rating
    for i in range(1, n):
        if ratings[i] > ratings[i-1]:
            indegree[i] += 1
        elif ratings[i] < ratings[i-1]:
            indegree[i-1] += 1

    # Topological sort
    queue = deque([i for i in range(n) if indegree[i] == 0])

    while queue:
        node = queue.popleft()

        # Check left neighbor
        if node > 0 and ratings[node] > ratings[node-1]:
            candies[node] = max(candies[node],
candies[node-1] + 1)
            indegree[node-1] -= 1
            if indegree[node-1] == 0:
                queue.append(node-1)

        # Check right neighbor
        if node < n-1 and ratings[node] > ratings[node+1]:
            candies[node] = max(candies[node],
candies[node+1] + 1)
            indegree[node+1] -= 1
            if indegree[node+1] == 0:
                queue.append(node+1)

    return sum(candies)
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: Treat as directed graph, process nodes with no constraints first.

53. Maximum Subarray (Kadane's)

Given an integer array nums, find the subarray with the largest sum, and return *its sum*.

Example 1:

Input: nums = [-2,1,-3,4,-1,2,1,-5,4]
Output: 6
Explanation: The subarray [4,-1,2,1] has the largest sum 6.

Example 2:

Input: nums = [1]
Output: 1
Explanation: The subarray [1] has the largest sum 1.

Example 3:

Input: nums = [5,4,-1,7,8]
Output: 23
Explanation: The subarray [5,4,-1,7,8] has the largest sum 23.

Solution 1: Kadane's Algorithm

```
def maxSubArray(nums):
    max_ending_here = nums[0]
    max_so_far = nums[0]

    for num in nums[1:]:
        max_ending_here = max(num, max_ending_here +
num)
        max_so_far = max(max_so_far, max_ending_here)

    return max_so_far
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: At each position, either extend previous subarray or start new one.

Solution 2: Divide and Conquer

```
def maxSubArray(nums):
    def divide_conquer(left, right):
        if left == right:
            return nums[left]

        mid = (left + right) // 2

        # Max in left half, right half, or crossing mid
        left_max = divide_conquer(left, mid)
        right_max = divide_conquer(mid + 1, right)

        # Cross sum
        left_sum = right_sum = float('-inf')
        curr_sum = 0
        for i in range(mid, left - 1, -1):
            curr_sum += nums[i]
            left_sum = max(left_sum, curr_sum)

        curr_sum = 0
        for i in range(mid + 1, right + 1):
            curr_sum += nums[i]
            right_sum = max(right_sum, curr_sum)

        cross_max = left_sum + right_sum

        return max(left_max, right_max, cross_max)

    return divide_conquer(0, len(nums) - 1)
```

Time Complexity: O(n log n) | Space Complexity: O(log n)
Idea: Recursively find max in left, right, and crossing subarrays.

Solution 3: Dynamic Programming

```
def maxSubArray(nums):
    n = len(nums)
    dp = [0] * n
    dp[0] = nums[0]

    for i in range(1, n):
        dp[i] = max(nums[i], dp[i-1] + nums[i])

    return max(dp)
```

Time Complexity: O(n) | Space Complexity: O(n)
Idea: dp[i] = maximum sum ending at position i.

Solution 4: Prefix Sum with Min Prefix

```
def maxSubArray(nums):
    prefix_sum = 0
    min_prefix = 0
    max_sum = float('-inf')

    for num in nums:
        prefix_sum += num
        max_sum = max(max_sum, prefix_sum - min_prefix)
        min_prefix = min(min_prefix, prefix_sum)

    return max_sum
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Maximum subarray sum = max(prefix[i] - min_prefix[j]) where j < i.

---

860. Lemonade Change

At a lemonade stand, each lemonade costs $5. Customers are standing in a queue to buy from you and order one at a time (in the order specified by bills). Each customer will only buy one lemonade and pay with either a $5, $10, or $20 bill. You must provide the correct change to each customer so that the net transaction is that the customer pays $5.

Note that you do not have any change in hand at first.

Given an integer array bills where bills[i] is the bill the i_{th} customer pays, return true *if you can provide every customer with the correct change, or* false *otherwise*.

Example 1:

Input: bills = [5,5,5,10,20]
Output: true
Explanation:
From the first 3 customers, we collect three $5 bills in order.
From the fourth customer, we collect a $10 bill and give back a $5.
From the fifth customer, we give a $10 bill and a $5 bill.
Since all customers got correct change, we output true.

Example 2:

Input: bills = [5,5,10,10,20]
Output: false
Explanation:
From the first two customers in order, we collect two $5 bills.
For the next two customers in order, we collect a $10 bill and give back a $5 bill.
For the last customer, we can not give the change of $15 back because we only have two $10 bills.
Since not every customer received the correct change, the answer is false.

Solution 1: Greedy with Counters

```
def lemonadeChange(bills):
    five = ten = 0

    for bill in bills:
        if bill == 5:
            five += 1
        elif bill == 10:
            if five == 0:
                return False
            five -= 1
            ten += 1
        else:  # bill == 20
            if ten > 0 and five > 0:
                ten -= 1
                five -= 1
            elif five >= 3:
                five -= 3
            else:
                return False

    return True
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Always give $10 + $5 change for $20 when possible (greedy choice).

Solution 2: Simulation with Queue

```
def lemonadeChange(bills):
    from collections import deque

    change = []

    for bill in bills:
        change.append(bill)
        change.sort()  # Keep sorted

        if bill == 10:
            if 5 in change:
                change.remove(5)
            else:
                return False

        elif bill == 20:
            # Try 10 + 5 first
            if 10 in change and 5 in change:
                change.remove(10)
                change.remove(5)
            elif change.count(5) >= 3:
                for _ in range(3):
                    change.remove(5)
            else:
                return False

    return True
```

Time Complexity: O(n²) | Space Complexity: O(n)
Idea: Simulate actual change giving, trying optimal combinations.

Solution 3: State Machine

```
def lemonadeChange(bills):
    state = {5: 0, 10: 0, 20: 0}

    for bill in bills:
        state[bill] += 1

        change_needed = bill - 5

        while change_needed >= 20 and state[20] > 0:
            state[20] -= 1
            change_needed -= 20

        while change_needed >= 10 and state[10] > 0:
            state[10] -= 1
            change_needed -= 10

        while change_needed >= 5 and state[5] > 0:
            state[5] -= 1
            change_needed -= 5

        if change_needed > 0:
            return False

    return True
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Track bill counts, always use larger bills first when making change.

Solution 4: Backtracking/DFS

```
def lemonadeChange(bills):
    from functools import lru_cache

    @lru_cache(None)
    def dfs(i, fives, tens):
        if i == len(bills):
            return True

        bill = bills[i]

        if bill == 5:
            return dfs(i + 1, fives + 1, tens)

        elif bill == 10:
            if fives > 0:
                return dfs(i + 1, fives - 1, tens + 1)
            return False

        else:  # bill == 20
            # Option 1: 10 + 5
            if tens > 0 and fives > 0:
                if dfs(i + 1, fives - 1, tens - 1):
                    return True

            # Option 2: 5 + 5 + 5
            if fives >= 3:
                if dfs(i + 1, fives - 3, tens):
                    return True

            return False

    return dfs(0, 0, 0)
```

Time Complexity: O(2^n) | Space Complexity: O(n)
Idea: Explore all possible change-giving strategies (theoretical, not practical).

---

455. Assign Cookies

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input: g = [1,2,3], s = [1,1]
Output: 1
Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.
And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.
You need to output 1.

Example 2:

Input: g = [1,2], s = [1,2,3]
Output: 2
Explanation: You have 2 children and 3 cookies. The greed factors of 2 children are 1, 2.
You have 3 cookies and their sizes are big enough to gratify all of the children,
You need to output 2.

Solution 1: Greedy with Sorting (Two Pointers)

```
def findContentChildren(g, s):
    g.sort()
    s.sort()
    i = j = 0
    content = 0

    while i < len(g) and j < len(s):
        if s[j] >= g[i]:
            content += 1
            i += 1
        j += 1

    return content
```

Time Complexity: O(n log n + m log m)
Space Complexity: O(1)
Idea: Sort both arrays, match smallest cookie to smallest greed child.

Solution 2: Greedy with Binary Search

```
def findContentChildren(g, s):
    g.sort()
    s.sort()
    content = 0

    for cookie in s:
        idx = bisect.bisect_right(g, cookie)
        if idx > content:
            content += 1

    return content
```

Time Complexity: O(n log n + m log m) | Space Complexity: O(1)
Idea: Sort, use binary search to find first child that can be satisfied.

Solution 3: Max-Heap Approach

```
def findContentChildren(g, s):
    heapq.heapify(g)
    heapq.heapify(s)
    content = 0

    while g and s:
        if s[0] >= g[0]:
            heapq.heappop(g)
            heapq.heappop(s)
            content += 1
        else:
            heapq.heappop(s)

    return content
```

Time Complexity: O(n log n + m log m) | Space Complexity: O(1)
Idea: Use heaps to always access smallest greed and cookie.

Solution 4: Counter/Hash Map

```
def findContentChildren(g, s):
    g_count = Counter(g)
    s_count = Counter(s)
    content = 0

    # Try to satisfy each greed factor
    for greed in sorted(g_count.keys()):
        for cookie in sorted(s_count.keys()):
            if cookie >= greed and s_count[cookie] > 0:
                pairs = min(g_count[greed], s_count[cookie])
                content += pairs
                g_count[greed] -= pairs
                s_count[cookie] -= pairs
                if g_count[greed] == 0:
                    break

    return content
```

Time Complexity: O(n log n + m log m) | Space Complexity: O(n + m)
Idea: Count frequencies, match greed factors with cookies.

406. Queue Reconstruction by Height

You are given an array of people, people, which are the attributes of some people in a queue (not necessarily in order). Each people[i] = [h_i, k_i] represents the i_{th} person of height h_i with **exactly** k_i other people in front who have a height greater than or equal to h_i.

Reconstruct and return *the queue that is represented by the input array* people. The returned queue should be formatted as an array queue, where queue[j] = [h_j, k_j] is the attributes of the j_{th} person in the queue (queue[0] is the person at the front of the queue).

Example 1:

Input: people = [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]
Output: [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]
Explanation:
Person 0 has height 5 with no other people taller or the same height in front.
Person 1 has height 7 with no other people taller or the same height in front.
Person 2 has height 5 with two persons taller or the same height in front, which is person 0 and 1.
Person 3 has height 6 with one person taller or the same height in front, which is person 1.
Person 4 has height 4 with four people taller or the same height in front, which are people 0, 1, 2, and 3.
Person 5 has height 7 with one person taller or the same height in front, which is person 1.
Hence [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]] is the reconstructed queue.

Example 2:

Input: people = [[6,0],[5,0],[4,0],[3,2],[2,2],[1,4]]
Output: [[4,0],[5,0],[2,2],[3,2],[1,4],[6,0]]

Solution 1: Greedy by Height Descending

```
def reconstructQueue(people):
    # Sort by height descending, then by k ascending
    people.sort(key=lambda x: (-x[0], x[1]))

    result = []
    for p in people:
        result.insert(p[1], p)

    return result
```

Time Complexity: O(n²) | Space Complexity: O(n)
Idea: Tallest people first, insert at k position (k = number of taller people).

Solution 2: Segment Tree for Insertion

```
class SegmentTree:
    def __init__(self, n):
        self.n = n
        self.tree = [0] * (4 * n)
        self.build(1, 0, n-1)

    def build(self, node, l, r):
        if l == r:
            self.tree[node] = 1
            return
        mid = (l + r) // 2
        self.build(node*2, l, mid)
        self.build(node*2+1, mid+1, r)
        self.tree[node] = self.tree[node*2] +
self.tree[node*2+1]

    def update(self, pos, node=1, l=0, r=None):
        if r is None:
            r = self.n - 1
        if l == r:
            self.tree[node] = 0
            return l
        mid = (l + r) // 2
        left_count = self.tree[node*2]

        if pos < left_count:
            res = self.update(pos, node*2, l, mid)
        else:
            res = self.update(pos-left_count, node*2+1, mid+1,
r)

        self.tree[node] = self.tree[node*2] +
self.tree[node*2+1]
        return res

def reconstructQueue(people):
    people.sort(key=lambda x: (x[0], -x[1]))
    n = len(people)
    seg_tree = SegmentTree(n)
    result = [None] * n

    for h, k in people:
        pos = seg_tree.update(k)
        result[pos] = [h, k]

    return result
```

Time Complexity: O(n log n) | Space Complexity: O(n)
Idea: Use segment tree to find k-th empty position efficiently.

Solution 3: Binary Indexed Tree

```
class BIT:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)
        for i in range(n):
            self.update(i, 1)

    def update(self, i, delta):
        i += 1
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i

    def query(self, i):
        i += 1
        res = 0
        while i > 0:
            res += self.bit[i]
            i -= i & -i
        return res

    def find_kth(self, k):
        # Find smallest i such that sum[0..i] >= k+1
        pos = 0
        bit_mask = 1 << (self.n.bit_length())

        while bit_mask:
            next_pos = pos + bit_mask
            if next_pos <= self.n and self.bit[next_pos] <= k:
                pos = next_pos
                k -= self.bit[pos]
            bit_mask >>= 1

        return pos

def reconstructQueue(people):
    people.sort(key=lambda x: (x[0], -x[1]))
    n = len(people)
    bit = BIT(n)
    result = [None] * n

    for h, k in people:
        pos = bit.find_kth(k)
        result[pos] = [h, k]
        bit.update(pos, -1)

    return result
```

Time Complexity: O(n log n) | Space Complexity: O(n)
Idea: Use BIT to maintain available positions and find k-th position.

Solution 4: Linked List Simulation

```
class ListNode:
    def __init__(self, val=None):
        self.val = val
        self.next = None

def reconstructQueue(people):
    people.sort(key=lambda x: (x[0], -x[1]))

    # Create linked list of empty slots
    dummy = ListNode()
    curr = dummy
    for i in range(len(people)):
        curr.next = ListNode()
        curr = curr.next

    result = [None] * len(people)

    for h, k in people:
        # Find the (k+1)-th empty slot
        prev, curr = dummy, dummy.next
        count = 0

        while count < k:
            if curr.val is None:
                count += 1
            prev, curr = curr, curr.next

        # Skip any filled positions
        while curr and curr.val is not None:
            prev, curr = curr, curr.next

        # Place person
        if curr:
            curr.val = [h, k]
            result[curr._index] = [h, k]

    return result
```

Time Complexity: O(n²) | Space Complexity: O(n)
Idea: Simulate insertion using linked list of empty positions.

---

392. Is Subsequence

Given two strings s and t, return true *if* s *is a* ***subsequence*** *of* t*, or* false *otherwise*.

A **subsequence** of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

Example 1:

Input: s = "abc", t = "ahbgdc"
Output: true

Example 2:

Input: s = "axc", t = "ahbgdc"
Output: false

Solution 1: Two Pointers

```
def isSubsequence(s, t):
    i = j = 0
    while i < len(s) and j < len(t):
        if s[i] == t[j]:
            i += 1
        j += 1
    return i == len(s)
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Simple pointer traversal, match characters in order.

Solution 2: Dynamic Programming (LCS)

```
def isSubsequence(s, t):
    m, n = len(s), len(t)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s[i-1] == t[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n] == m
```

Time Complexity: O(m*n) | Space Complexity: O(m*n)
Idea: Check if LCS length equals length of s.

Solution 3: Binary Search for Multiple Queries

```
class SubsequenceChecker:
    def __init__(self, t):
        self.char_positions = defaultdict(list)
        for i, char in enumerate(t):
            self.char_positions[char].append(i)

    def isSubsequence(self, s):
        prev_pos = -1
        for char in s:
            if char not in self.char_positions:
                return False

            positions = self.char_positions[char]
            # Find first position greater than prev_pos
            idx = bisect.bisect_right(positions, prev_pos)
            if idx == len(positions):
                return False
            prev_pos = positions[idx]

        return True
```

Time Complexity: O(m + n) for preprocessing, O(k log n) per query | Space Complexity: O(n)
Idea: Preprocess positions for each character, use binary search for each character in s.

Solution 4: Recursion with Memoization

```
def isSubsequence(s, t):
    from functools import lru_cache

    @lru_cache(None)
    def dfs(i, j):
        if i == len(s):
            return True
        if j == len(t):
            return False

        if s[i] == t[j]:
            return dfs(i + 1, j + 1)
        return dfs(i, j + 1)

    return dfs(0, 0)
```

Time Complexity: O(m*n) | Space Complexity: O(m*n)
Idea: Memoized DFS exploring match or skip decisions.

435. Non-overlapping Intervals

Given an array of intervals intervals where intervals[i] = [$start_i$, end_i], return *the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping*.

Note that intervals which only touch at a point are **non-overlapping**. For example, [1, 2] and [2, 3] are non-overlapping.

Example 1:

Input: intervals = [[1,2],[2,3],[3,4],[1,3]]
Output: 1
Explanation: [1,3] can be removed and the rest of the intervals are non-overlapping.

Example 2:

Input: intervals = [[1,2],[1,2],[1,2]]
Output: 2
Explanation: You need to remove two [1,2] to make the rest of the intervals non-overlapping.

Example 3:

Input: intervals = [[1,2],[2,3]]
Output: 0
Explanation: You don't need to remove any of the intervals since they're already non-overlapping.

Solution 1: Greedy by End Time

```
def eraseOverlapIntervals(intervals):
    if not intervals:
        return 0

    intervals.sort(key=lambda x: x[1])
    end = intervals[0][1]
    count = 0

    for i in range(1, len(intervals)):
        if intervals[i][0] < end:
            count += 1  # Remove current interval
        else:
            end = intervals[i][1]

    return count
```

Time Complexity: O(n log n) | Space Complexity: O(1)
Idea: Sort by end time, keep interval if it doesn't overlap with last kept.

Solution 2: Greedy by Start Time

```
def eraseOverlapIntervals(intervals):
    if not intervals:
        return 0

    intervals.sort()
    end = intervals[0][1]
    count = 0

    for i in range(1, len(intervals)):
        if intervals[i][0] < end:
            count += 1
            # Remove the interval with larger end time
            end = min(end, intervals[i][1])
        else:
            end = intervals[i][1]

    return count
```

Time Complexity: O(n log n) | Space Complexity: O(1)
Idea: Sort by start time, when overlap occurs, keep interval with smaller end.

Solution 3: Dynamic Programming (LIS)

```
def eraseOverlapIntervals(intervals):
    if not intervals:
        return 0

    intervals.sort()
    n = len(intervals)
    dp = [1] * n  # dp[i] = max non-overlapping intervals ending at i

    for i in range(n):
        for j in range(i):
            if intervals[j][1] <= intervals[i][0]:
                dp[i] = max(dp[i], dp[j] + 1)

    max_non_overlap = max(dp)
    return n - max_non_overlap
```

Time Complexity: O(n^2) | Space Complexity: O(n)
Idea: Find maximum size of non-overlapping subset (LIS variant).

Solution 4: Interval Graph Coloring

```
def eraseOverlapIntervals(intervals):
    if not intervals:
        return 0

    # Create events: start (+1), end (-1)
    events = []
    for start, end in intervals:
        events.append((start, 1))
        events.append((end, -1))

    events.sort(key=lambda x: (x[0], x[1]))

    max_concurrent = 0
    current = 0

    for _, event_type in events:
        current += event_type
        max_concurrent = max(max_concurrent, current)

    # In worst case, need to remove (max_concurrent - 1) intervals
    # from each overlapping group
    return len(intervals) - (len(intervals) // max_concurrent if max_concurrent > 0 else 0)
```

Time Complexity: O(n log n) | Space Complexity: O(n)
Idea: Find maximum number of overlapping intervals at any point.

621. Task Scheduler

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of **at least** n intervals between two tasks with the same label.

Return the **minimum** number of CPU intervals required to complete all tasks.

Example 1:

Input: tasks = ["A","A","A","B","B","B"], n = 2

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.

Example 2:

Input: tasks = ["A","C","A","B","D","B"], n = 1

Output: 6

Explanation: A possible sequence is: A -> B -> C -> D -> A -> B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: tasks = ["A","A","A", "B","B","B"], n = 3

Output: 10

Explanation: A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Solution 1: Max Frequency Greedy

```
def leastInterval(tasks, n):

    freq = Counter(tasks)
    max_freq = max(freq.values())

    # Count tasks with max frequency
    max_count = sum(1 for f in freq.values() if f ==
max_freq)

    # Calculate minimum intervals
    # Part 1: (max_freq - 1) full cycles
    # Part 2: slots in each cycle
    # Part 3: tasks with max frequency at the end
    return max(len(tasks), (max_freq - 1) * (n + 1) +
max_count)
```

Time Complexity: O(n) | Space Complexity: O(1)
Idea: Arrange most frequent tasks with cooldown slots in between.

Solution 2: Simulation with Max-Heap

```
def leastInterval(tasks, n):

    freq = Counter(tasks)
    max_heap = [-f for f in freq.values()]
    heapq.heapify(max_heap)

    time = 0
    queue = deque()  # (time_available, count)

    while max_heap or queue:
        time += 1

        if max_heap:
            count = heapq.heappop(max_heap) + 1  # +1
because negative
            if count < 0:
                queue.append((time + n, count))

        if queue and queue[0][0] == Time Complexity:
            heapq.heappush(max_heap, queue.popleft()[1])

    return time
```

Time Complexity: O(n log k) where k is unique tasks | Space Complexity: O(k)
Idea: Simulate task execution, use queue for cooldown.

Solution 3: Mathematical Formula

```
def leastInterval(tasks, n):
    from collections import Counter

    freq = Counter(tasks)
    freq_list = sorted(freq.values(), reverse=True)

    # Calculate idle slots
    max_freq = freq_list[0]
    idle_slots = (max_freq - 1) * n

    # Fill idle slots with other tasks
    for f in freq_list[1:]:
        idle_slots -= min(max_freq - 1, f)

    idle_slots = max(0, idle_slots)

    return len(tasks) + idle_slots
```

Time Complexity: O(n log n) | Space Complexity: O(1)
Idea: Calculate idle slots mathematically and try to fill them.

Solution 4: Priority Queue with Cooldown Map

```
def leastInterval(tasks, n):
    from collections import Counter
    import heapq

    freq = Counter(tasks)
    cooldown = {}  # task -> next available time
    max_heap = [(-f, task) for task, f in freq.items()]
    heapq.heapify(max_heap)

    time = 0
    temp_tasks = []

    while max_heap or temp_tasks:
        # Get available tasks (not in cooldown)
        available = []
        while max_heap:
            neg_f, task = heapq.heappop(max_heap)
            if task not in cooldown or cooldown[task] <= Time
Complexity:
                available.append((neg_f, task))
            else:
                temp_tasks.append((neg_f, task))

        if available:
            # Execute most frequent available task
            neg_f, task =
heapq.heappop(heapq.heapify(available) if available else
[])
            f = -neg_f - 1
            if f > 0:
                heapq.heappush(max_heap, (-f, task))
                cooldown[task] = time + n + 1

        # Return temp tasks to heap
        max_heap.extend(temp_tasks)
        heapq.heapify(max_heap)
        temp_tasks = []

        time += 1

    return time
```

Time Complexity: O(n log k) | Space Complexity: O(k)
Idea: Track cooldown for each task separately.

1710. Maximum Units on a Truck

You are assigned to put some amount of boxes onto **one truck**. You are given a 2D array boxTypes, where boxTypes[i] = [$numberOfBoxes_i$, $numberOfUnitsPerBox_i$]:

- $numberOfBoxes_i$ is the number of boxes of type i.
- $numberOfUnitsPerBox_i$ is the number of units in each box of the type i.

You are also given an integer truckSize, which is the **maximum** number of **boxes** that can be put on the truck. You can choose any boxes to put on the truck as long as the number of boxes does not exceed truckSize.

Return *the **maximum** total number of **units** that can be put on the truck.d*

Example 1:

Input: boxTypes = [[1,3],[2,2],[3,1]], truckSize = 4
Output: 8
Explanation: There are:
- 1 box of the first type that contains 3 units.
- 2 boxes of the second type that contain 2 units each.
- 3 boxes of the third type that contain 1 unit each.
You can take all the boxes of the first and second types, and one box of the third type.
The total number of units will be = (1 * 3) + (2 * 2) + (1 * 1) = 8.

Example 2:

Input: boxTypes = [[5,10],[2,5],[4,7],[3,9]], truckSize = 10
Output: 91

Approach 1: Greedy + Sorting (Most common)

Idea: Take boxes with highest `unitsPerBox` first.

```
class Solution:
    def maximumUnits(self, boxTypes, truckSize):
        boxTypes.sort(key=lambda x: x[1], reverse=True)

        total = 0
        for boxes, units in boxTypes:
            take = min(truckSize, boxes)
            total += take * units
            truckSize -= take
            if truckSize == 0:
                break
        return total
```

Time: `O(n log n)`
Space: `O(1)` extra

Approach 2: Counting Sort / Bucket (Faster if units range small)

Constraints: `unitsPerBox <= 1000`
So we can store counts in buckets instead of sorting.

```
class Solution:
    def maximumUnits(self, boxTypes, truckSize):
        buckets = [0] * 1001  # buckets[u] = total boxes having u units

        for boxes, units in boxTypes:
            buckets[units] += boxes

        total = 0
        for units in range(1000, 0, -1):
            if truckSize == 0:
                break
            take = min(truckSize, buckets[units])
            total += take * units
            truckSize -= take

        return total
```

Time: `O(n + 1000)` ≈ `O(n)`
Space: `O(1000)`

Approach 3: Max Heap (Priority Queue)

Idea: Always pop the box type with max `unitsPerBox`.

```
class Solution:
    def maximumUnits(self, boxTypes, truckSize):
        heap = []
        for boxes, units in boxTypes:
            heapq.heappush(heap, (-units, boxes))  # max heap using negative

        total = 0
        while heap and truckSize > 0:
            neg_units, boxes = heapq.heappop(heap)
            units = -neg_units

            take = min(truckSize, boxes)
            total += take * units
            truckSize -= take

        return total
```

Time: `O(n log n)`
Space: `O(n)`

HASH MAP

1. Two Sum

Given an array of integers nums and an integer target, return *indices of the two numbers such that they add up to target*.

You may assume that each input would have ***exactly* one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].

Example 2:

Input: nums = [3,2,4], target = 6
Output: [1,2]

Example 3:

Input: nums = [3,3], target = 6
Output: [0,1]

Approach 1: Brute Force

```
def twoSum(nums, target):
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []
```

Time Complexity: O(n²)
Space Complexity: O(1)

Approach 2: Two-pass Hash Table

```
def twoSum(nums, target):
    num_map = {}
    # First pass: store values and indices
    for i, num in enumerate(nums):
        num_map[num] = i
    # Second pass: find complement
    for i, num in enumerate(nums):
        complement = target - num
        if complement in num_map and num_map[complement] != i:
            return [i, num_map[complement]]
    return []
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 3: One-pass Hash Table

```
def twoSum(nums, target):
    num_map = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in num_map:
            return [num_map[complement], i]
        num_map[num] = i
    return []
```

Time Complexity: O(n)
Space Complexity: O(n)

---

217. Contains Duplicate

Given an integer array nums, return true if any value appears **at least twice** in the array, and return false if every element is distinct.

Example 1:

Input: nums = [1,2,3,1]

Output: true

Explanation:

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: nums = [1,2,3,4]

Output: false

Explanation:

All elements are distinct.

Example 3:

Input: nums = [1,1,1,3,3,4,3,2,4,2]

Output: true

Approach 1: Brute Force

```
def containsDuplicate(nums):
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] == nums[j]:
                return True
    return False
```

Time Complexity: O(n²)
Space Complexity: O(1)

Approach 2: Sorting

```
def containsDuplicate(nums):
    nums.sort()
    for i in range(1, len(nums)):
        if nums[i] == nums[i-1]:
            return True
    return False
```

Time Complexity: O(n log n)
Space Complexity: O(1)

Approach 3: HashSet

```
def containsDuplicate(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return True
        seen.add(num)
    return False
```

Time Complexity: O(n)
Space Complexity: O(n)

49. Group Anagrams

Given an array of strings strs, group the anagrams together. You can return the answer in **any order**.

Example 1:

Input: strs = ["eat","tea","tan","ate","nat","bat"]

Output: [["bat"],["nat","tan"],["ate","eat","tea"]]

Explanation:

- There is no string in strs that can be rearranged to form "bat".
- The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.
- The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other.

Example 2:

Input: strs = [""]

Output: [[""]]

Example 3:

Input: strs = ["a"]

Output: [["a"]]

Approach 1: Sorted String as Key

```
def groupAnagrams(strs):
    from collections import defaultdict

    anagrams = defaultdict(list)
    for s in strs:
        # Sort string to use as key
        sorted_str = ''.join(sorted(s))
        anagrams[sorted_str].append(s)

    return list(anagrams.values())
```

Time Complexity: O(n * k log k) where k is max string length
Space Complexity: O(n * k)

Approach 2: Character Count as Key

```
def groupAnagrams(strs):
    from collections import defaultdict

    anagrams = defaultdict(list)
    for s in strs:
        # Create count array of 26 zeros
        count = [0] * 26
        for char in s:
            count[ord(char) - ord('a')] += 1
        # Use tuple as key (hashable)
        anagrams[tuple(count)].append(s)

    return list(anagrams.values())
```

Time Complexity: O(n * k) where k is max string length
Space Complexity: O(n * k)

Approach 3: Prime Product as Key

```
def groupAnagrams(strs):
    from collections import defaultdict
    import math

    # Assign prime numbers to each letter
    primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
              43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101]

    anagrams = defaultdict(list)
    for s in strs:
        product = 1
        for char in s:
            product *= primes[ord(char) - ord('a')]
        anagrams[product].append(s)

    return list(anagrams.values())
```

Time Complexity: O(n * k) where k is max string length
Space Complexity: O(n * k)

---

347. Top K Frequent Elements

Given an integer array nums and an integer k, return *the* k *most frequent elements*. You may return the answer in **any order**.

Example 1:

Input: nums = [1,1,1,2,2,3], k = 2

Output: [1,2]

Example 2:

Input: nums = [1], k = 1

Output: [1]

Example 3:

Input: nums = [1,2,1,2,1,2,3,1,3,2], k = 2

Output: [1,2]

Approach 1: Hash Map + Sorting

```
def topKFrequent(nums, k):
    from collections import Counter

    # Count frequencies
    count = Counter(nums)
    # Sort by frequency and get top k
    return [item for item, freq in sorted(count.items(),
key=lambda x: x[1], reverse=True)[:k]]
```

Time Complexity: O(n log n)
Space Complexity: O(n)

Approach 2: Hash Map + Heap

```
def topKFrequent(nums, k):
    from collections import Counter
    import heapq

    count = Counter(nums)
    # Use heap of size k (min-heap)
    heap = []
    for num, freq in count.items():
        heapq.heappush(heap, (freq, num))
        if len(heap) > k:
            heapq.heappop(heap)

    return [num for freq, num in heap]
```

Time Complexity: O(n log k)
Space Complexity: O(n + k)

Approach 3: Bucket Sort

```
def topKFrequent(nums, k):
    from collections import Counter

    count = Counter(nums)
    # Create buckets where index = frequency
    buckets = [[] for _ in range(len(nums) + 1)]
    for num, freq in count.items():
        buckets[freq].append(num)

    # Collect from highest frequency bucket
    result = []
    for i in range(len(buckets) - 1, 0, -1):
        for num in buckets[i]:
            result.append(num)
            if len(result) == k:
                return result
    return result
```

Time Complexity: O(n)
Space Complexity: O(n)

---

3. Longest Substring Without Repeating Characters

Given a string s, find the length of the **longest** **substring** without duplicate characters.

Example 1:

Input: s = "abcabcbb"
Output: 3
Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: s = "bbbbb"
Output: 1
Explanation: The answer is "b", with the length of 1.

Example 3:

Input: s = "pwwkew"
Output: 3
Explanation: The answer is "wke", with the length of 3. Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Approach 1: Brute Force

```
def lengthOfLongestSubstring(s):
    def all_unique(substring):
        return len(set(substring)) == len(substring)

    n = len(s)
    max_len = 0
    for i in range(n):
        for j in range(i, n):
            if all_unique(s[i:j+1]):
                max_len = max(max_len, j - i + 1)
    return max_len
```

Time Complexity: O(n³)
Space Complexity: O(min(n, m)) where m is charset size

Approach 2: Sliding Window with Set

```
def lengthOfLongestSubstring(s):
    char_set = set()
    left = max_len = 0

    for right in range(len(s)):
        while s[right] in char_set:
            char_set.remove(s[left])
            left += 1
        char_set.add(s[right])
        max_len = max(max_len, right - left + 1)

    return max_len
```

Time Complexity: O(n)
Space Complexity: O(min(n, m))

Approach 3: Optimized Sliding Window with HashMap

```
def lengthOfLongestSubstring(s):
    char_map = {}  # Store character -> last index seen
    left = max_len = 0

    for right, char in enumerate(s):
        if char in char_map and char_map[char] >= left:
            left = char_map[char] + 1
        char_map[char] = right
        max_len = max(max_len, right - left + 1)

    return max_len
```

Time Complexity: O(n)
Space Complexity: O(min(n, m))

---

560. Subarray Sum Equals K

Given an array of integers nums and an integer k, return *the total number of subarrays whose sum equals to* k.

A subarray is a contiguous **non-empty** sequence of elements within an array.

Example 1:

Input: nums = [1,1,1], k = 2
Output: 2

Example 2:

Input: nums = [1,2,3], k = 3
Output: 2

Approach 1: Brute Force

```
def subarraySum(nums, k):
    count = 0
    n = len(nums)

    for i in range(n):
        current_sum = 0
        for j in range(i, n):
            current_sum += nums[j]
            if current_sum == k:
                count += 1

    return count
```

Time Complexity: O(n²)
Space Complexity: O(1)

Approach 2: Prefix Sum with HashMap

```
def subarraySum(nums, k):
    count = 0
    prefix_sum = 0
    sum_map = {0: 1}  # Store prefix sum frequencies

    for num in nums:
        prefix_sum += num
        # Check if (prefix_sum - k) exists in map
        if prefix_sum - k in sum_map:
            count += sum_map[prefix_sum - k]
        sum_map[prefix_sum] = sum_map.get(prefix_sum,
0) + 1

    return count
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 3: Cumulative Sum Array

```
def subarraySum(nums, k):
    n = len(nums)
    # Create cumulative sum array
    cumulative = [0] * (n + 1)
    for i in range(n):
        cumulative[i + 1] = cumulative[i] + nums[i]

    count = 0
    for i in range(n):
        for j in range(i + 1, n + 1):
            if cumulative[j] - cumulative[i] == k:
                count += 1

    return count
```

Time Complexity: O(n²)
Space Complexity: O(n)

---

128. Longest Consecutive Sequence

Given an unsorted array of integers nums, return *the length of the longest consecutive elements sequence.*

You must write an algorithm that runs in O(n) time.

Example 1:

Input: nums = [100,4,200,1,3,2]
Output: 4
Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.

Example 2:

Input: nums = [0,3,7,2,5,8,4,6,0,1]
Output: 9

Example 3:

Input: nums = [1,0,1,2]
Output: 3

Approach 1: Sorting

```
def longestConsecutive(nums):
    if not nums:
        return 0

    nums.sort()
    max_len = 1
    current_len = 1

    for i in range(1, len(nums)):
        if nums[i] == nums[i-1]:
            continue
        elif nums[i] == nums[i-1] + 1:
            current_len += 1
            max_len = max(max_len, current_len)
        else:
            current_len = 1

    return max_len
```

Time Complexity: O(n log n)
Space Complexity: O(1) or O(n) depending on sorting

Approach 2: HashSet

```
def longestConsecutive(nums):
    num_set = set(nums)
    max_len = 0

    for num in num_set:
        # Check if it's the start of a sequence
        if num - 1 not in num_set:
            current_num = num
            current_len = 1

            while current_num + 1 in num_set:
                current_num += 1
                current_len += 1

            max_len = max(max_len, current_len)

    return max_len
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 3: Union-Find (Disjoint Set)

```
def longestConsecutive(nums):
    if not nums:
        return 0

    parent = {}
    size = {}

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    def union(x, y):
        root_x, root_y = find(x), find(y)
        if root_x != root_y:
            if size[root_x] < size[root_y]:
                parent[root_x] = root_y
                size[root_y] += size[root_x]
            else:
                parent[root_y] = root_x
                size[root_x] += size[root_y]

    # Initialize each number as its own parent
    for num in nums:
        parent[num] = num
        size[num] = 1

    # Union consecutive numbers
    for num in nums:
        if num + 1 in parent:
            union(num, num + 1)

    return max(size.values()) if size else 0
```

Time Complexity: O(n * α(n)) where α is inverse Ackermann function
Space Complexity: O(n)

---

438. Find All Anagrams in a String

Given two strings s and p, return an array of all the start indices of p's anagrams in s. You may return the answer in **any order**.

Example 1:

Input: s = "cbaebabacd", p = "abc"
Output: [0,6]
Explanation:
The substring with start index = 0 is "cba", which is an anagram of "abc".
The substring with start index = 6 is "bac", which is an anagram of "abc".

Example 2:

Input: s = "abab", p = "ab"
Output: [0,1,2]
Explanation:
The substring with start index = 0 is "ab", which is an anagram of "ab".
The substring with start index = 1 is "ba", which is an anagram of "ab".
The substring with start index = 2 is "ab", which is an anagram of "ab".

Approach 1: Brute Force

```
def findAnagrams(s, p):
    from collections import Counter

    result = []
    p_count = Counter(p)
    p_len = len(p)

    for i in range(len(s) - p_len + 1):
        if Counter(s[i:i + p_len]) == p_count:
            result.append(i)

    return result
```

Time Complexity: O(n * m) where n = len(s), m = len(p)
Space Complexity: O(m)

Approach 2: Sliding Window with HashMap

```
def findAnagrams(s, p):
    from collections import Counter

    result = []
    p_count = Counter(p)
    s_count = Counter()
    p_len = len(p)

    for i in range(len(s)):
        # Add current character to window
        s_count[s[i]] += 1

        # Remove character outside window
        if i >= p_len:
            if s_count[s[i - p_len]] == 1:
                del s_count[s[i - p_len]]
            else:
                s_count[s[i - p_len]] -= 1

        # Compare window with p
        if s_count == p_count:
            result.append(i - p_len + 1)

    return result
```

Time Complexity: O(n)
Space Complexity: O(m)

Approach 3: Sliding Window with Array (Optimized)

```
def findAnagrams(s, p):
    result = []
    p_len, s_len = len(p), len(s)

    if p_len > s_len:
        return result

    # Use arrays of size 26 for character counts
    p_count = [0] * 26
    s_count = [0] * 26

    # Count characters in p and first window of s
    for i in range(p_len):
        p_count[ord(p[i]) - ord('a')] += 1
        s_count[ord(s[i]) - ord('a')] += 1

    # Check first window
    if s_count == p_count:
        result.append(0)

    # Slide the window
    for i in range(p_len, s_len):
        # Add new character
        s_count[ord(s[i]) - ord('a')] += 1
        # Remove old character
        s_count[ord(s[i - p_len]) - ord('a')] -= 1

        if s_count == p_count:
            result.append(i - p_len + 1)

    return result
```

Time Complexity: O(n)
Space Complexity: O(1)

---

76. Minimum Window Substring

Given two strings s and t of lengths m and n respectively, return *the* ***minimum window substring*** *of* s *such that every character in* t (***including duplicates***) *is included in the window*. If there is no such substring, return *the empty string* "".

The testcases will be generated such that the answer is **unique**.

Example 1:

Input: s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"
Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string t.

Example 2:

Input: s = "a", t = "a"
Output: "a"
Explanation: The entire string s is the minimum window.

Example 3:

Input: s = "a", t = "aa"
Output: ""

Approach 1: Brute Force

```
def minWindow(s, t):
    from collections import Counter

    if not s or not t:
        return ""

    t_count = Counter(t)
    min_len = float('inf')
    result = ""

    for i in range(len(s)):
        for j in range(i, len(s)):
            window = s[i:j+1]
            window_count = Counter(window)

            # Check if window contains all chars from t
            valid = True
            for char, count in t_count.items():
                if window_count.get(char, 0) < count:
                    valid = False
                    break

            if valid and (j - i + 1) < min_len:
                min_len = j - i + 1
                result = window

    return result
```

Time Complexity: O(n² * m) where n = len(s), m = len(t)
Space Complexity: O(n + m)

Approach 2: Sliding Window with Two Pointers

```
def minWindow(s, t):
    from collections import Counter, defaultdict

    if not s or not t:
        return ""

    t_count = Counter(t)
    window_count = defaultdict(int)

    required = len(t_count)
    formed = 0

    left = right = 0
    min_len = float('inf')
    result = ""

    while right < len(s):
        # Add character to window
        char = s[right]
        window_count[char] += 1

        if char in t_count and window_count[char] ==
t_count[char]:
            formed += 1

        # Try to minimize window
        while left <= right and formed == required:
            if right - left + 1 < min_len:
                min_len = right - left + 1
                result = s[left:right+1]

            # Remove left character
            left_char = s[left]
            window_count[left_char] -= 1
            if left_char in t_count and window_count[left_char]
< t_count[left_char]:
                formed -= 1

            left += 1

        right += 1

    return result
```

Time Complexity: O(n)
Space Complexity: O(m)

Approach 3: Optimized Sliding Window with Filtered String

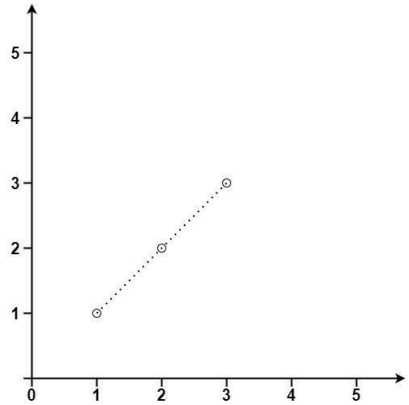
```
def minWindow(s, t):
    from collections import Counter

    if not s or not t:
        return ""

    t_count = Counter(t)

    # Filter s to only include characters in t
    filtered_s = []
    for i, char in enumerate(s):
        if char in t_count:
            filtered_s.append((i, char))

    window_count = Counter()
    required = len(t_count)
    formed = 0
    left = 0
    min_len = float('inf')
    result = ""

    for right in range(len(filtered_s)):
        # Add character to window
        char = filtered_s[right][1]
        window_count[char] += 1

        if window_count[char] == t_count[char]:
            formed += 1

        # Try to minimize window
        while left <= right and formed == required:
            start = filtered_s[left][0]
            end = filtered_s[right][0]

            if end - start + 1 < min_len:
                min_len = end - start + 1
                result = s[start:end+1]

            # Remove left character
            left_char = filtered_s[left][1]
            window_count[left_char] -= 1
            if window_count[left_char] < t_count[left_char]:
                formed -= 1

            left += 1

    return result
```

Time Complexity: O(n + m)
Space Complexity: O(n + m)

149. Max Points on a Line

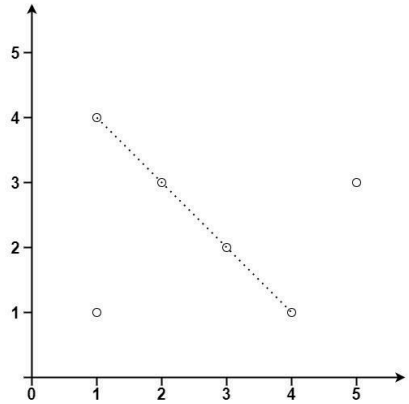
Given an array of points where points[i] = [x_i, y_i] represents a point on the **X-Y** plane, return *the maximum number of points that lie on the same straight line*.

Example 1:



Input: points = [[1,1],[2,2],[3,3]]
Output: 3

Example 2:



Input: points = [[1,1],[3,2],[5,3],[4,1],[2,3],[1,4]]
Output: 4

Approach 1: Brute Force with Slope

```
def maxPoints(points):
    if len(points) <= 2:
        return len(points)

    max_points = 2

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            # Count points on line through points[i] and
points[j]
            count = 2

            for k in range(len(points)):
                if k != i and k != j:
                    # Check if point k lies on same line
                    # Using cross product to check collinearity
                    x1, y1 = points[i]
                    x2, y2 = points[j]
                    x3, y3 = points[k]

                    if (y2 - y1) * (x3 - x1) == (y3 - y1) * (x2 - x1):
                        count += 1

            max_points = max(max_points, count)

    return max_points
```

Time Complexity: $O(n^3)$
Space Complexity: $O(1)$

Approach 2: Hash Map with Slope

```
def maxPoints(points):
    from collections import defaultdict
    from math import gcd

    if len(points) <= 2:
        return len(points)

    max_points = 2

    for i in range(len(points)):
        slope_map = defaultdict(int)
        duplicate = 1  # Count duplicate points
        vertical = 0

        for j in range(len(points)):
            if i == j:
                continue

            x1, y1 = points[i]
            x2, y2 = points[j]
            if x1 == x2 and y1 == y2:
                duplicate += 1
            elif x1 == x2:
                vertical += 1
            else:
                # Calculate slope as reduced fraction to avoid
floating point issues
                dx = x2 - x1
                dy = y2 - y1
                g = gcd(dx, dy)
                slope = (dx // g, dy // g)
                slope_map[slope] += 1

        current_max = duplicate  # Start with duplicate points
        if vertical:
            current_max = max(current_max, vertical +
duplicate)
        for count in slope_map.values():
            current_max = max(current_max, count +
duplicate)

        max_points = max(max_points, current_max)

    return max_points
```

Time Complexity: $O(n^2)$
Space Complexity: $O(n)$

Approach 3: Optimized with Line Equation

```
def maxPoints(points):
    from collections import defaultdict
    from math import gcd

    if len(points) <= 2:
        return len(points)

    max_points = 2

    for i in range(len(points)):
        line_map = defaultdict(int)
        x1, y1 = points[i]

        for j in range(i + 1, len(points)):
            x2, y2 = points[j]

            # Use line equation ax + by + c = 0
            # For two points, we can use:
            # (y2 - y1)x - (x2 - x1)y + (x2*y1 - y2*x1) = 0
            A = y2 - y1
            B = x1 - x2
            C = x2 * y1 - y2 * x1

            # Normalize the line representation
            g = gcd(gcd(A, B), C)
            if g != 0:
                A //= g
                B //= g
                C //= g

            # Ensure consistent representation
            if A < 0 or (A == 0 and B < 0):
                A, B, C = -A, -B, -C

            line_map[(A, B, C)] += 1

        if line_map:
            max_points = max(max_points,
max(line_map.values()) + 1)

    return max_points
```

Time Complexity: $O(n^2)$
Space Complexity: $O(n)$

SORTING

912. Sort an Array

Given an array of integers nums, sort the array in ascending order and return it.

You must solve the problem **without using any built-in** functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

Example 1:

Input: nums = [5,2,3,1]
Output: [1,2,3,5]
Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

Example 2:

Input: nums = [5,1,1,2,0,0]
Output: [0,0,1,1,2,5]
Explanation: Note that the values of nums are not necessarily unique

---

Approach 1: Merge Sort

```
def sortArray(nums):
    def merge_sort(arr):
        if len(arr) <= 1:
            return arr

        mid = len(arr) // 2
        left = merge_sort(arr[:mid])
        right = merge_sort(arr[mid:])

        return merge(left, right)

    def merge(left, right):
        result = []
        i = j = 0

        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                result.append(left[i])
                i += 1
            else:
                result.append(right[j])
                j += 1

        result.extend(left[i:])
        result.extend(right[j:])
        return result

    return merge_sort(nums)
```

Time Complexity: O(n log n)
Space Complexity: O(n)

Approach 2: Quick Sort

```
def sortArray(nums):
    def quick_sort(arr, low, high):
        if low < high:
            pi = partition(arr, low, high)
            quick_sort(arr, low, pi - 1)
            quick_sort(arr, pi + 1, high)

    def partition(arr, low, high):
        pivot = arr[high]
        i = low - 1

        for j in range(low, high):
            if arr[j] <= pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]

        arr[i + 1], arr[high] = arr[high], arr[i + 1]
        return i + 1

    quick_sort(nums, 0, len(nums) - 1)
    return nums
```

Time Complexity: O(n log n) average, O(n²) worst
Space Complexity: O(log n)

Approach 3: Heap Sort

```
def sortArray(nums):
    import heapq

    heapq.heapify(nums)
    return [heapq.heappop(nums) for _ in
range(len(nums))]
```

Time Complexity: O(n log n)
Space Complexity: O(n)

---

147. Insertion Sort List

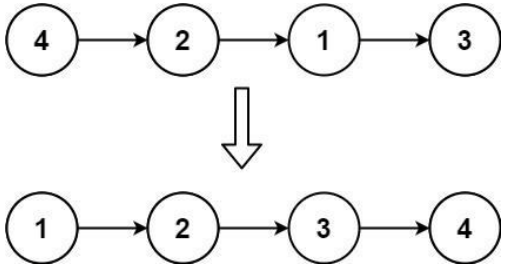
Given the head of a singly linked list, sort the list using **insertion sort**, and return *the sorted list's head*.

The steps of the **insertion sort** algorithm:

1. Insertion sort iterates, consuming one input element each repetition and growing a sorted output list.
2. At each iteration, insertion sort removes one element from the input data, finds the location it belongs within the sorted list and inserts it there.
3. It repeats until no input elements remain.

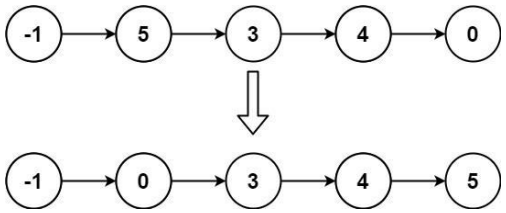
The following is a graphical example of the insertion sort algorithm. The partially sorted list (black) initially contains only the first element in the list. One element (red) is removed from the input data and inserted in-place into the sorted list with each iteration.

Example 1:



Input: head = [4,2,1,3]
Output: [1,2,3,4]

Example 2:



Input: head = [-1,5,3,4,0]
Output: [-1,0,3,4,5]

Approach 1: Standard Insertion Sort

```
def insertionSortList(head):
    if not head or not head.next:
        return head

    dummy = ListNode(0)
    dummy.next = head
    curr = head

    while curr and curr.next:
        if curr.val <= curr.next.val:
            curr = curr.next
        else:
            # Find position to insert
            prev = dummy
            while prev.next.val <= curr.next.val:
                prev = prev.next

            # Insert node
            temp = curr.next
            curr.next = temp.next
            temp.next = prev.next
            prev.next = temp

    return dummy.next
```

Time Complexity: O(n²)
Space Complexity: O(1)

Approach 2: Optimized with Early Break

```
def insertionSortList(head):
    if not head or not head.next:
        return head

    dummy = ListNode(0)
    dummy.next = head
    last_sorted = head
    curr = head.next

    while curr:
        if last_sorted.val <= curr.val:
            last_sorted = last_sorted.next
        else:
            prev = dummy
            while prev.next.val <= curr.val:
                prev = prev.next

            last_sorted.next = curr.next
            curr.next = prev.next
            prev.next = curr

        curr = last_sorted.next

    return dummy.next
```

Time Complexity: O(n²)
Space Complexity: O(1)

Approach 3: Array Conversion

```
def insertionSortList(head):
    # Convert to array
    values = []
    curr = head
    while curr:
        values.append(curr.val)
        curr = curr.next

    # Sort array
    for i in range(1, len(values)):
        key = values[i]
        j = i - 1
        while j >= 0 and values[j] > key:
            values[j + 1] = values[j]
            j -= 1
        values[j + 1] = key

    # Reconstruct linked list
    dummy = ListNode(0)
    curr = dummy
    for val in values:
        curr.next = ListNode(val)
        curr = curr.next

    return dummy.next
```

Time Complexity: O(n²)
Space Complexity: O(n)

---

75. Sort Colors

Given an array nums with n objects colored red, white, or blue, sort them **in-place** so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

Input: nums = [2,0,2,1,1,0]
Output: [0,0,1,1,2,2]

Example 2:

Input: nums = [2,0,1]
Output: [0,1,2]

Approach 1: Counting Sort

```
def sortColors(nums):
    count = [0, 0, 0]

    # Count occurrences
    for num in nums:
        count[num] += 1

    # Overwrite array
    index = 0
    for color in range(3):
        for _ in range(count[color]):
            nums[index] = color
            index += 1

    return nums
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 2: Two-Pointer (Dutch National Flag)

```
def sortColors(nums):
    low = mid = 0
    high = len(nums) - 1

    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:  # nums[mid] == 2
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1

    return nums
```

Time Complexity: O(n)
Space Complexity: O(1)

Approach 3: Bubble Sort

```
def sortColors(nums):
    n = len(nums)
    for i in range(n):
        for j in range(n - 1):
            if nums[j] > nums[j + 1]:
                nums[j], nums[j + 1] = nums[j + 1], nums[j]
    return nums
```

Time Complexity: O(n²)
Space Complexity: O(1)

---

215. Kth Largest Element in an Array

Given an integer array nums and an integer k, return *the* k_{th} *largest element in the array.*

Note that it is the k_{th} largest element in the sorted order, not the k_{th} distinct element.

Can you solve it without sorting?

Example 1:

Input: nums = [3,2,1,5,6,4], k = 2
Output: 5

Example 2:

Input: nums = [3,2,3,1,2,4,5,5,6], k = 4
Output: 4

Approach 1: Sorting

```
def findKthLargest(nums, k):
    nums.sort()
    return nums[-k]
```

Time Complexity: O(n log n)
Space Complexity: O(1)

Approach 2: Quick Select

```
def findKthLargest(nums, k):
    def quick_select(left, right, k_smallest):
        if left == right:
            return nums[left]

        pivot_index = partition(left, right)
        if k_smallest == pivot_index:
            return nums[k_smallest]
        elif k_smallest < pivot_index:
            return quick_select(left, pivot_index - 1,
k_smallest)
        else:
            return quick_select(pivot_index + 1, right,
k_smallest)

    def partition(left, right):
        pivot = nums[right]
        i = left
        for j in range(left, right):
            if nums[j] <= pivot:
                nums[i], nums[j] = nums[j], nums[i]
                i += 1
        nums[i], nums[right] = nums[right], nums[i]
        return i

    # kth largest = (n - k)th smallest
    return quick_select(0, len(nums) - 1, len(nums) - k)
```

Time Complexity: O(n) average, O(n²) worst
Space Complexity: O(log n)

Approach 3: Min Heap

```
def findKthLargest(nums, k):
    import heapq

    heap = nums[:k]
    heapq.heapify(heap)

    for num in nums[k:]:
        if num > heap[0]:
            heapq.heapreplace(heap, num)

    return heap[0]
```

Time Complexity: O(n log k)
Space Complexity: O(k)

1051. Height Checker

A school is trying to take an annual photo of all the students. The students are asked to stand in a single file line in **non-decreasing order** by height. Let this ordering be represented by the integer array expected where expected[i] is the expected height of the i_{th} student in line.

You are given an integer array heights representing the **current order** that the students are standing in. Each heights[i] is the height of the i_{th} student in line (**0-indexed**).

Return *the **number of indices** where* heights[i] != expected[i].

Example 1:

Input: heights = [1,1,4,2,1,3]
Output: 3
Explanation:
heights: [1,1,4,2,1,3]
expected: [1,1,1,2,3,4]
Indices 2, 4, and 5 do not match.

Example 2:

Input: heights = [5,1,2,3,4]
Output: 5
Explanation:
heights: [5,1,2,3,4]
expected: [1,2,3,4,5]
All indices do not match.

Example 3:

Input: heights = [1,2,3,4,5]
Output: 0
Explanation:
heights: [1,2,3,4,5]
expected: [1,2,3,4,5]
All indices match.

Approach 1: Sorting and Comparison

```
def heightChecker(heights):
    expected = sorted(heights)
    count = 0

    for i in range(len(heights)):
        if heights[i] != expected[i]:
            count += 1

    return count
```

Time Complexity: O(n log n)
Space Complexity: O(n)

Approach 2: Counting Sort (Since heights are 1-100)

```
def heightChecker(heights):
    # Counting sort since heights are between 1 and 100
    count = [0] * 101

    for h in heights:
        count[h] += 1

    expected = []
    for height in range(101):
        expected.extend([height] * count[height])

    # Compare
    mismatches = 0
    for i in range(len(heights)):
        if heights[i] != expected[i]:
            mismatches += 1

    return mismatches
```

Time Complexity: O(n + 100) ≈ O(n)
Space Complexity: O(n)

Approach 3: Bubble Sort Simulation

```
def heightChecker(heights):
    # Create a copy and sort using bubble sort
    expected = heights.copy()
    n = len(expected)

    # Bubble sort
    for i in range(n):
        for j in range(n - 1):
            if expected[j] > expected[j + 1]:
                expected[j], expected[j + 1] = expected[j + 1],
expected[j]

    # Count mismatches
    return sum(1 for i in range(n) if heights[i] != expected[i])
```

Time Complexity: O(n²)
Space Complexity: O(n)

1122. Relative Sort Array

Given two arrays arr1 and arr2, the elements of arr2 are distinct, and all elements in arr2 are also in arr1.

Sort the elements of arr1 such that the relative ordering of items in arr1 are the same as in arr2. Elements that do not appear in arr2 should be placed at the end of arr1 in **ascending** order.

Example 1:

Input: arr1 = [2,3,1,3,2,4,6,7,9,2,19], arr2 = [2,1,4,3,9,6]
Output: [2,2,2,1,4,3,3,9,6,7,19]

Example 2:

Input: arr1 = [28,6,22,8,44,17], arr2 = [22,28,8,6]
Output: [22,28,8,6,17,44]

Approach 1: Counting Sort

```
def relativeSortArray(arr1, arr2):
    # Count occurrences in arr1
    count = [0] * 1001
    for num in arr1:
        count[num] += 1

    result = []
    # Add elements from arr2 in order
    for num in arr2:
        result.extend([num] * count[num])
        count[num] = 0

    # Add remaining elements in ascending order
    for num in range(1001):
        if count[num] > 0:
            result.extend([num] * count[num])

    return result
```

Time Complexity: O(n + m)
Space Complexity: O(n)

Approach 2: Custom Sort with Dictionary

```
def relativeSortArray(arr1, arr2):
    # Create order map
    order = {num: i for i, num in enumerate(arr2)}

    # Custom sort key
    def sort_key(x):
        # If in arr2, use its index, else use its value +
len(arr2)
        return (order.get(x, len(arr2)), x)

    return sorted(arr1, key=sort_key)
```

Time Complexity: O(n log n)
Space Complexity: O(n)

Approach 3: Two-Pass with List Comprehension

```
def relativeSortArray(arr1, arr2):
    result = []

    # Add elements from arr2 in order
    for num in arr2:
        result.extend([num] * arr1.count(num))

    # Add remaining elements sorted
    remaining = [x for x in arr1 if x not in set(arr2)]
    result.extend(sorted(remaining))

    return result
```

Time Complexity: O(n²) due to count() calls
Space Complexity: O(n)

347. Top K Frequent Elements (Bucket Sort)

Given an integer array nums and an integer k, return *the* k *most frequent elements*. You may return the answer in **any order**.

Example 1:

Input: nums = [1,1,1,2,2,3], k = 2

Output: [1,2]

Example 2:

Input: nums = [1], k = 1

Output: [1]

Example 3:

Input: nums = [1,2,1,2,1,2,3,1,3,2], k = 2

Output: [1,2]

Approach 1: Bucket Sort

```
def topKFrequent(nums, k):

    # Count frequencies
    count = Counter(nums)

    # Create buckets where index = frequency
    buckets = [[] for _ in range(len(nums) + 1)]
    for num, freq in count.items():
        buckets[freq].append(num)

    # Collect from highest frequency bucket
    result = []
    for i in range(len(buckets) - 1, -1, -1):
        for num in buckets[i]:
            result.append(num)
            if len(result) == k:
                return result

    return result
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: Heap + Hash Map

```
def topKFrequent(nums, k):

    count = Counter(nums)
    return [num for num, freq in heapq.nlargest(k, count.items(), key=lambda x: x[1])]
```

Time Complexity: O(n log k)
Space Complexity: O(n + k)

Approach 3: Quick Select

```
def topKFrequent(nums, k):

    count = Counter(nums)
    unique = list(count.keys())

    def quick_select(left, right, k_smallest):
        if left == right:
            return

        pivot_index = partition(left, right)
        if k_smallest == pivot_index:
            return
        elif k_smallest < pivot_index:
            quick_select(left, pivot_index - 1, k_smallest)
        else:
            quick_select(pivot_index + 1, right, k_smallest)

    def partition(left, right):
        pivot_freq = count[unique[right]]
        i = left
        for j in range(left, right):
            if count[unique[j]] <= pivot_freq:
                unique[i], unique[j] = unique[j], unique[i]
                i += 1
        unique[i], unique[right] = unique[right], unique[i]
        return i

    n = len(unique)
    quick_select(0, n - 1, n - k)
    return unique[n - k:]
```

Time Complexity: O(n) average
Space Complexity: O(n)

164. Maximum Gap (Radix Sort)

Given an integer array nums, return *the maximum difference between two successive elements in its sorted form*. If the array contains less than two elements, return 0.

You must write an algorithm that runs in linear time and uses linear extra space.

Example 1:

Input: nums = [3,6,9,1]
Output: 3
Explanation: The sorted form of the array is [1,3,6,9], either (3,6) or (6,9) has the maximum difference 3.

Example 2:

Input: nums = [10]
Output: 0
Explanation: The array contains less than 2 elements, therefore return 0.

Approach 1: Radix Sort

```
def maximumGap(nums):
    if len(nums) < 2:
        return 0

    # Radix sort
    max_num = max(nums)
    exp = 1
    n = len(nums)

    while max_num // exp > 0:
        counting_sort(nums, exp)
        exp *= 10

    # Find maximum gap
    max_gap = 0
    for i in range(1, n):
        max_gap = max(max_gap, nums[i] - nums[i - 1])

    return max_gap

def counting_sort(nums, exp):
    n = len(nums)
    output = [0] * n
    count = [0] * 10

    # Count occurrences
    for num in nums:
        index = (num // exp) % 10
        count[index] += 1

    # Cumulative count
    for i in range(1, 10):
        count[i] += count[i - 1]

    # Build output array
    for i in range(n - 1, -1, -1):
        index = (nums[i] // exp) % 10
        output[count[index] - 1] = nums[i]
        count[index] -= 1

    # Copy back
    for i in range(n):
        nums[i] = output[i]
```

Time Complexity: O(n * d) where d is number of digits
Space Complexity: O(n)

Approach 2: Sorting + Linear Scan

```
def maximumGap(nums):
    if len(nums) < 2:
        return 0

    nums.sort()
    max_gap = 0

    for i in range(1, len(nums)):
        max_gap = max(max_gap, nums[i] - nums[i - 1])

    return max_gap
```

Time Complexity: O(n log n)
Space Complexity: O(1)

Approach 3: Bucket Sort (Pigeonhole Principle)

```
def maximumGap(nums):
    if len(nums) < 2:
        return 0

    min_num, max_num = min(nums), max(nums)
    if min_num == max_num:
        return 0

    n = len(nums)
    bucket_size = max(1, (max_num - min_num) // (n - 1))
    bucket_count = (max_num - min_num) // bucket_size + 1

    buckets = [[None, None] for _ in range(bucket_count)]

    # Fill buckets with min and max
    for num in nums:
        idx = (num - min_num) // bucket_size
        if buckets[idx][0] is None:
            buckets[idx][0] = buckets[idx][1] = num
        else:
            buckets[idx][0] = min(buckets[idx][0], num)
            buckets[idx][1] = max(buckets[idx][1], num)

    # Find maximum gap
    max_gap = 0
    prev_max = min_num

    for bucket in buckets:
        if bucket[0] is None:
            continue
        max_gap = max(max_gap, bucket[0] - prev_max)
        prev_max = bucket[1]

    return max_gap
```

Time Complexity: O(n)
Space Complexity: O(n)

---

969. Pancake Sorting (Pancake Sort)

Given an array of integers arr, sort the array by performing a series of **pancake flips**.

In one pancake flip we do the following steps:

- Choose an integer k where 1 <= k <= arr.length.
- Reverse the sub-array arr[0...k-1] (**0-indexed**).

For example, if arr = [3,2,1,4] and we performed a pancake flip choosing k = 3, we reverse the sub-array [3,2,1], so arr = [1,2,3,4] after the pancake flip at k = 3.

Return *an array of the* k*-values corresponding to a sequence of pancake flips that sort* arr. Any valid answer that sorts the array within 10 * arr.length flips will be judged as correct.

Example 1:

Input: arr = [3,2,4,1]
Output: [4,2,4,3]
Explanation:
We perform 4 pancake flips, with k values 4, 2, 4, and 3.
Starting state: arr = [3, 2, 4, 1]
After 1st flip (k = 4): arr = [1, 4, 2, 3]
After 2nd flip (k = 2): arr = [4, 1, 2, 3]
After 3rd flip (k = 4): arr = [3, 2, 1, 4]
After 4th flip (k = 3): arr = [1, 2, 3, 4], which is sorted.

Example 2:

Input: arr = [1,2,3]
Output: []
Explanation: The input is already sorted, so there is no need to flip anything.
Note that other answers, such as [3, 3], would also be accepted.

Approach 1: Pancake Sort Algorithm

```
def pancakeSort(arr):
    result = []
    n = len(arr)

    for size in range(n, 1, -1):
        # Find index of max in current window
        max_index = arr.index(max(arr[:size]))

        if max_index != size - 1:
            # Flip to bring max to front
            if max_index != 0:
                result.append(max_index + 1)
                arr[:max_index + 1] = arr[:max_index + 1][::-1]

            # Flip to put max in correct position
            result.append(size)
            arr[:size] = arr[:size][::-1]

    return result
```

Time Complexity: O(n²)
Space Complexity: O(1)

Approach 2: Optimized Pancake Sort

```
def pancakeSort(arr):
    result = []

    def flip(k):
        arr[:k] = arr[:k][::-1]
        result.append(k)

    for i in range(len(arr) - 1, 0, -1):
        # Find index of maximum element in unsorted part
        max_index = arr.index(max(arr[:i + 1]))

        if max_index != i:
            # If max is not at correct position
            if max_index != 0:
                flip(max_index + 1)
            flip(i + 1)

    return result
```

Time Complexity: O(n²)
Space Complexity: O(1)

Approach 3: Recursive Pancake Sort

```
def pancakeSort(arr):
    result = []

    def pancake_sort_recursive(n):
        if n == 1:
            return

        # Find index of max in first n elements
        max_index = arr.index(max(arr[:n]))

        if max_index != n - 1:
            if max_index != 0:
                result.append(max_index + 1)
                arr[:max_index + 1] = arr[:max_index + 1][::-1]

            result.append(n)
            arr[:n] = arr[:n][::-1]

        pancake_sort_recursive(n - 1)

    pancake_sort_recursive(len(arr))
    return result
```

Time Complexity: O(n²)
Space Complexity: O(n) due to recursion

56. Merge Intervals

Given an array of intervals where intervals[i] = [start$_i$, end$_i$], merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input*.

Example 1:

Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].

Example 2:

Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
Explanation: Intervals [1,4] and [4,5] are considered overlapping.

Example 3:

Input: intervals = [[4,7],[1,4]]
Output: [[1,7]]
Explanation: Intervals [1,4] and [4,7] are considered overlapping.

Approach 1: Sorting + Linear Scan

```
def merge(intervals):
    if not intervals:
        return []

    # Sort by start time
    intervals.sort(key=lambda x: x[0])

    merged = [intervals[0]]

    for interval in intervals[1:]:
        # If current interval overlaps with last merged
        if interval[0] <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], interval[1])
        else:
            merged.append(interval)

    return merged
```

Time Complexity: O(n log n)
Space Complexity: O(n)

Approach 2: Sorting with Graph Coloring

```
def merge(intervals):
    if not intervals:
        return []

    # Sort by start time
    intervals.sort(key=lambda x: x[0])

    n = len(intervals)
    merged = []
    i = 0

    while i < n:
        current_start = intervals[i][0]
        current_end = intervals[i][1]

        # Merge all overlapping intervals
        j = i + 1
        while j < n and intervals[j][0] <= current_end:
            current_end = max(current_end, intervals[j][1])
            j += 1

        merged.append([current_start, current_end])
        i = j

    return merged
```

Time Complexity: O(n log n)
Space Complexity: O(n)

Approach 3: Two-Pointer with In-place Modification

```
def merge(intervals):
    if not intervals:
        return []

    # Sort by start time
    intervals.sort(key=lambda x: x[0])

    i = 0
    while i < len(intervals) - 1:
        # If current interval overlaps with next
        if intervals[i][1] >= intervals[i + 1][0]:
            # Merge
            intervals[i][1] = max(intervals[i][1], intervals[i +
1][1])
            # Remove next interval
            intervals.pop(i + 1)
        else:
            i += 1

    return intervals
```

Time Complexity: O(n log n)
Space Complexity: O(1) excluding output

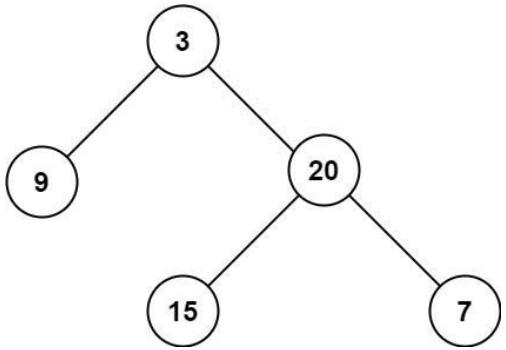
BFS / DFS

104. Maximum Depth of Binary Tree

Given the root of a binary tree, return *its maximum depth*.

A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: 3

Example 2:

Input: root = [1,null,2]
Output: 2

Approach 1: Recursive DFS

```
def maxDepth(root):
    if not root:
        return 0
    return 1 + max(maxDepth(root.left),
maxDepth(root.right))
```

Time Complexity: O(n)
Space Complexity: O(h) where h is height of tree

Approach 2: Iterative BFS (Level Order)

```
def maxDepth(root):
    if not root:
        return 0

    queue = deque([root])
    depth = 0

    while queue:
        level_size = len(queue)
        for _ in range(level_size):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        depth += 1

    return depth
```

Time Complexity: O(n)
Space Complexity: O(w) where w is max width of tree

Approach 3: Iterative DFS (Stack)

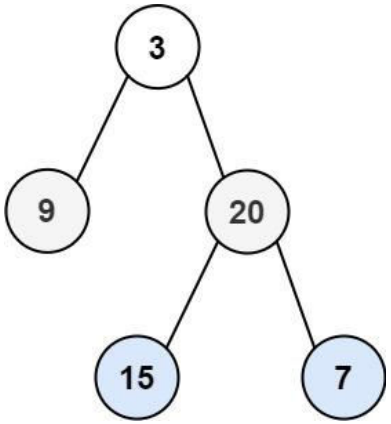
```
def maxDepth(root):
    if not root:
        return 0

    stack = [(root, 1)]
    max_depth = 0

    while stack:
        node, depth = stack.pop()
        max_depth = max(max_depth, depth)
        if node.left:
            stack.append((node.left, depth + 1))
        if node.right:
            stack.append((node.right, depth + 1))

    return max_depth
```

Time Complexity: O(n)
Space Complexity: O(h)

102. Binary Tree Level Order Traversal

Given the root of a binary tree, return *the level order traversal of its nodes' values*. (i.e., from left to right, level by level).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Approach 1: BFS with Queue

```
def levelOrder(root):
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)
        current_level = []

        for _ in range(level_size):
            node = queue.popleft()
            current_level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(current_level)

    return result
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: Recursive DFS with Level Tracking

```
def levelOrder(root):
    result = []

    def dfs(node, level):
        if not node:
            return

        if len(result) <= level:
            result.append([])

        result[level].append(node.val)
        dfs(node.left, level + 1)
        dfs(node.right, level + 1)

    dfs(root, 0)
    return result
```

Time Complexity: O(n)
Space Complexity: O(h)

Approach 3: BFS with Two Queues

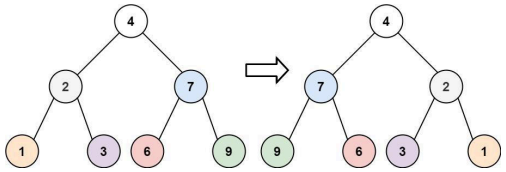
```
def levelOrder(root):
    if not root:
        return []

    result = []
    current_queue = deque([root])

    while current_queue:
        next_queue = deque()
        current_level = []

        while current_queue:
            node = current_queue.popleft()
            current_level.append(node.val)
            if node.left:
                next_queue.append(node.left)
            if node.right:
                next_queue.append(node.right)

        result.append(current_level)
        current_queue = next_queue

    return result
```

Time Complexity: O(n)
Space Complexity: O(n)

226. Invert Binary Tree

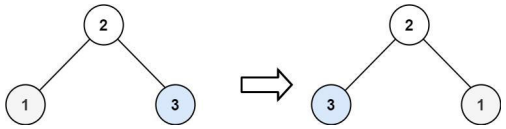
Given the root of a binary tree, invert the tree, and return *its root*.

Example 1:



Input: root = [4,2,7,1,3,6,9]
Output: [4,7,2,9,6,3,1]

Example 2:



Input: root = [2,1,3]
Output: [2,3,1]

Example 3:

Input: root = []
Output: []

Approach 1: Recursive DFS

```
def invertTree(root):
    if not root:
        return None

    # Swap children
    root.left, root.right = root.right, root.left

    # Recursively invert subtrees
    invertTree(root.left)
    invertTree(root.right)

    return root
```

Time Complexity: O(n)
Space Complexity: O(h)

Approach 2: Iterative BFS

```
def invertTree(root):
    if not root:
        return None

    queue = deque([root])

    while queue:
        node = queue.popleft()

        # Swap children
        node.left, node.right = node.right, node.left

        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)

    return root
```

Time Complexity: O(n)
Space Complexity: O(w)

Approach 3: Iterative DFS (Stack)

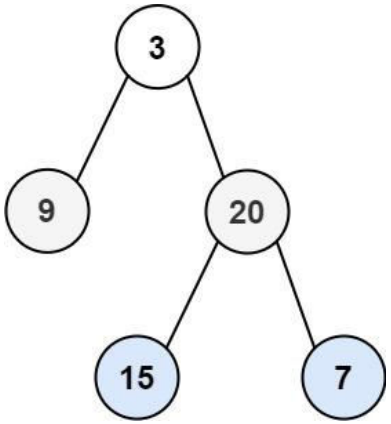
```
def invertTree(root):
    if not root:
        return None

    stack = [root]

    while stack:
        node = stack.pop()

        # Swap children
        node.left, node.right = node.right, node.left

        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)

    return root
```

Time Complexity: O(n)
Space Complexity: O(h)

103. Binary Tree Zigzag Level Order Traversal

Given the root of a binary tree, return *the zigzag level order traversal of its nodes' values.* (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[20,9],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Approach 1: BFS with Level Flag

```
def zigzagLevelOrder(root):
    if not root:
        return []

    result = []
    queue = deque([root])
    left_to_right = True

    while queue:
        level_size = len(queue)
        current_level = deque()

        for _ in range(level_size):
            node = queue.popleft()

            if left_to_right:
                current_level.append(node.val)
            else:
                current_level.appendleft(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(list(current_level))
        left_to_right = not left_to_right

    return result
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: Recursive DFS with Level Tracking

```
def zigzagLevelOrder(root):
    result = []

    def dfs(node, level):
        if not node:
            return

        if len(result) <= level:
            result.append([])

        if level % 2 == 0:
            result[level].append(node.val)
        else:
            result[level].insert(0, node.val)

        dfs(node.left, level + 1)
        dfs(node.right, level + 1)

    dfs(root, 0)
    return result
```

Time Complexity: O(n²) due to insert(0) operations
Space Complexity: O(h)

Approach 3: Two Stacks

```
def zigzagLevelOrder(root):
    if not root:
        return []

    result = []
    current_stack = [root]
    next_stack = []
    left_to_right = True

    while current_stack:
        current_level = []

        while current_stack:
            node = current_stack.pop()
            current_level.append(node.val)

            if left_to_right:
                if node.left:
                    next_stack.append(node.left)
                if node.right:
                    next_stack.append(node.right)
            else:
                if node.right:
                    next_stack.append(node.right)
                if node.left:
                    next_stack.append(node.left)

        result.append(current_level)
        current_stack, next_stack = next_stack,
current_stack
        left_to_right = not left_to_right

    return result
```

Time Complexity: O(n)
Space Complexity: O(n)

---

199. Binary Tree Right Side View

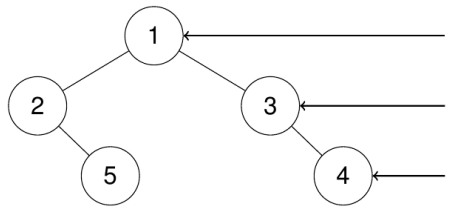
Given the root of a binary tree, imagine yourself standing on the **right side** of it, return *the values of the nodes you can see ordered from top to bottom.*

Example 1:

Input: root = [1,2,3,null,5,null,4]

Output: [1,3,4]

Explanation:

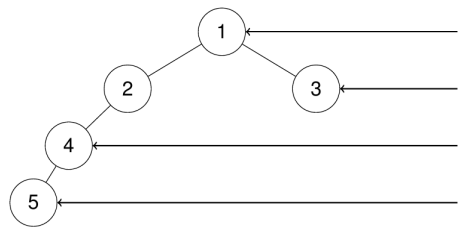


Example 2:

Input: root = [1,2,3,4,null,null,null,5]

Output: [1,3,4,5]

Explanation:



Example 3:

Input: root = [1,null,3]

Output: [1,3]

Example 4:

Input: root = []

Output: []

Approach 1: BFS Level Order

```
def rightSideView(root):
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)

        for i in range(level_size):
            node = queue.popleft()
            # Last node of each level is visible from right
            if i == level_size - 1:
                result.append(node.val)

            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

    return result
```

Time Complexity: O(n)
Space Complexity: O(w)

Approach 2: Recursive DFS with Level Tracking

```
def rightSideView(root):
    result = []

    def dfs(node, level):
        if not node:
            return

        # First time visiting this level, add node
        if len(result) == level:
            result.append(node.val)

        # Go right first, then left
        dfs(node.right, level + 1)
        dfs(node.left, level + 1)

    dfs(root, 0)
    return result
```

Time Complexity: O(n)
Space Complexity: O(h)

Approach 3: Iterative DFS with Stack

```
def rightSideView(root):
    if not root:
        return []

    result = []
    stack = [(root, 0)]
    level_seen = set()

    while stack:
        node, level = stack.pop()

        if level not in level_seen:
            result.append(node.val)
            level_seen.add(level)

        # Push left first, then right (so right gets processed
first)
        if node.left:
            stack.append((node.left, level + 1))
        if node.right:
            stack.append((node.right, level + 1))

    return result
```

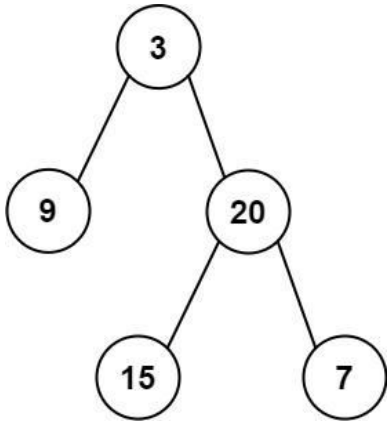
Time Complexity: O(n)
Space Complexity: O(h)

---

105. Construct Binary Tree from Preorder and Inorder

Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return *the binary tree*.

Example 1:



Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
Output: [3,9,20,null,null,15,7]

Example 2:

Input: preorder = [-1], inorder = [-1]
Output: [-1]

Approach 1: Recursive with Array Slicing

```
def buildTree(preorder, inorder):
    if not preorder or not inorder:
        return None

    root_val = preorder[0]
    root = TreeNode(root_val)

    # Find root index in inorder
    root_index = inorder.index(root_val)

    # Recursively build left and right subtrees
    root.left = buildTree(preorder[1:root_index + 1],
inorder[:root_index])
    root.right = buildTree(preorder[root_index + 1:],
inorder[root_index + 1:])

    return root
```

Time Complexity: O(n²) due to index() and slicing
Space Complexity: O(n²) due to slicing

Approach 2: Recursive with HashMap (Optimized)

```
def buildTree(preorder, inorder):
    inorder_map = {val: idx for idx, val in
enumerate(inorder)}

    def build(pre_start, pre_end, in_start, in_end):
        if pre_start > pre_end or in_start > in_end:
            return None

        root_val = preorder[pre_start]
        root = TreeNode(root_val)
        root_idx = inorder_map[root_val]
        left_size = root_idx - in_start

        root.left = build(pre_start + 1, pre_start + left_size, in_start, root_idx - 1)
        root.right = build(pre_start + left_size + 1, pre_end, root_idx + 1, in_end)

        return root

    return build(0, len(preorder) - 1, 0, len(inorder) - 1)
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 3: Iterative with Stack

```
def buildTree(preorder, inorder):
    if not preorder:
        return None

    root = TreeNode(preorder[0])
    stack = [root]
    inorder_index = 0

    for i in range(1, len(preorder)):
        preorder_val = preorder[i]
        node = stack[-1]

        if node.val != inorder[inorder_index]:
            # Build left child
            node.left = TreeNode(preorder_val)
            stack.append(node.left)
        else:
            # Pop stack until we find correct parent
            while stack and stack[-1].val == inorder[inorder_index]:
                node = stack.pop()
                inorder_index += 1
            # Build right child
            node.right = TreeNode(preorder_val)
            stack.append(node.right)

    return root
```

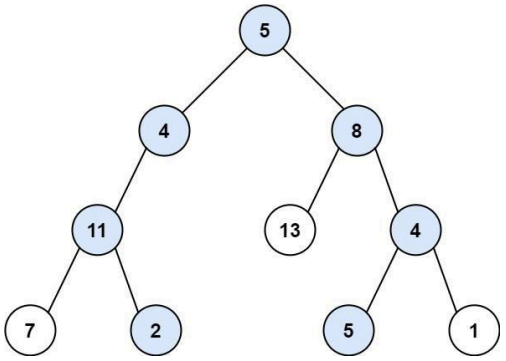
Time Complexity: O(n)
Space Complexity: O(h)

113. Path Sum II

Given the root of a binary tree and an integer targetSum, return *all **root-to-leaf** paths where the sum of the node values in the path equals* targetSum. *Each path should be returned as a list of the node **values**, not node references.*

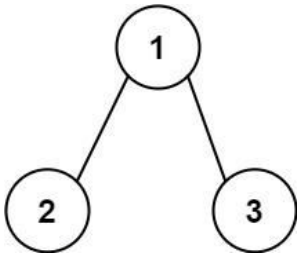
A **root-to-leaf** path is a path starting from the root and ending at any leaf node. A **leaf** is a node with no children.

Example 1:



Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22
Output: [[5,4,11,2],[5,8,4,5]]
Explanation: There are two paths whose sum equals targetSum:
5 + 4 + 11 + 2 = 22
5 + 8 + 4 + 5 = 22

Example 2:



Input: root = [1,2,3], targetSum = 5
Output: []

Example 3:

Input: root = [1,2], targetSum = 0
Output: []

Approach 1: Recursive DFS with Backtracking

```
def pathSum(root, targetSum):
    result = []

    def dfs(node, remaining, path):
        if not node:
            return

        path.append(node.val)
        remaining -= node.val

        # Check if leaf and sum matches
        if not node.left and not node.right and remaining ==
0:
            result.append(path[:])

        dfs(node.left, remaining, path)
        dfs(node.right, remaining, path)

        # Backtrack
        path.pop()

    dfs(root, targetSum, [])
    return result
```

Time Complexity: O(n²) in worst case (copying paths)
Space Complexity: O(h) for recursion + O(n²) for storing paths

---

Approach 2: Recursive with Path Copy

```
def pathSum(root, targetSum):
    result = []

    def dfs(node, remaining, path):
        if not node:
            return

        new_path = path + [node.val]
        remaining -= node.val

        if not node.left and not node.right and remaining == 0:
            result.append(new_path)

        dfs(node.left, remaining, new_path)
        dfs(node.right, remaining, new_path)

    dfs(root, targetSum, [])
    return result
```

Time Complexity: O(n²)
Space Complexity: O(n²)

Approach 3: Iterative DFS with Stack

```
def pathSum(root, targetSum):
    if not root:
        return []

    result = []
    stack = [(root, targetSum - root.val, [root.val])]

    while stack:
        node, remaining, path = stack.pop()

        if not node.left and not node.right and remaining == 0:
            result.append(path)

        if node.right:
            stack.append((node.right, remaining -
node.right.val, path + [node.right.val]))
        if node.left:
            stack.append((node.left, remaining - node.left.val,
path + [node.left.val]))

    return result
```

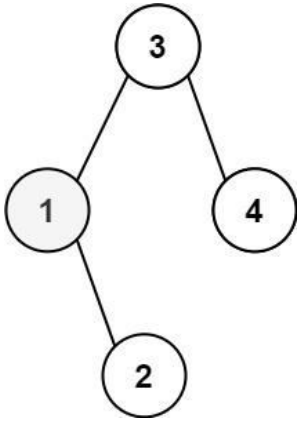
Time Complexity: O(n²)
Space Complexity: O(n²)

---

230. Kth Smallest Element in a BST

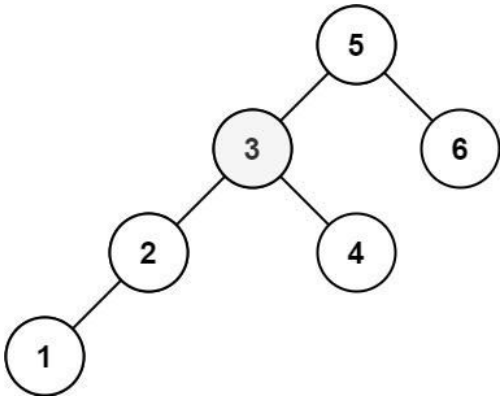
Given the root of a binary search tree, and an integer k, return *the* k_{th} *smallest value (**1-indexed**) of all the values of the nodes in the tree.*

Example 1:



Input: root = [3,1,4,null,2], k = 1
Output: 1

Example 2:



Input: root = [5,3,6,2,4,null,null,1], k = 3
Output: 3

Approach 1: Inorder Traversal (Recursive)

```
def kthSmallest(root, k):
    def inorder(node):
        if not node:
            return []
        return inorder(node.left) + [node.val] +
inorder(node.right)

    return inorder(root)[k - 1]
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: Iterative Inorder (Stack)

```
def kthSmallest(root, k):
    stack = []
    curr = root

    while stack or curr:
        # Go to leftmost node
        while curr:
            stack.append(curr)
            curr = curr.left

        curr = stack.pop()
        k -= 1
        if k == 0:
            return curr.val

        curr = curr.right

    return -1
```

Time Complexity: O(h + k) where h is height
Space Complexity: O(h)

Approach 3: Binary Search on BST Property

```
def kthSmallest(root, k):
    def count_nodes(node):
        if not node:
            return 0
        return 1 + count_nodes(node.left) +
count_nodes(node.right)

    left_count = count_nodes(root.left)

    if k <= left_count:
        return kthSmallest(root.left, k)
    elif k == left_count + 1:
        return root.val
    else:
        return kthSmallest(root.right, k - left_count - 1)
```

Time Complexity: O(n * h) worst case
Space Complexity: O(h)

---

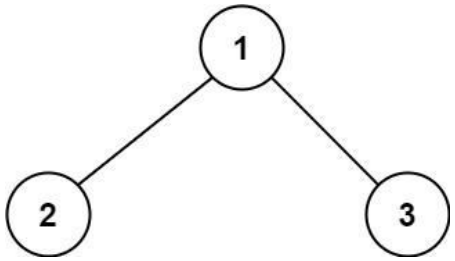
124. Binary Tree Maximum Path Sum

A **path** in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence **at most once**. Note that the path does not need to pass through the root.

The **path sum** of a path is the sum of the node's values in the path.

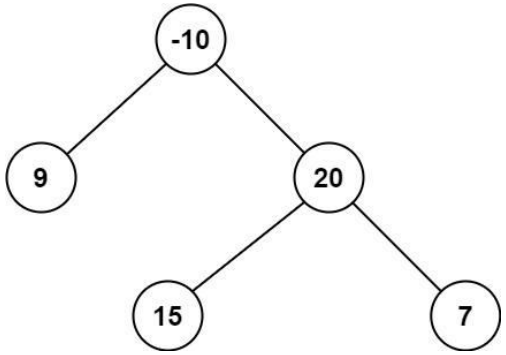
Given the root of a binary tree, return *the maximum **path sum** of any **non-empty** path.*

Example 1:



Input: root = [1,2,3]
Output: 6
Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of 2 + 1 + 3 = 6.

Example 2:



Input: root = [-10,9,20,null,null,15,7]
Output: 42
Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of 15 + 20 + 7 = 42.

Approach 1: Recursive DFS with Global Variable

```
def maxPathSum(root):
    max_sum = float('-inf')

    def max_gain(node):
        nonlocal max_sum
        if not node:
            return 0

        # Max sum from left and right (only take positive gains)
        left_gain = max(max_gain(node.left), 0)
        right_gain = max(max_gain(node.right), 0)

        # Path through current node
        current_path_sum = node.val + left_gain + right_gain
        max_sum = max(max_sum, current_path_sum)

        # Return max gain if continuing through parent
        return node.val + max(left_gain, right_gain)

    max_gain(root)
    return max_sum
```

Time Complexity: O(n)
Space Complexity: O(h)

Approach 2: Recursive with Result Array

```
def maxPathSum(root):
    result = [float('-inf')]

    def dfs(node):
        if not node:
            return 0

        left = max(dfs(node.left), 0)
        right = max(dfs(node.right), 0)

        result[0] = max(result[0], node.val + left + right)

        return node.val + max(left, right)

    dfs(root)
    return result[0]
```

Time Complexity: O(n)
Space Complexity: O(h)

Approach 3: Iterative Post-order with Stack

```
def maxPathSum(root):
    if not root:
        return 0

    max_sum = float('-inf')
    stack = [(root, False)]
    gains = {}

    while stack:
        node, visited = stack.pop()

        if not node:
            continue

        if not visited:
            stack.append((node, True))
            stack.append((node.right, False))
            stack.append((node.left, False))
        else:
            # Process node
            left_gain = max(gains.get(node.left, 0), 0)
            right_gain = max(gains.get(node.right, 0), 0)

            # Update max_sum
            max_sum = max(max_sum, node.val + left_gain + right_gain)

            # Store gain for parent
            gains[node] = node.val + max(left_gain, right_gain)

    return max_sum
```

Time Complexity: O(n)
Space Complexity: O(n)

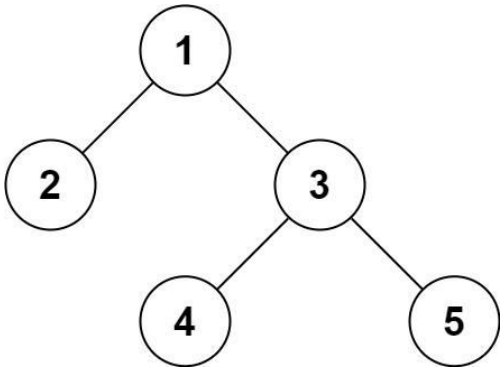
297. Serialize and Deserialize Binary Tree

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as [how LeetCode serializes a binary tree](). You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:



Input: root = [1,2,3,null,null,4,5]
Output: [1,2,3,null,null,4,5]

Example 2:

Input: root = []
Output: []

Approach 1: BFS Level Order (with Nulls)

```
class Codec:
    def serialize(self, root):
        if not root:
            return ""

        result = []
        queue = deque([root])

        while queue:
            node = queue.popleft()
            if node:
                result.append(str(node.val))
                queue.append(node.left)
                queue.append(node.right)
            else:
                result.append("null")

        return ",".join(result)

    def deserialize(self, data):
        if not data:
            return None

        values = data.split(",")
        root = TreeNode(int(values[0]))
        queue = deque([root])
        i = 1

        while queue and i < len(values):
            node = queue.popleft()

            # Left child
            if values[i] != "null":
                node.left = TreeNode(int(values[i]))
                queue.append(node.left)
            i += 1

            # Right child
            if i < len(values) and values[i] != "null":
                node.right = TreeNode(int(values[i]))
                queue.append(node.right)
            i += 1

        return root
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: DFS Preorder (with Nulls)

```
class Codec:
    def serialize(self, root):
        def dfs(node):
            if not node:
                return "null"
            return str(node.val) + "," + dfs(node.left) + "," +
dfs(node.right)

        return dfs(root)

    def deserialize(self, data):
        def dfs():
            val = next(values)
            if val == "null":
                return None
            node = TreeNode(int(val))
            node.left = dfs()
            node.right = dfs()
            return node

        values = iter(data.split(","))
        return dfs()
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 3: Parenthesis Representation

```
class Codec:
    def serialize(self, root):
        if not root:
            return ""

        # Format: val(left)(right)
        result = str(root.val)
        if root.left or root.right:
            result += "(" + self.serialize(root.left) + ")"
        if root.right:
            result += "(" + self.serialize(root.right) + ")"
        return result

    def deserialize(self, data):
        if not data:
            return None

        def parse(s, start):
            # Parse value
            end = start
            while end < len(s) and s[end].isdigit():
                end += 1

            node = TreeNode(int(s[start:end]))

            # Parse left subtree
            if end < len(s) and s[end] == "(":
                left_end, left_node = parse(s, end + 1)
                node.left = left_node
                end = left_end + 1

            # Parse right subtree
            if end < len(s) and s[end] == "(":
                right_end, right_node = parse(s, end + 1)
                node.right = right_node
                end = right_end + 1

            return end, node

        _, root = parse(data, 0)
        return root
```

Time Complexity: O(n)
Space Complexity: O(n)

STACK

20. Valid Parentheses

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: s = "()"

Output: true

Example 2:

Input: s = "()[]{}"

Output: true

Example 3:

Input: s = "(]"

Output: false

Example 4:

Input: s = "([])"

Output: true

Example 5:

Input: s = "([)]"

Output: false

Approach 1: Stack with Mapping

```
def isValid(s):
    stack = []
    mapping = {")": "(", "}": "{", "]": "["}

    for char in s:
        if char in mapping:
            if not stack or stack[-1] != mapping[char]:
                return False
            stack.pop()
        else:
            stack.append(char)

    return len(stack) == 0
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: Stack with Direct Comparison

```
def isValid(s):
    stack = []

    for char in s:
        if char == '(' or char == '{' or char == '[':
            stack.append(char)
        else:
            if not stack:
                return False
            if char == ')' and stack[-1] != '(':
                return False
            if char == '}' and stack[-1] != '{':
                return False
            if char == ']' and stack[-1] != '[':
                return False
            stack.pop()

    return len(stack) == 0
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 3: Replace Method (Not Efficient)

```
def isValid(s):
    while '()' in s or '{}' in s or '[]' in s:
        s = s.replace('()', '')
        s = s.replace('{}', '')
        s = s.replace('[]', '')

    return s == ''
```

Time Complexity: O(n²)
Space Complexity: O(n)

1047. Remove All Adjacent Duplicates In String

You are given a string s consisting of lowercase English letters. A **duplicate removal** consists of choosing two **adjacent** and **equal** letters and removing them.

We repeatedly make **duplicate removals** on s until we no longer can.

Return *the final string after all such duplicate removals have been made*. It can be proven that the answer is **unique**.

Example 1:

Input: s = "abbaca"
Output: "ca"
Explanation:
For example, in "abbaca" we could remove "bb" since the letters are adjacent and equal, and this is the only possible move. The result of this move is that the string is "aaca", of which only "aa" is possible, so the final string is "ca".

Example 2:

Input: s = "azxxzy"
Output: "ay"

Approach 1: Stack

```
def removeDuplicates(s):
    stack = []

    for char in s:
        if stack and stack[-1] == char:
            stack.pop()
        else:
            stack.append(char)

    return ''.join(stack)
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: Two Pointers (In-place)

```
def removeDuplicates(s):
    s = list(s)
    write_ptr = 0

    for read_ptr in range(len(s)):
        s[write_ptr] = s[read_ptr]

        if write_ptr > 0 and s[write_ptr] == s[write_ptr - 1]:
            write_ptr -= 1
        else:
            write_ptr += 1

    return ''.join(s[:write_ptr])
```

Time Complexity: O(n)
Space Complexity: O(n) for list conversion

Approach 3: Recursive

```
def removeDuplicates(s):
    for i in range(len(s) - 1):
        if s[i] == s[i + 1]:
            return removeDuplicates(s[:i] + s[i+2:])
    return s
```

Time Complexity: O(n²)
Space Complexity: O(n) for recursion stack

155. Min Stack

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

- MinStack() initializes the stack object.
- void push(int val) pushes the element val onto the stack.
- void pop() removes the element on the top of the stack.
- int top() gets the top element of the stack.
- int getMin() retrieves the minimum element in the stack.

You must implement a solution with O(1) time complexity for each function.

Example 1:

Input
["MinStack","push","push","push","getMin","pop","top","getMin"]
[[],[-2],[0],[-3],[],[],[],[]]

Output
[null,null,null,null,-3,null,0,-2]

Explanation
MinStack minStack = new MinStack();
minStack.push(-2);
minStack.push(0);
minStack.push(-3);
minStack.getMin(); // return -3
minStack.pop();
minStack.top(); // return 0
minStack.getMin(); // return -2

Approach 1: Two Stacks

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val):
        self.stack.append(val)
        if not self.min_stack or val <= self.min_stack[-1]:
            self.min_stack.append(val)

    def pop(self):
        if self.stack:
            val = self.stack.pop()
            if val == self.min_stack[-1]:
                self.min_stack.pop()

    def top(self):
        return self.stack[-1] if self.stack else None

    def getMin(self):
        return self.min_stack[-1] if self.min_stack else None
```

Time Complexity: O(1) for all operations
Space Complexity: O(n)

Approach 2: Single Stack with Tuple

```
class MinStack:
    def __init__(self):
        self.stack = []

    def push(self, val):
        current_min = val
        if self.stack:
            current_min = min(val, self.stack[-1][1])
        self.stack.append((val, current_min))

    def pop(self):
        if self.stack:
            self.stack.pop()

    def top(self):
        return self.stack[-1][0] if self.stack else None

    def getMin(self):
        return self.stack[-1][1] if self.stack else None
```

Time Complexity: O(1) for all operations
Space Complexity: O(n)

Approach 3: Single Stack with Min Tracking

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_val = float('inf')

    def push(self, val):
        if val <= self.min_val:
            self.stack.append(self.min_val)
            self.min_val = val
        self.stack.append(val)

    def pop(self):
        if self.stack:
            val = self.stack.pop()
            if val == self.min_val and self.stack:
                self.min_val = self.stack.pop()

    def top(self):
        return self.stack[-1] if self.stack else None

    def getMin(self):
        return self.min_val if self.min_val != float('inf') else None
```

Time Complexity: O(1) for all operations
Space Complexity: O(n)

---

150. Evaluate Reverse Polish Notation

You are given an array of strings tokens that represents an arithmetic expression in a Reverse Polish Notation.

Evaluate the expression. Return *an integer that represents the value of the expression*.

Note that:

- The valid operators are '+', '-', '*', and '/'.
- Each operand may be an integer or another expression.
- The division between two integers always **truncates toward zero**.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.
- The answer and all the intermediate calculations can be represented in a **32-bit** integer.

Example 1:

Input: tokens = ["2","1","+","3","*"]
Output: 9
Explanation: ((2 + 1) * 3) = 9

Example 2:

Input: tokens = ["4","13","5","/","+"]
Output: 6
Explanation: (4 + (13 / 5)) = 6

Example 3:

Input: tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"]
Output: 22
Explanation: ((10 * (6 / ((9 + 3) * -11))) + 17) + 5
= ((10 * (6 / (12 * -11))) + 17) + 5
= ((10 * (6 / -132)) + 17) + 5
= ((10 * 0) + 17) + 5
= (0 + 17) + 5
= 17 + 5
= 22

Approach 1: Stack with Operations

```
def evalRPN(tokens):
    stack = []
    operations = {
        '+': lambda a, b: a + b,
        '-': lambda a, b: a - b,
        '*': lambda a, b: a * b,
        '/': lambda a, b: int(a / b)  # Truncate toward zero
    }

    for token in tokens:
        if token in operations:
            b = stack.pop()
            a = stack.pop()
            result = operations[token](a, b)
            stack.append(result)
        else:
            stack.append(int(token))

    return stack[0]
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: Stack with If-Else

```
def evalRPN(tokens):
    stack = []

    for token in tokens:
        if token == '+':
            stack.append(stack.pop() + stack.pop())
        elif token == '-':
            b = stack.pop()
            a = stack.pop()
            stack.append(a - b)
        elif token == '*':
            stack.append(stack.pop() * stack.pop())
        elif token == '/':
            b = stack.pop()
            a = stack.pop()
            stack.append(int(a / b))
        else:
            stack.append(int(token))

    return stack[0]
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 3: Recursive

```
def evalRPN(tokens):
    def evaluate(idx):
        token = tokens[idx]
        if token in {'+', '-', '*', '/'}:
            right, right_idx = evaluate(idx - 1)
            left, left_idx = evaluate(right_idx - 1)

            if token == '+':
                return left + right, left_idx
            elif token == '-':
                return left - right, left_idx
            elif token == '*':
                return left * right, left_idx
            else:
                return int(left / right), left_idx
        else:
            return int(token), idx

    return evaluate(len(tokens) - 1)[0]
```

Time Complexity: O(n)
Space Complexity: O(n) for recursion stack

---

739. Daily Temperatures

Given an array of integers temperatures represents the daily temperatures, return *an array* answer *such that* answer[i] *is the number of days you have to wait after the* i_{th} *day to get a warmer temperature*. If there is no future day for which this is possible, keep answer[i] == 0 instead.

Example 1:

Input: temperatures = [73,74,75,71,69,72,76,73]
Output: [1,1,4,2,1,1,0,0]

Example 2:

Input: temperatures = [30,40,50,60]
Output: [1,1,1,0]

Example 3:

Input: temperatures = [30,60,90]
Output: [1,1,0]

Approach 1: Monotonic Stack

```
def dailyTemperatures(temperatures):
    n = len(temperatures)
    result = [0] * n
    stack = []  # Store indices

    for i in range(n):
        while stack and temperatures[i] >
temperatures[stack[-1]]:
            prev_idx = stack.pop()
            result[prev_idx] = i - prev_idx
        stack.append(i)

    return result
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: Next Array (Optimized)

```
def dailyTemperatures(temperatures):
    n = len(temperatures)
    result = [0] * n
    hottest = 0

    for i in range(n - 1, -1, -1):
        if temperatures[i] >= hottest:
            hottest = temperatures[i]
            continue

        days = 1
        while temperatures[i + days] <= temperatures[i]:
            days += result[i + days]
        result[i] = days

    return result
```

Time Complexity: O(n)
Space Complexity: O(1) excluding output

Approach 3: Brute Force

```
def dailyTemperatures(temperatures):
    n = len(temperatures)
    result = [0] * n

    for i in range(n):
        for j in range(i + 1, n):
            if temperatures[j] > temperatures[i]:
                result[i] = j - i
                break

    return result
```

Time Complexity: O(n²)
Space Complexity: O(1) excluding output

---

503. Next Greater Element II

Given a circular integer array nums (i.e., the next element of nums[nums.length - 1] is nums[0]), return *the **next greater number** for every element in* nums.

The **next greater number** of a number x is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return -1 for this number.

Example 1:

Input: nums = [1,2,1]
Output: [2,-1,2]
Explanation: The first 1's next greater number is 2;
The number 2 can't find next greater number.
The second 1's next greater number needs to search circularly, which is also 2.

Example 2:

Input: nums = [1,2,3,4,3]
Output: [2,3,4,-1,4]

Approach 1: Stack with Double Array

```
def nextGreaterElements(nums):
    n = len(nums)
    result = [-1] * n
    stack = []

    # Iterate twice to handle circular array
    for i in range(2 * n):
        idx = i % n
        while stack and nums[stack[-1]] < nums[idx]:
            result[stack.pop()] = nums[idx]
        if i < n:
            stack.append(idx)

    return result
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: Stack with Index Tracking

```
def nextGreaterElements(nums):
    n = len(nums)
    result = [-1] * n
    stack = []

    for i in range(2 * n - 1, -1, -1):
        idx = i % n
        while stack and stack[-1] <= nums[idx]:
            stack.pop()
        if stack:
            result[idx] = stack[-1]
        stack.append(nums[idx])

    return result
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 3: Brute Force with Circular Check

```
def nextGreaterElements(nums):
    n = len(nums)
    result = [-1] * n

    for i in range(n):
        for j in range(1, n):
            next_idx = (i + j) % n
            if nums[next_idx] > nums[i]:
                result[i] = nums[next_idx]
                break

    return result
```

Time Complexity: O(n²)
Space Complexity: O(1) excluding output

394. Decode String

Given an encoded string, return its decoded string.

The encoding rule is: k[encoded_string], where the encoded_string inside the square brackets is being repeated exactly k times. Note that k is guaranteed to be a positive integer.

You may assume that the input string is always valid; there are no extra white spaces, square brackets are well-formed, etc. Furthermore, you may assume that the original data does not contain any digits and that digits are only for those repeat numbers, k. For example, there will not be input like 3a or 2[4].

The test cases are generated so that the length of the output will never exceed 10_5.

Example 1:

Input: s = "3[a]2[bc]"
Output: "aaabcbc"

Example 2:

Input: s = "3[a2[c]]"
Output: "accaccacc"

Example 3:

Input: s = "2[abc]3[cd]ef"
Output: "abcabccdcdcdef"

Approach 1: Two Stacks

```
def decodeString(s):
    num_stack = []
    str_stack = []
    current_num = 0
    current_str = ''

    for char in s:
        if char.isdigit():
            current_num = current_num * 10 + int(char)
        elif char == '[':
            num_stack.append(current_num)
            str_stack.append(current_str)
            current_num = 0
            current_str = ''
        elif char == ']':
            num = num_stack.pop()
            prev_str = str_stack.pop()
            current_str = prev_str + current_str * num
        else:
            current_str += char

    return current_str
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: Recursive

```
def decodeString(s):
    def decode(i):
        result = ''
        num = 0

        while i < len(s):
            if s[i].isdigit():
                num = num * 10 + int(s[i])
            elif s[i] == '[':
                i, decoded = decode(i + 1)
                result += decoded * num
                num = 0
            elif s[i] == ']':
                return i, result
            else:
                result += s[i]
            i += 1

        return i, result

    return decode(0)[1]
```

Time Complexity: O(n)
Space Complexity: O(n) for recursion stack

Approach 3: Single Stack

```
def decodeString(s):
    stack = []

    for char in s:
        if char != ']':
            stack.append(char)
        else:
            # Get the encoded string
            encoded_str = ''
            while stack and stack[-1] != '[':
                encoded_str = stack.pop() + encoded_str
            stack.pop()  # Remove '['

            # Get the number
            num = ''
            while stack and stack[-1].isdigit():
                num = stack.pop() + num

            # Decode and push back
            stack.append(encoded_str * int(num))

    return ''.join(stack)
```

Time Complexity: O(n)
Space Complexity: O(n)

---

901. Online Stock Span

Design an algorithm that collects daily price quotes for some stock and returns **the span** of that stock's price for the current day.

The **span** of the stock's price in one day is the maximum number of consecutive days (starting from that day and going backward) for which the stock price was less than or equal to the price of that day.

- For example, if the prices of the stock in the last four days is [7,2,1,2] and the price of the stock today is 2, then the span of today is 4 because starting from today, the price of the stock was less than or equal 2 for 4 consecutive days.
- Also, if the prices of the stock in the last four days is [7,34,1,2] and the price of the stock today is 8, then the span of today is 3 because starting from today, the price of the stock was less than or equal 8 for 3 consecutive days.

Implement the StockSpanner class:

- StockSpanner() Initializes the object of the class.
- int next(int price) Returns the **span** of the stock's price given that today's price is price.

Example 1:

Input
["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]
[[], [100], [80], [60], [70], [60], [75], [85]]
Output
[null, 1, 1, 1, 2, 1, 4, 6]

Explanation
StockSpanner stockSpanner = new StockSpanner();
stockSpanner.next(100); // return 1
stockSpanner.next(80); // return 1
stockSpanner.next(60); // return 1
stockSpanner.next(70); // return 2
stockSpanner.next(60); // return 1
stockSpanner.next(75); // return 4, because the last 4 prices (including today's price of 75) were less than or equal to today's price.
stockSpanner.next(85); // return 6

Approach 1: Monotonic Stack

```
class StockSpanner:
    def __init__(self):
        self.stack = []  # (price, span)

    def next(self, price):
        span = 1
        while self.stack and self.stack[-1][0] <= price:
            span += self.stack.pop()[1]
        self.stack.append((price, span))
        return span
```

Time Complexity: O(1) amortized
Space Complexity: O(n)

Approach 2: Two Stacks

```
class StockSpanner:
    def __init__(self):
        self.prices = []
        self.spans = []

    def next(self, price):
        span = 1
        i = len(self.prices) - 1

        while i >= 0 and self.prices[i] <= price:
            span += self.spans[i]
            i -= self.spans[i]

        self.prices.append(price)
        self.spans.append(span)
        return span
```

Time Complexity: O(1) amortized
Space Complexity: O(n)

Approach 3: Brute Force

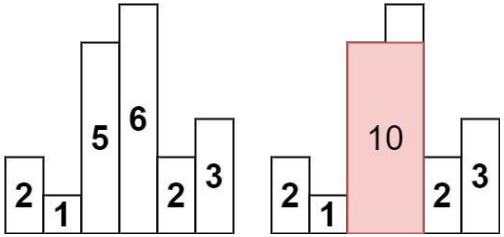
```
class StockSpanner:
    def __init__(self):
        self.prices = []

    def next(self, price):
        self.prices.append(price)
        span = 1

        for i in range(len(self.prices) - 2, -1, -1):
            if self.prices[i] <= price:
                span += 1
            else:
                break

        return span
```

Time Complexity: O(n) worst case
Space Complexity: O(n)

84. Largest Rectangle in Histogram

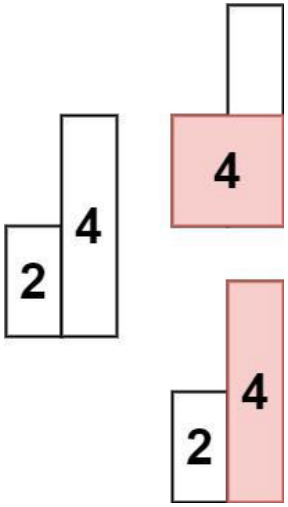
Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return *the area of the largest rectangle in the histogram*.

Example 1:



Input: heights = [2,1,5,6,2,3]
Output: 10
Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 units.

Example 2:



Input: heights = [2,4]
Output: 4

Approach 1: Stack (Optimized)

```
def largestRectangleArea(heights):
    stack = []
    max_area = 0
    heights.append(0)  # Sentinel

    for i in range(len(heights)):
        while stack and heights[stack[-1]] > heights[i]:
            h = heights[stack.pop()]
            w = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, h * w)
        stack.append(i)

    heights.pop()  # Remove sentinel
    return max_area
```

Time Complexity: O(n)
Space Complexity: O(n)

Approach 2: Divide and Conquer

```
def largestRectangleArea(heights):
    def calculate_area(left, right):
        if left > right:
            return 0

        # Find minimum height index
        min_idx = left
        for i in range(left, right + 1):
            if heights[i] < heights[min_idx]:
                min_idx = i

        current_area = heights[min_idx] * (right - left + 1)
        left_area = calculate_area(left, min_idx - 1)
        right_area = calculate_area(min_idx + 1, right)

        return max(current_area, left_area, right_area)

    return calculate_area(0, len(heights) - 1)
```

Time Complexity: O(n log n) average, O(n²) worst
Space Complexity: O(log n) for recursion

Approach 3: Brute Force

```
def largestRectangleArea(heights):
    max_area = 0
    n = len(heights)

    for i in range(n):
        min_height = heights[i]
        for j in range(i, n):
            min_height = min(min_height, heights[j])
            width = j - i + 1
            max_area = max(max_area, min_height * width)

    return max_area
```

Time Complexity: O(n²)
Space Complexity: O(1)

85. Maximal Rectangle

Given a rows x cols binary matrix filled with 0's and 1's, find the largest rectangle containing only 1's and return *its area*.

Example 1:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

Input: matrix = [["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]
Output: 6
Explanation: The maximal rectangle is shown in the above picture.

Example 2:

Input: matrix = [["0"]]
Output: 0

Example 3:

Input: matrix = [["1"]]
Output: 1

Approach 1: Histogram Method (Using 84)

```
def maximalRectangle(matrix):
    if not matrix or not matrix[0]:
        return 0

    def largestRectangleArea(heights):
        stack = []
        max_area = 0
        heights.append(0)

        for i in range(len(heights)):
            while stack and heights[stack[-1]] > heights[i]:
                h = heights[stack.pop()]
                w = i if not stack else i - stack[-1] - 1
                max_area = max(max_area, h * w)
            stack.append(i)

        heights.pop()
        return max_area

    rows, cols = len(matrix), len(matrix[0])
    heights = [0] * cols
    max_area = 0

    for i in range(rows):
        for j in range(cols):
            if matrix[i][j] == '1':
                heights[j] += 1
            else:
                heights[j] = 0

        max_area = max(max_area,
largestRectangleArea(heights))

    return max_area
```

Time Complexity: O(rows * cols)
Space Complexity: O(cols)

Approach 2: DP with Left and Right Arrays

```python
def maximalRectangle(matrix):
    if not matrix or not matrix[0]:
        return 0

    rows, cols = len(matrix), len(matrix[0])
    heights = [0] * cols
    left = [0] * cols
    right = [cols] * cols
    max_area = 0

    for i in range(rows):
        # Update heights
        for j in range(cols):
            if matrix[i][j] == '1':
                heights[j] += 1
            else:
                heights[j] = 0

        # Update left boundaries
        current_left = 0
        for j in range(cols):
            if matrix[i][j] == '1':
                left[j] = max(left[j], current_left)
            else:
                left[j] = 0
                current_left = j + 1

        # Update right boundaries
        current_right = cols
        for j in range(cols - 1, -1, -1):
            if matrix[i][j] == '1':
                right[j] = min(right[j], current_right)
            else:
                right[j] = cols
                current_right = j

        # Calculate area
        for j in range(cols):
            max_area = max(max_area, heights[j] * (right[j] - left[j]))

    return max_area
```

Time Complexity: O(rows * cols)
Space Complexity: O(cols)

Approach 3: Brute Force

```python
def maximalRectangle(matrix):
    if not matrix or not matrix[0]:
        return 0

    rows, cols = len(matrix), len(matrix[0])
    max_area = 0

    for top in range(rows):
        for bottom in range(top, rows):
            # Check if column has all 1's from top to bottom
            min_width = float('inf')
            for col in range(cols):
                # Check vertical line
                valid = True
                for row in range(top, bottom + 1):
                    if matrix[row][col] == '0':
                        valid = False
                        break

                if valid:
                    # Find width
                    width = 1
                    while col + width < cols:
                        valid_width = True
                        for row in range(top, bottom + 1):
                            if matrix[row][col + width] == '0':
                                valid_width = False
                                break
                        if valid_width:
                            width += 1
                        else:
                            break
                    max_area = max(max_area, (bottom - top + 1) * width)

    return max_area
```

Time Complexity: $O(rows^3 * cols^2)$
Space Complexity: O(1)